

CSCI 598H

Applied Computer Vision for Engineering Systems

Lecture 7-1: Motion & Video Understanding

Dr. Minghan Li

Assistant Professor

Computer Science Department

Colorado School of Mines

Oct/06/2026

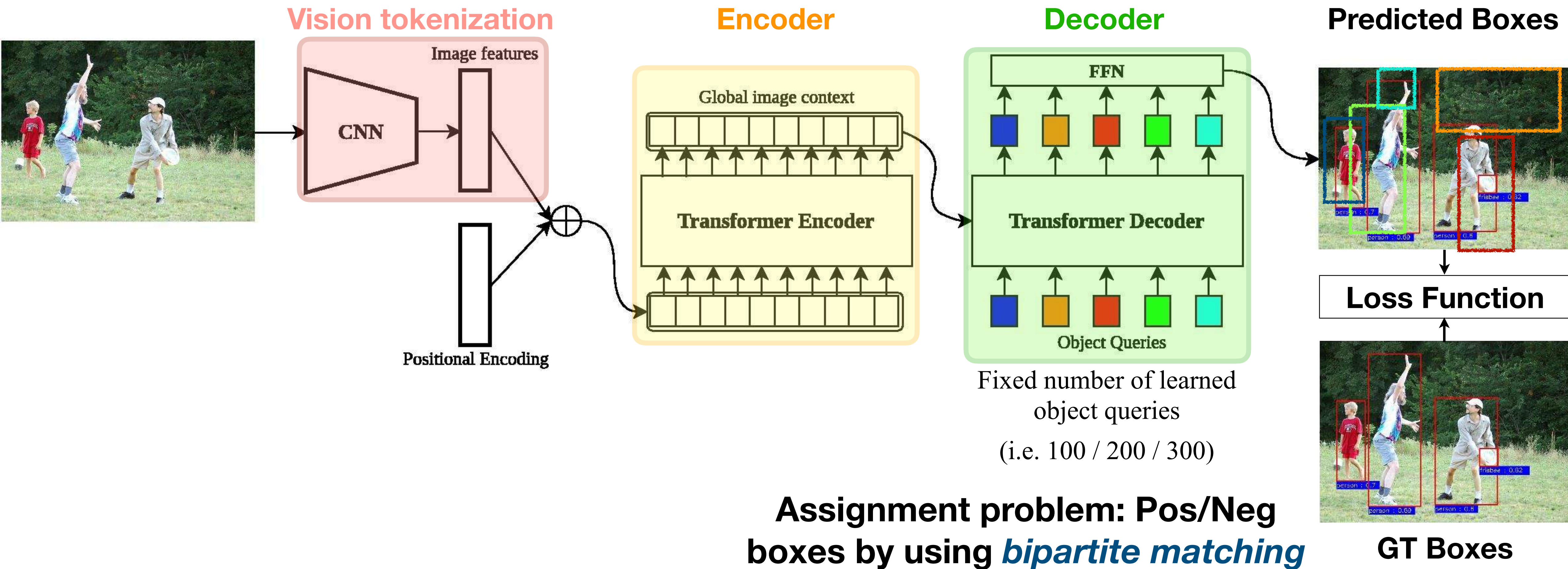


Administration

- Grading of Assignment 1 will be released next week.

Recap

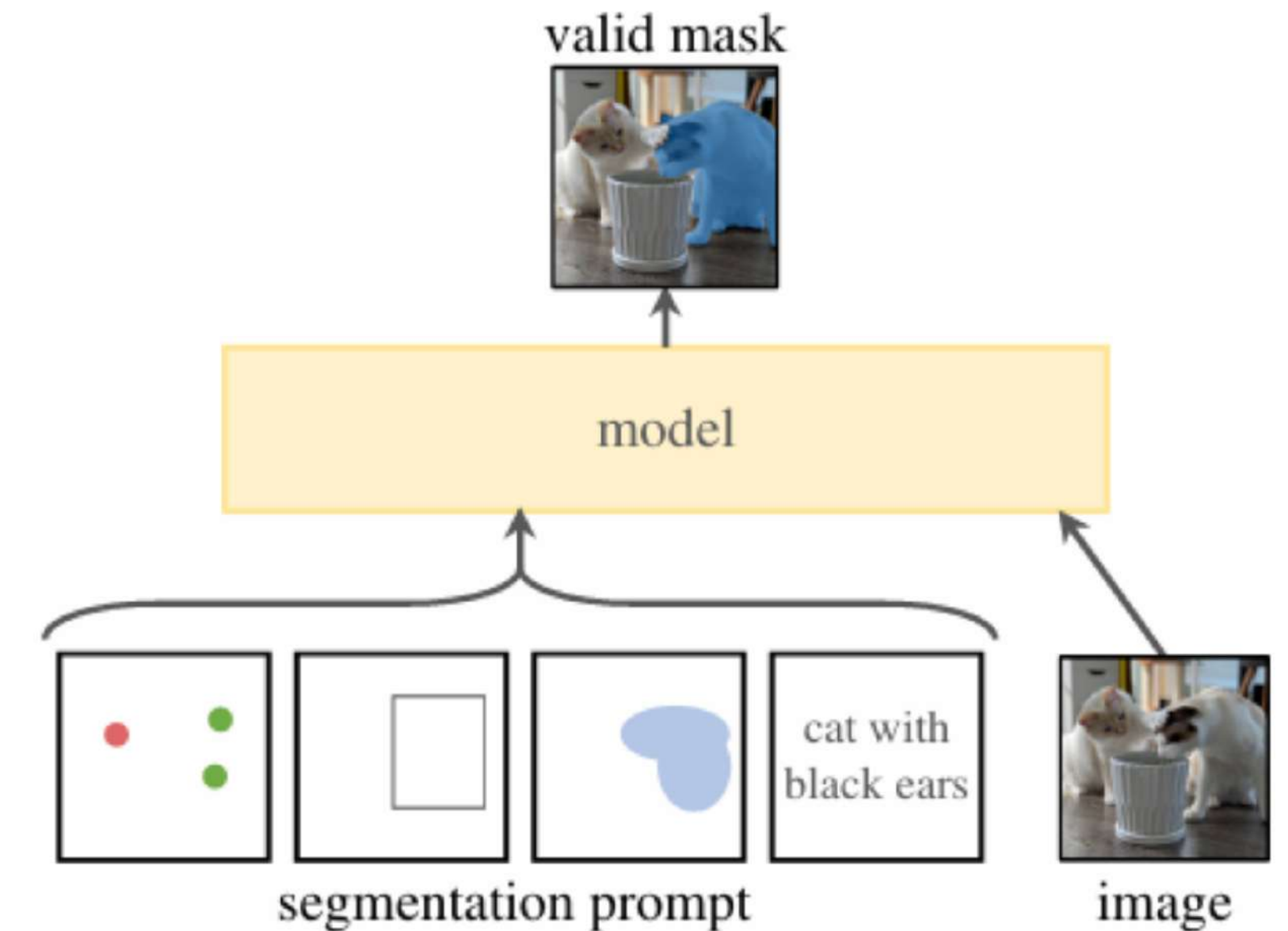
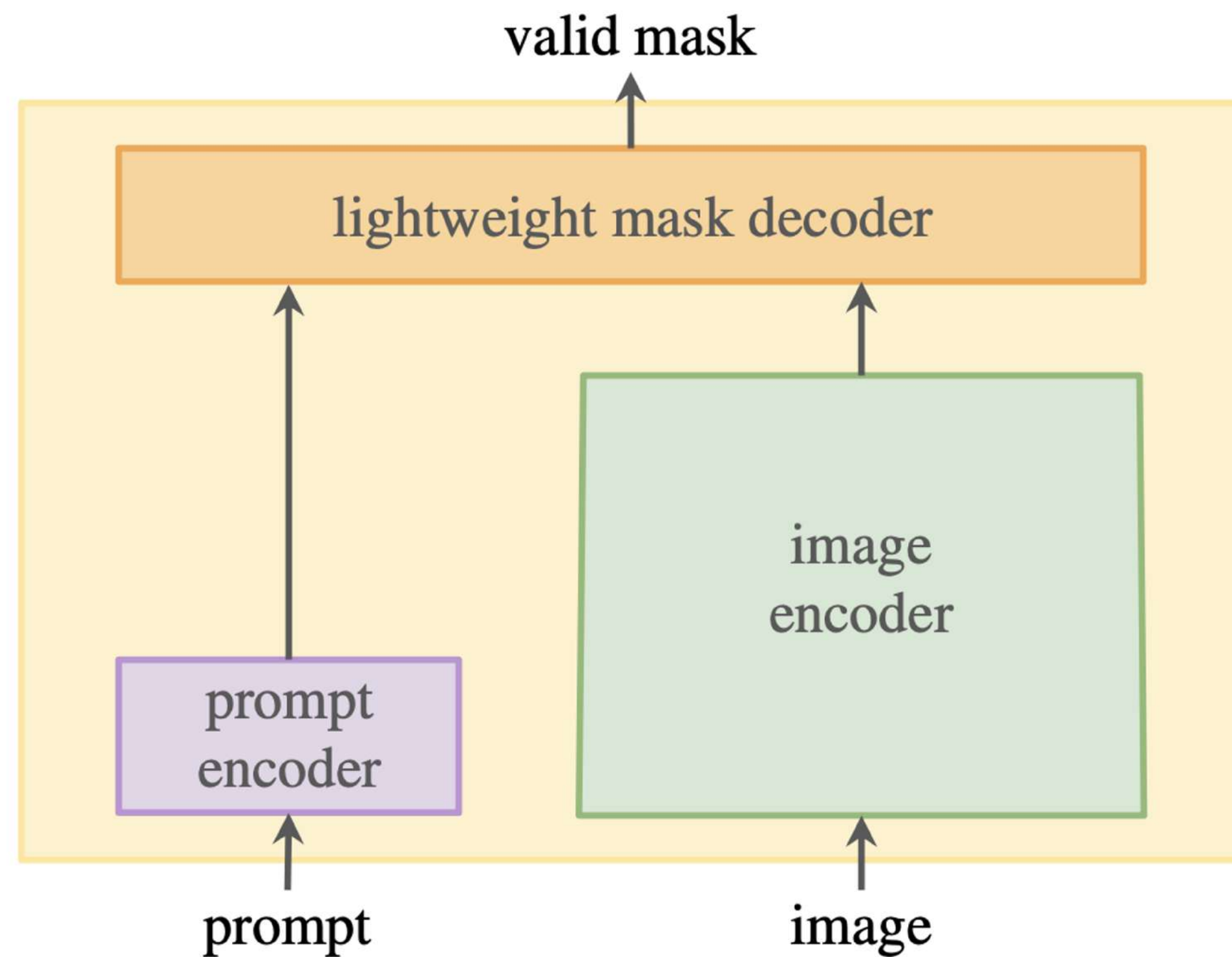
Object **D**etection with **T**ransformers: DETR



Recap

SAM: Segment Anything Models

Basic SAM Architecture



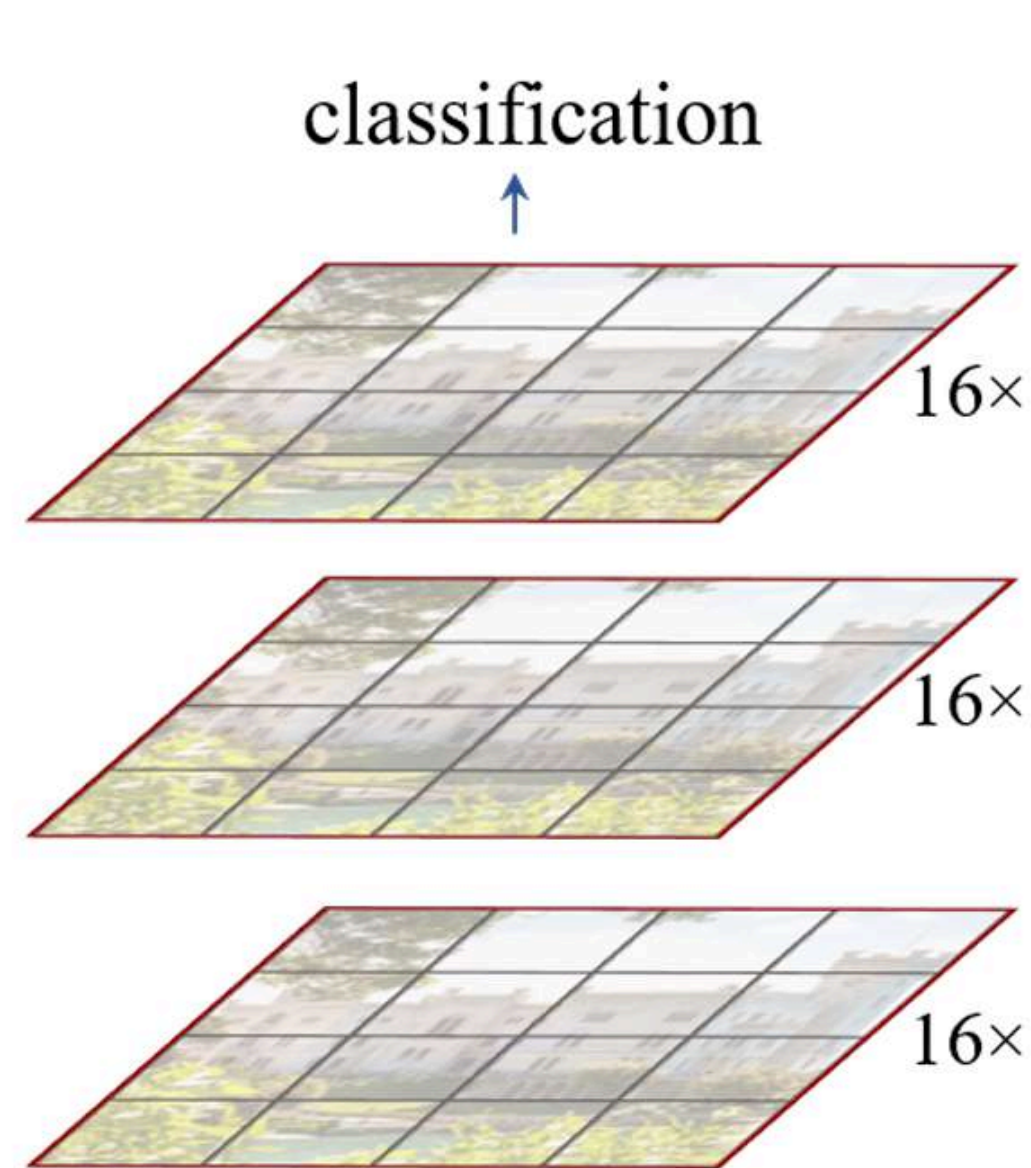
The image is encoded once, and different prompts can then be used to request different masks.

Kirillov, Alexander, et al. "Segment anything." ICCV 2023.

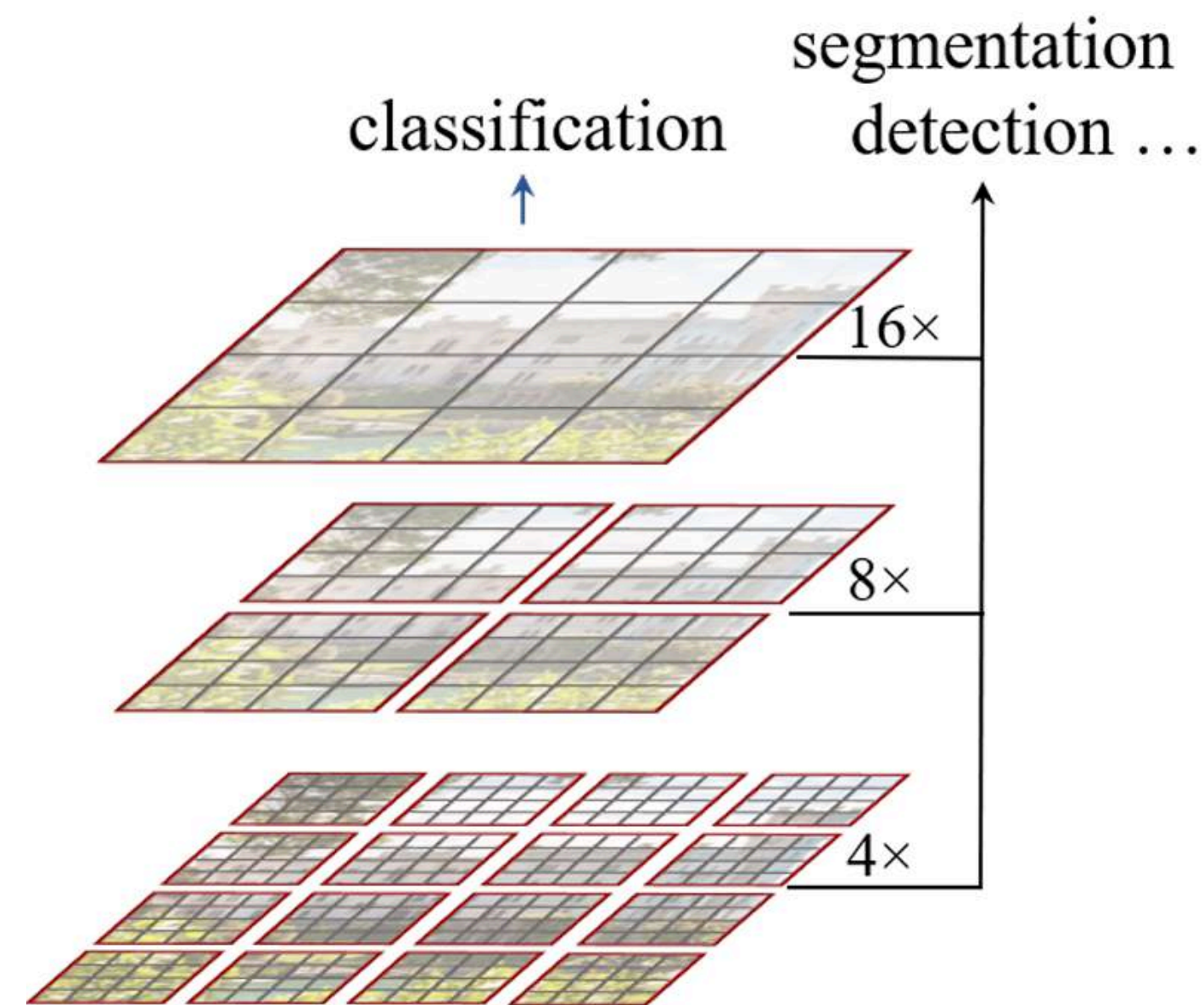
Recap Swin Transformer: Hierarchical Vision Transformer using Shifted Windows

- **Downsample 16x in the first step**
reduces spatial resolution too early,
making small objects harder to detect and
fine boundaries harder to segment.

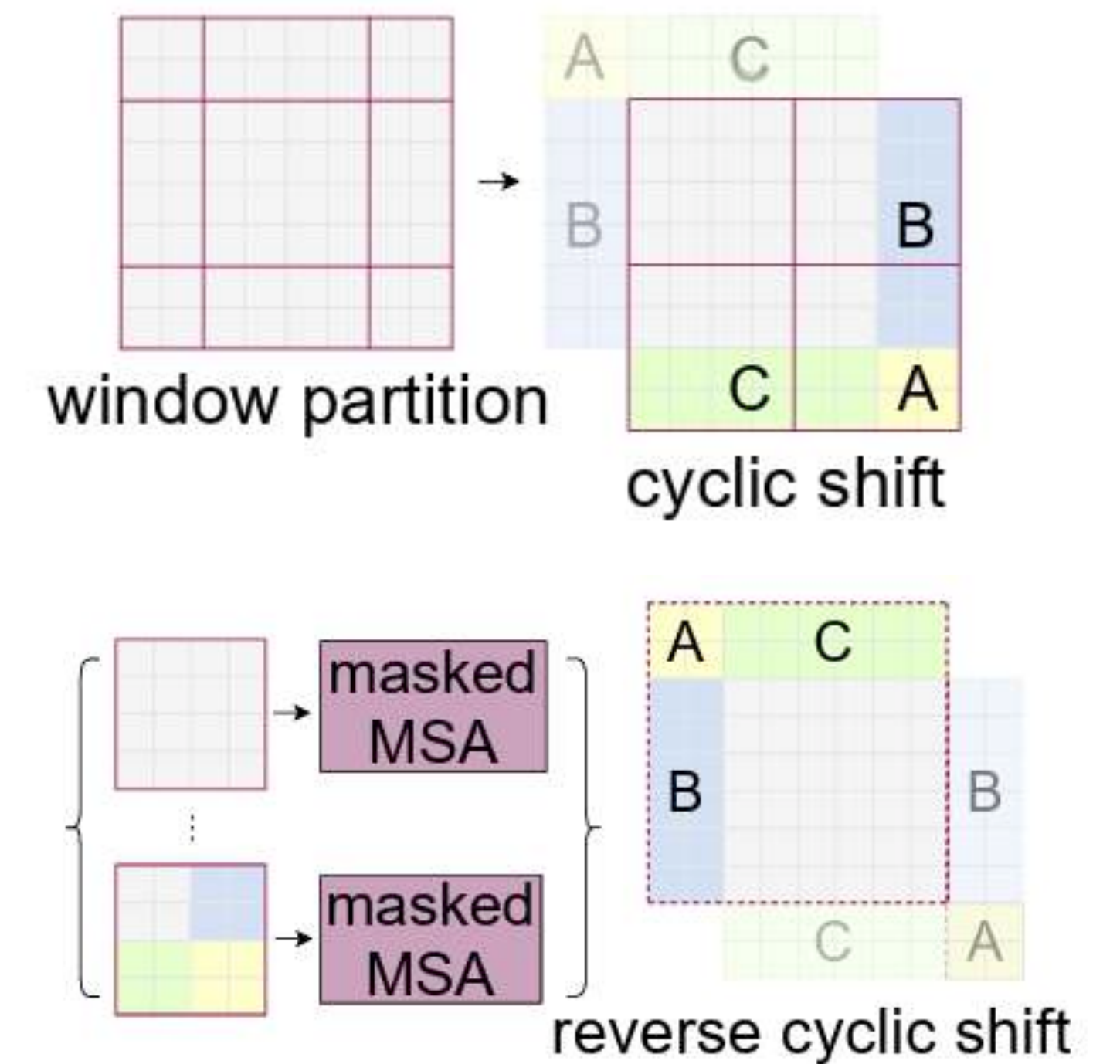
- **Split non-overlapping patches**, so
spatial continuity across patch
boundaries is not explicitly preserved



(b) ViT



(a) Swin Transformer (ours)



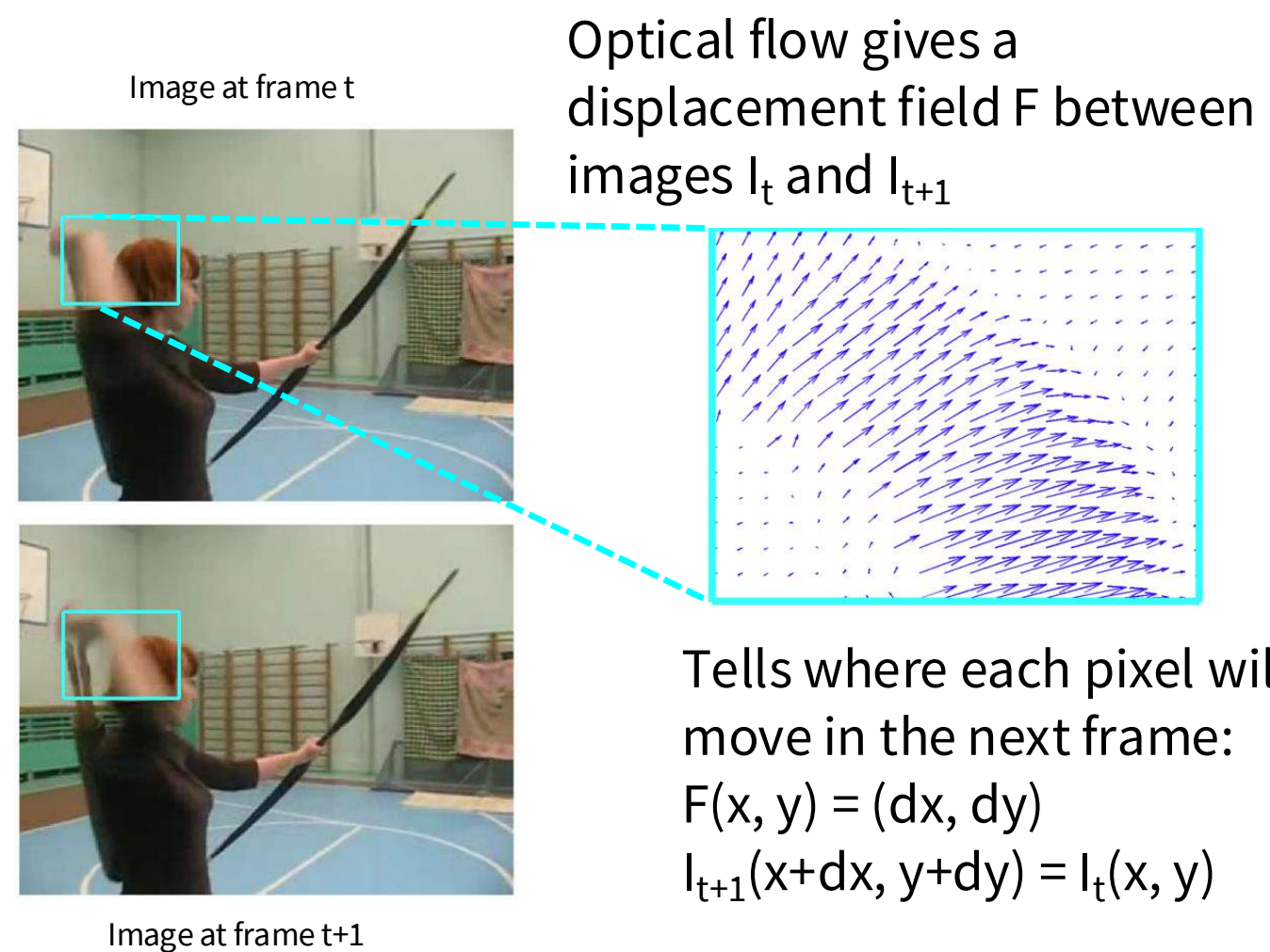
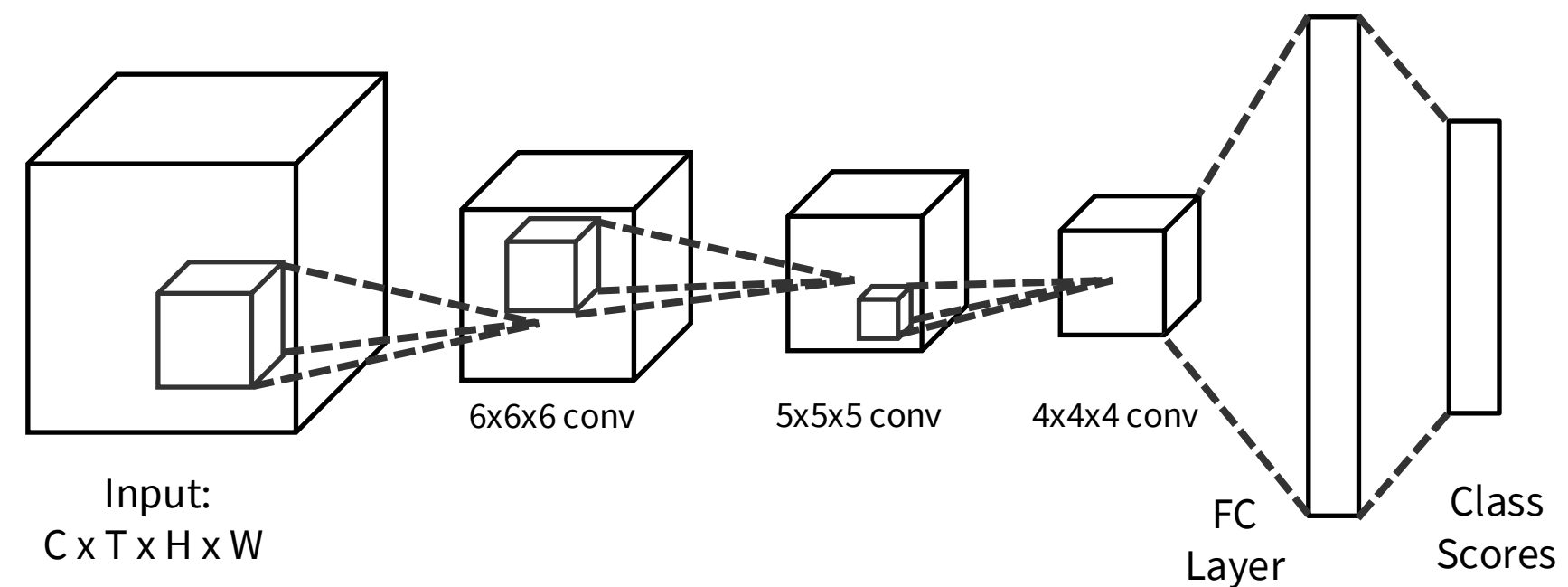
Shifted Window based Self-Attention
(SW-MSA)

Today: Motion & Video Understanding

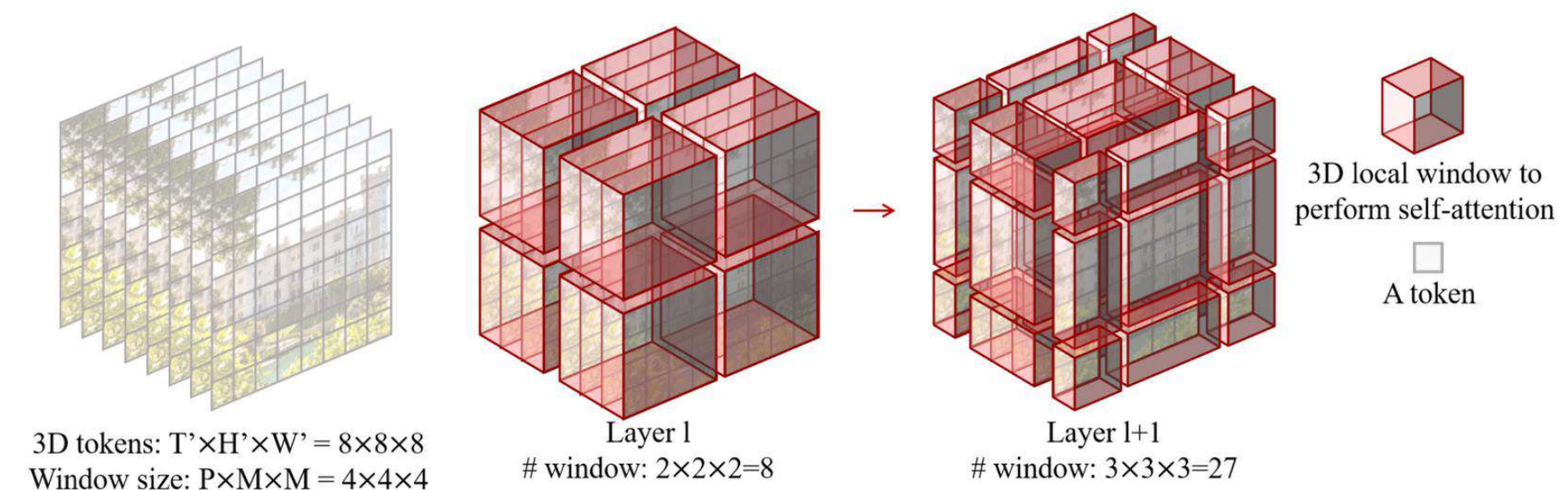
Video: $T \times 3 \times H \times W$



3D Convolution: High Level Idea



Video Swin Transformer (ViViT)



Today: Video = 2D + Time

A video is a sequence of images
4D tensor: $T \times 3 \times H \times W$



Example task: Video Classification



Images: Recognize objects



Dog
Cat
Fish
Truck



Videos: Recognize actions



Swimming
Running
Jumping
Eating
Standing

Example Video Dataset: Sports-1M



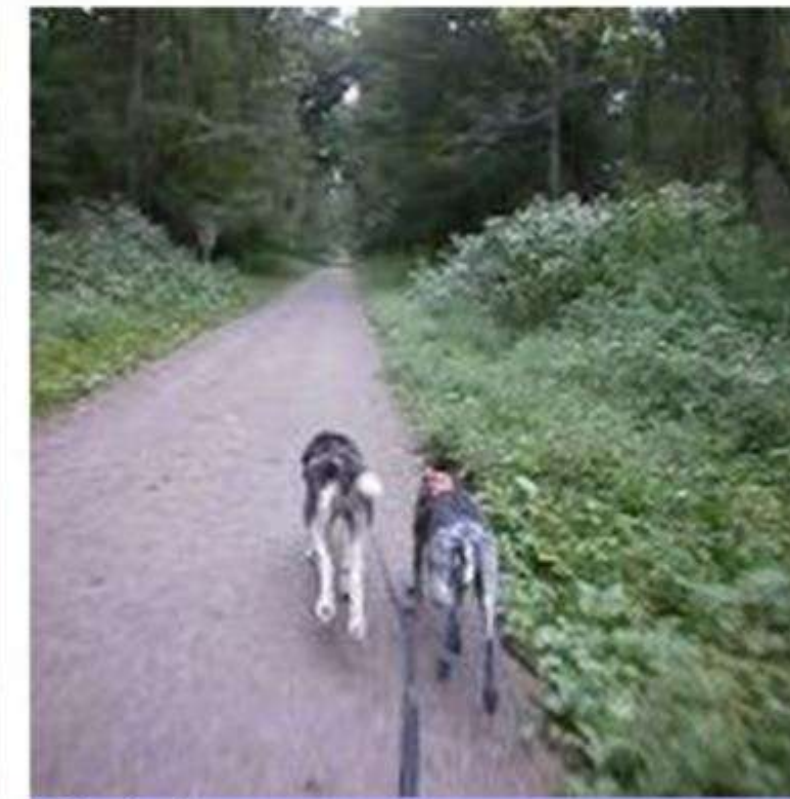
track cycling
cycling
track cycling
road bicycle racing
marathon
ultramarathon



ultramarathon
ultramarathon
half marathon
running
marathon
inline speed skating



heptathlon
heptathlon
decathlon
hurdles
pentathlon
sprint (running)



bikejoring
mushing
bikejoring
harness racing
skijoring
carting



longboarding
longboarding
aggressive inline skating
freestyle scootering
freeboard (skateboard)
sandboarding

1 million YouTube videos
annotated with labels for 487
different types of sports

Ground Truth

Correct prediction

Incorrect prediction

FRAME RATE

RECOMMENDED
180° SHUTTER SPEED

24 fps

1/48

25 fps

1/50

30 fps

1/60

48 fps

1/96

60 fps

1/120

120 fps

1/240



Problem: Videos are big!

Videos are often ~30 frames per second (fps)

Size of uncompressed video
(3 bytes per pixel):

SD (640 x 480): ~1.5 GB per minute

HD (1920 x 1080): ~10 GB per minute



Input video:

$T \times 3 \times H \times W$

Problem: Videos are big!

Videos are often ~30 frames per second (fps)

Size of uncompressed video
(3 bytes per pixel):

SD (640 x 480): ~1.5 GB per minute

HD (1920 x 1080): ~10 GB per minute

One Solution: Train on short clips:
low fps and low spatial resolution
e.g. $T = 16$, $H=W=112$
(3.2 seconds at 5 fps, 588 KB)



Input video:
 $T \times 3 \times H \times W$

Training vs Testing on Short Video Clips

Raw video: Long, high FPS

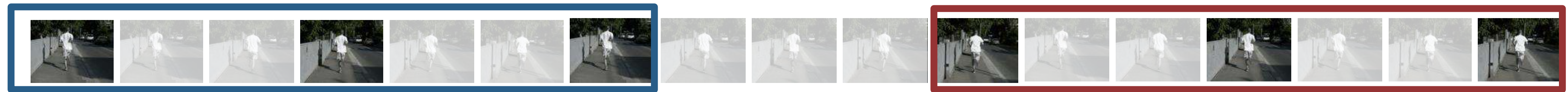


Training vs Testing on Video Clips

Raw video: Long, high FPS



Training: Train model to classify short clips with low FPS



Training vs Testing on Video Clips

Raw video: Long, high FPS



Training: Train model to classify short clips with low FPS



Testing: Run model on different clips, average predictions

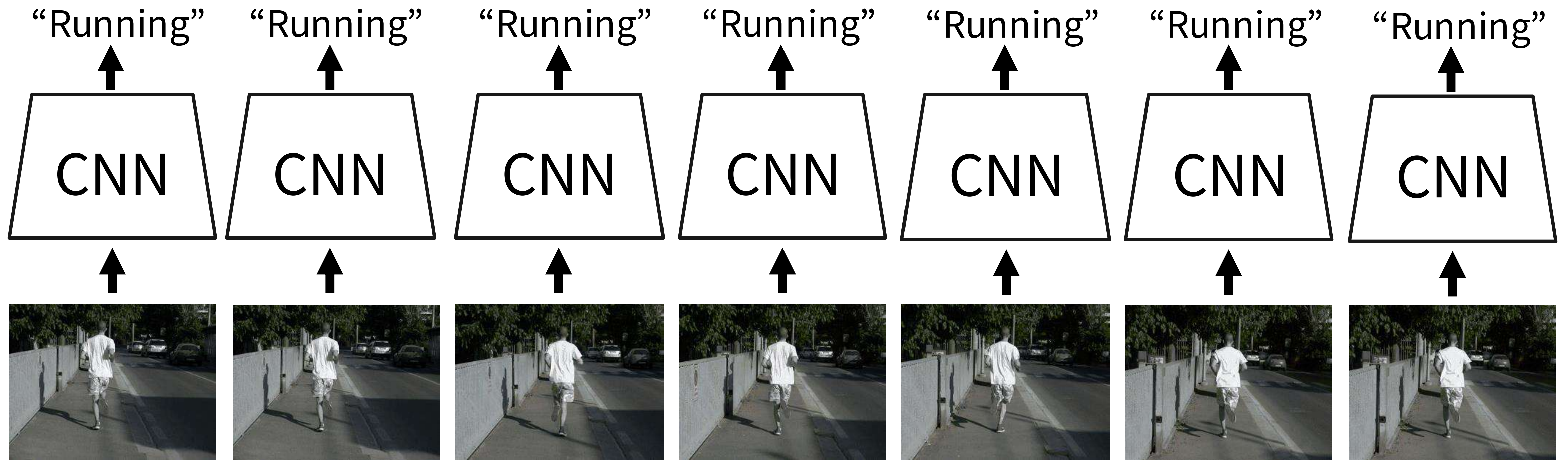


Video Classification: Single-Frame CNN

Simple idea: train normal 2D CNN to classify video frames independently!

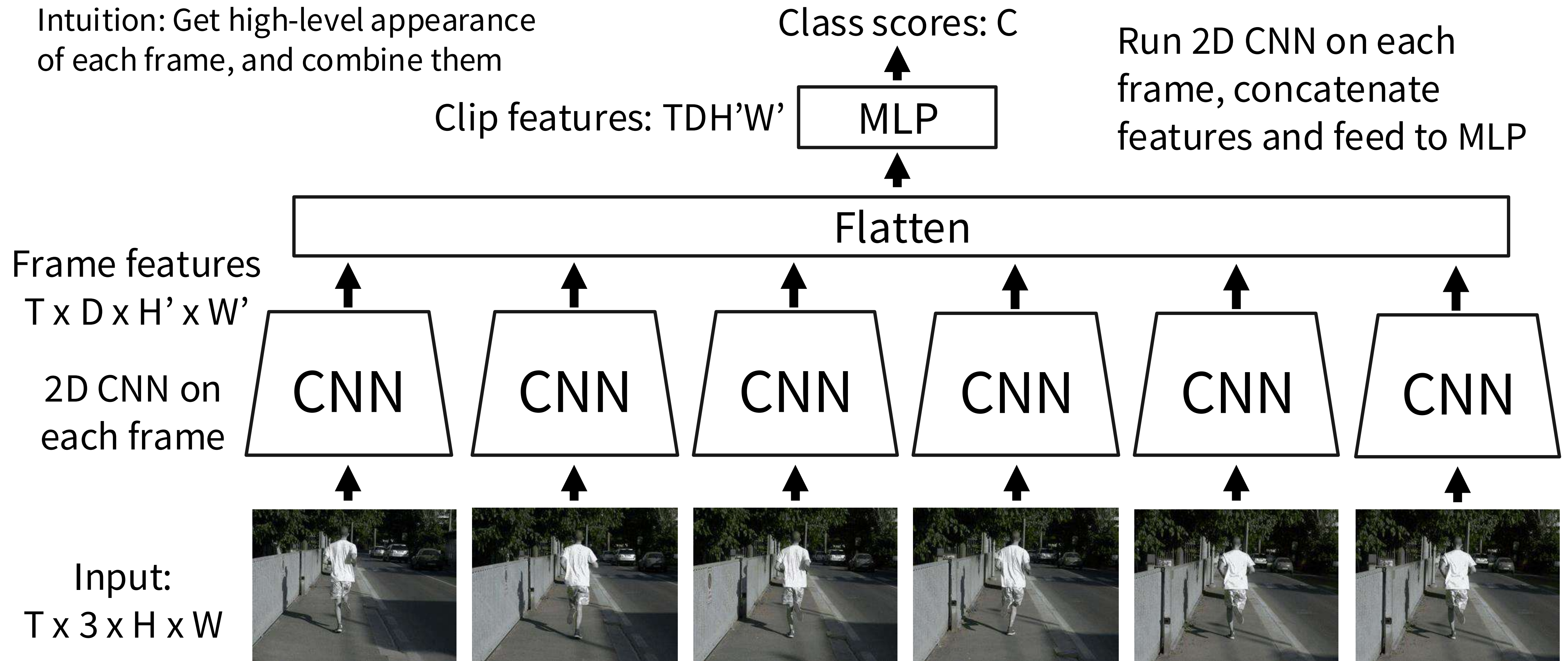
(Average predicted probs at test-time)

Often a very strong baseline for video classification



Video Classification: Late Fusion (with FC layers)

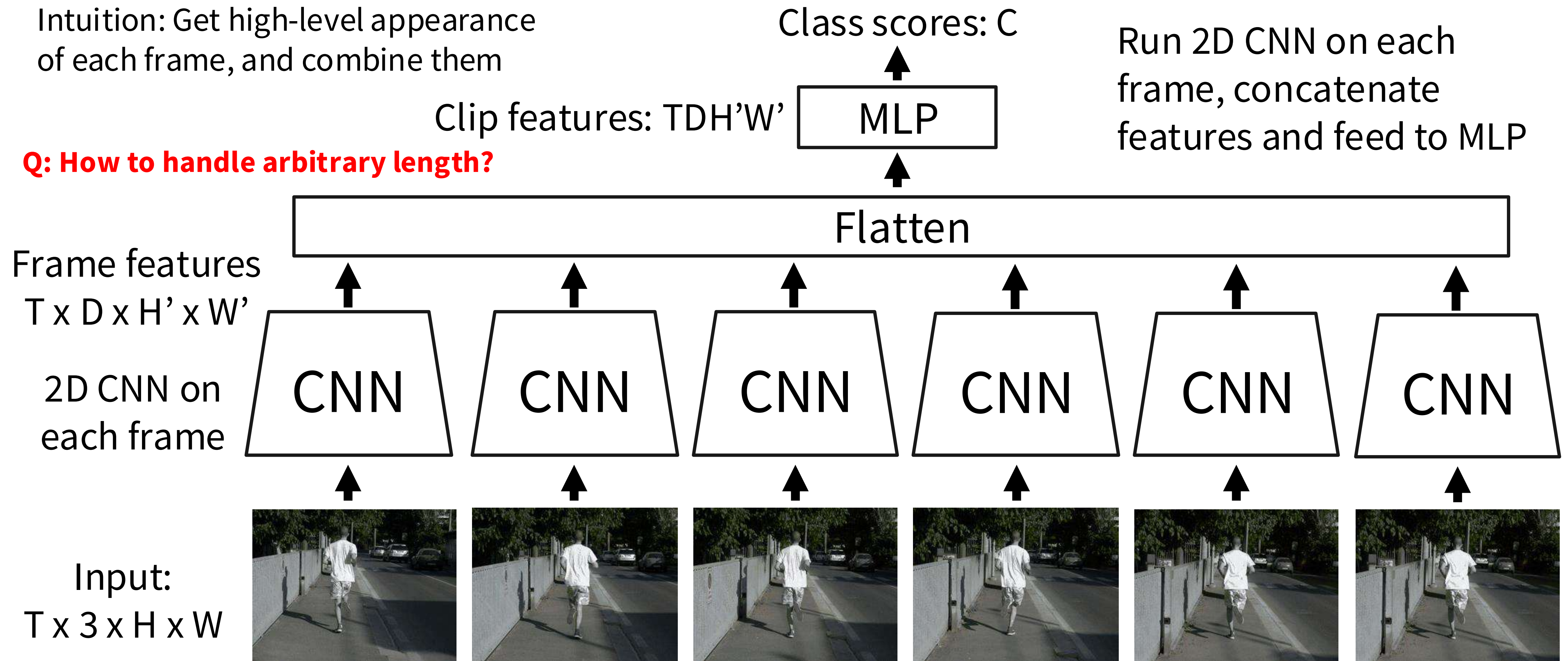
Intuition: Get high-level appearance of each frame, and combine them



Video Classification: Late Fusion (with FC layers)

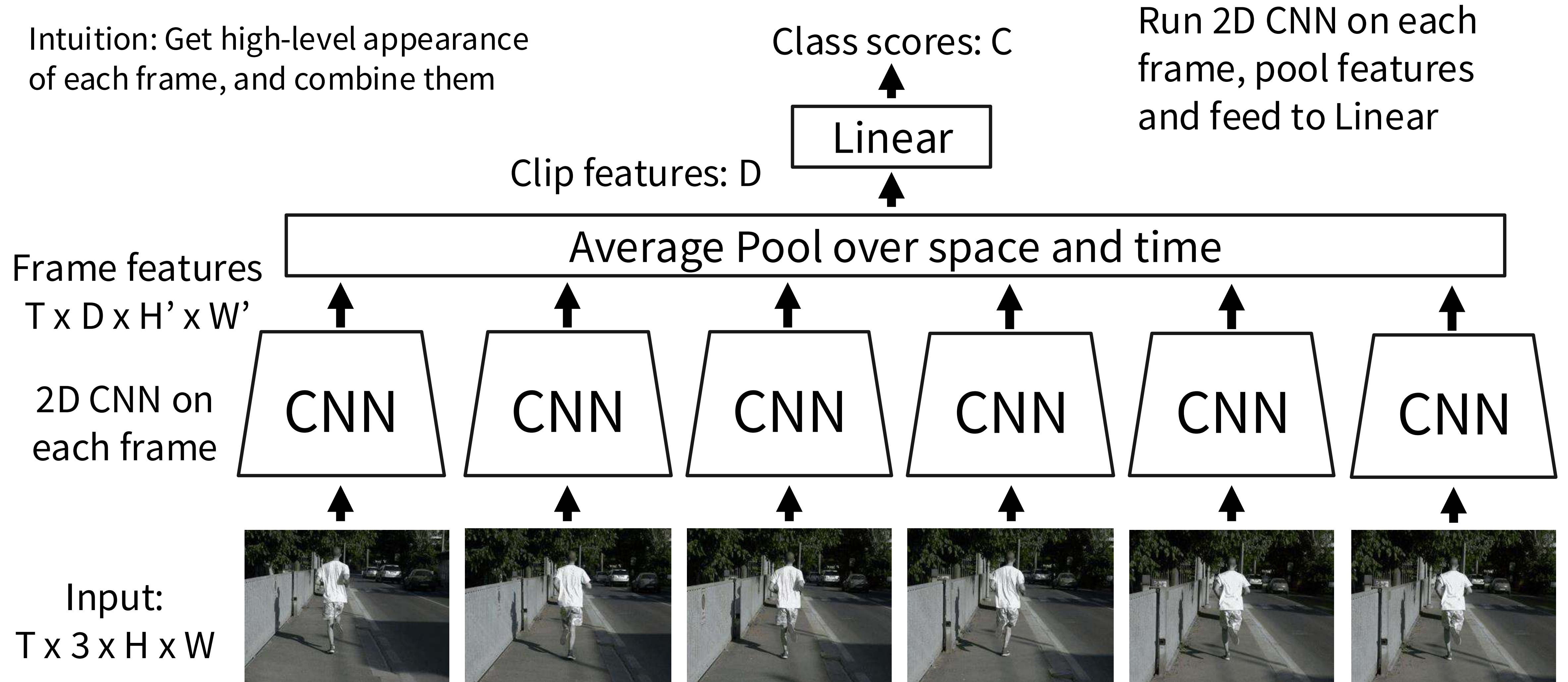
Intuition: Get high-level appearance of each frame, and combine them

Q: How to handle arbitrary length?



Video Classification: Late Fusion (with pooling)

Intuition: Get high-level appearance of each frame, and combine them

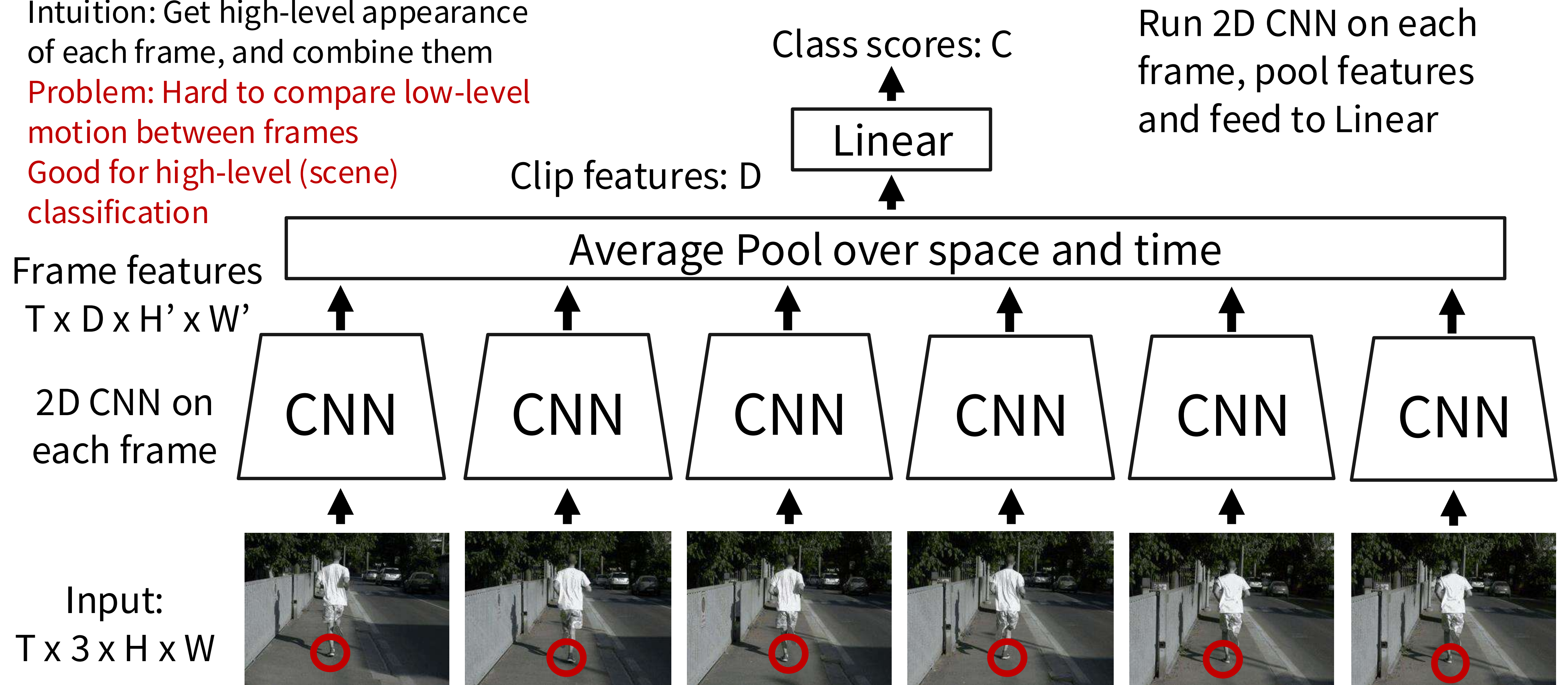


Video Classification: Late Fusion (with pooling)

Intuition: Get high-level appearance of each frame, and combine them

Problem: Hard to compare low-level motion between frames

Good for high-level (scene) classification



Video Classification: Early Fusion with 3D CNNs

Intuition: Use 3D versions of convolution and pooling to slowly fuse temporal information over the course of the network

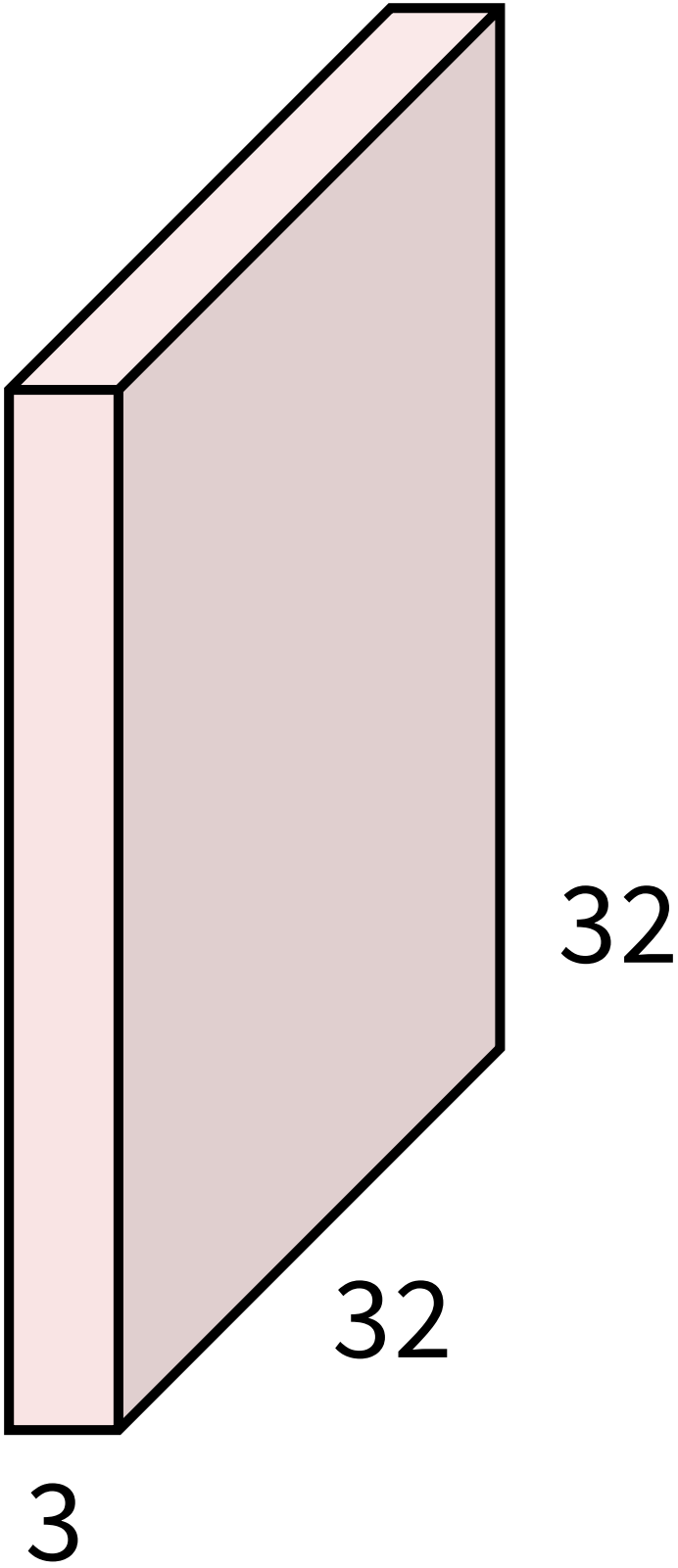
Each activation map in the network is a 4D tensor:
 $D \times T \times H \times W$
Use 3D conv and 3D pooling operations

Input:
 $3 \times T \times H \times W$

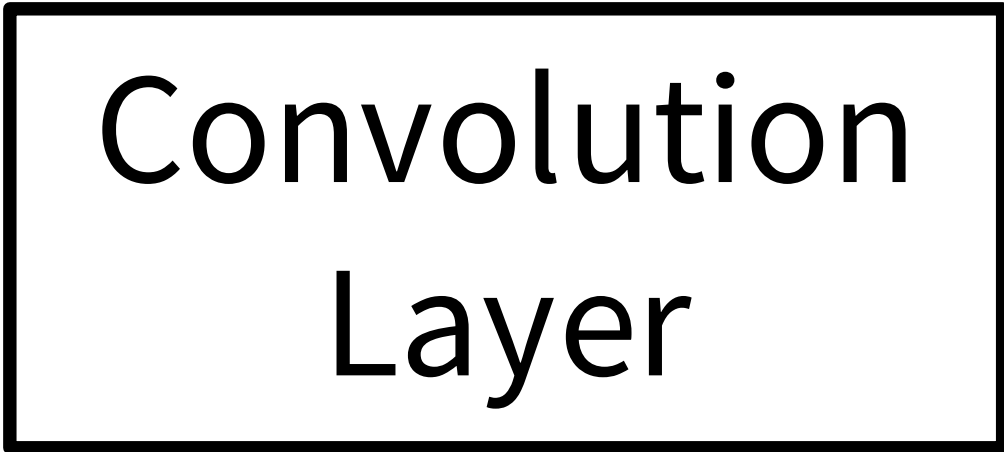
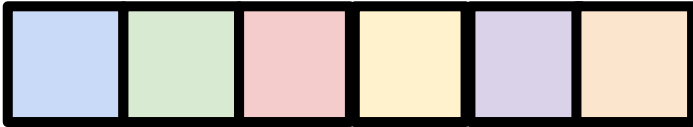


Recap: 2D Convolution Layer

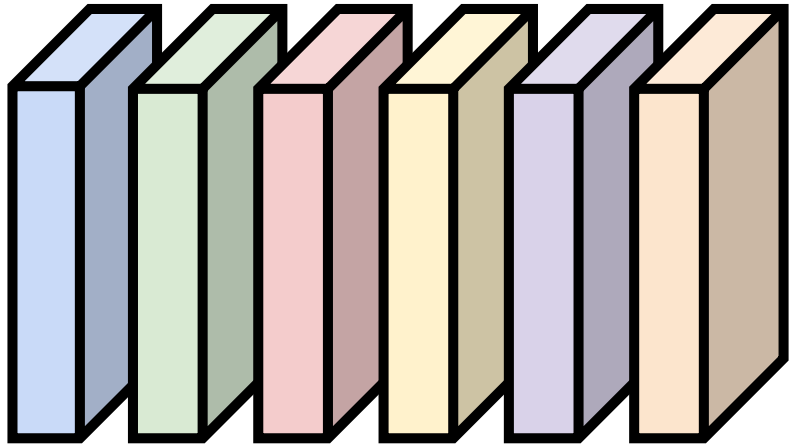
3x32x32 image



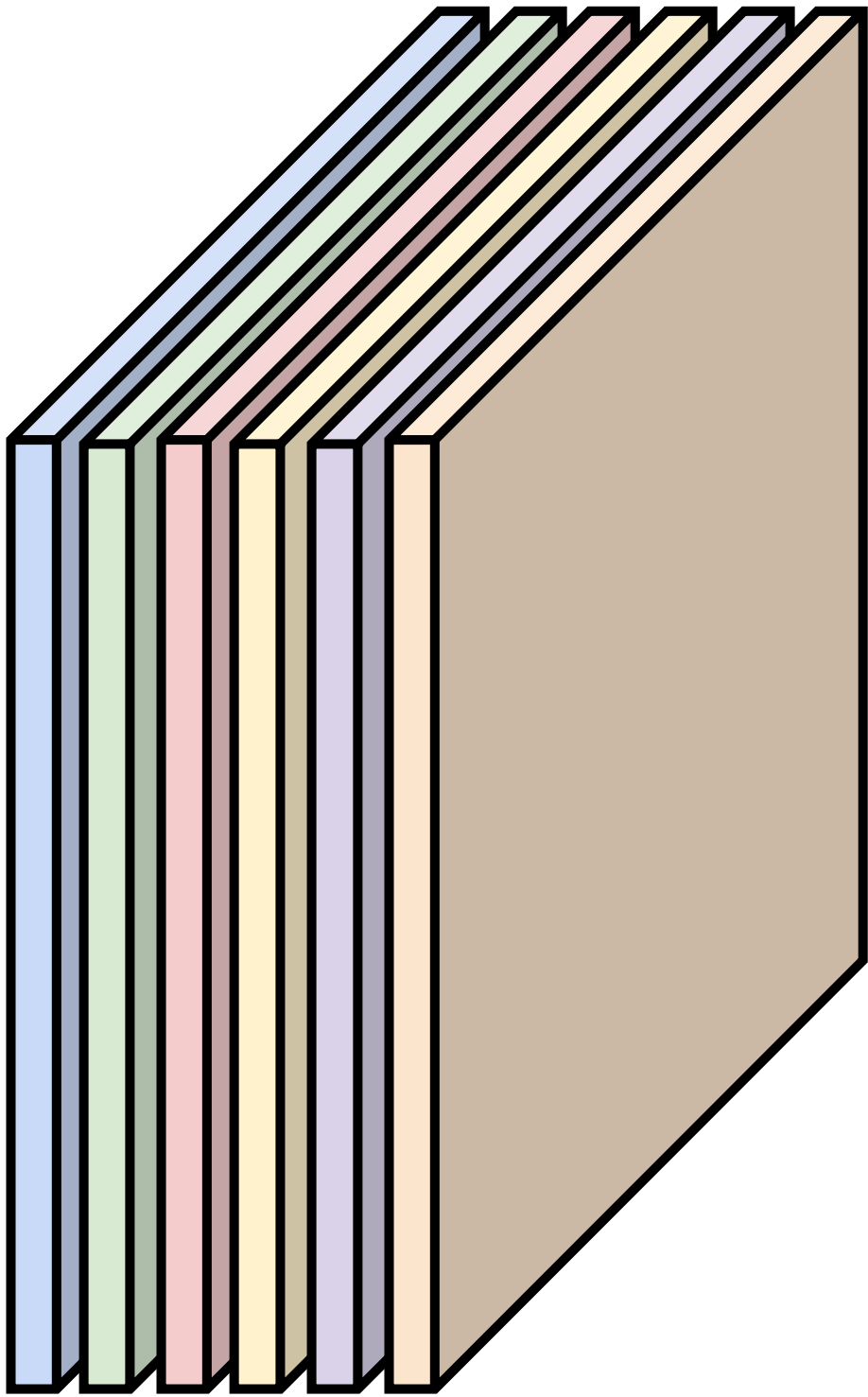
Also 6-dim bias vector:



6x3x5x5 filters

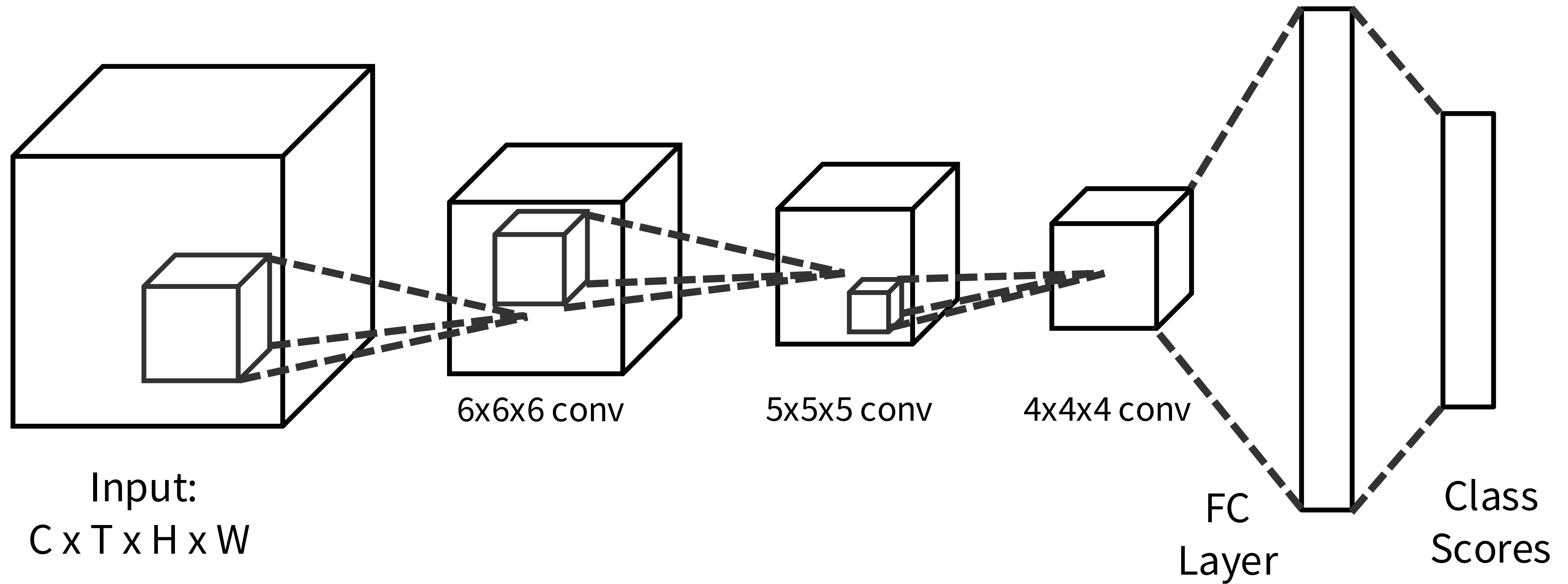


6 activation maps,
each 1x28x28



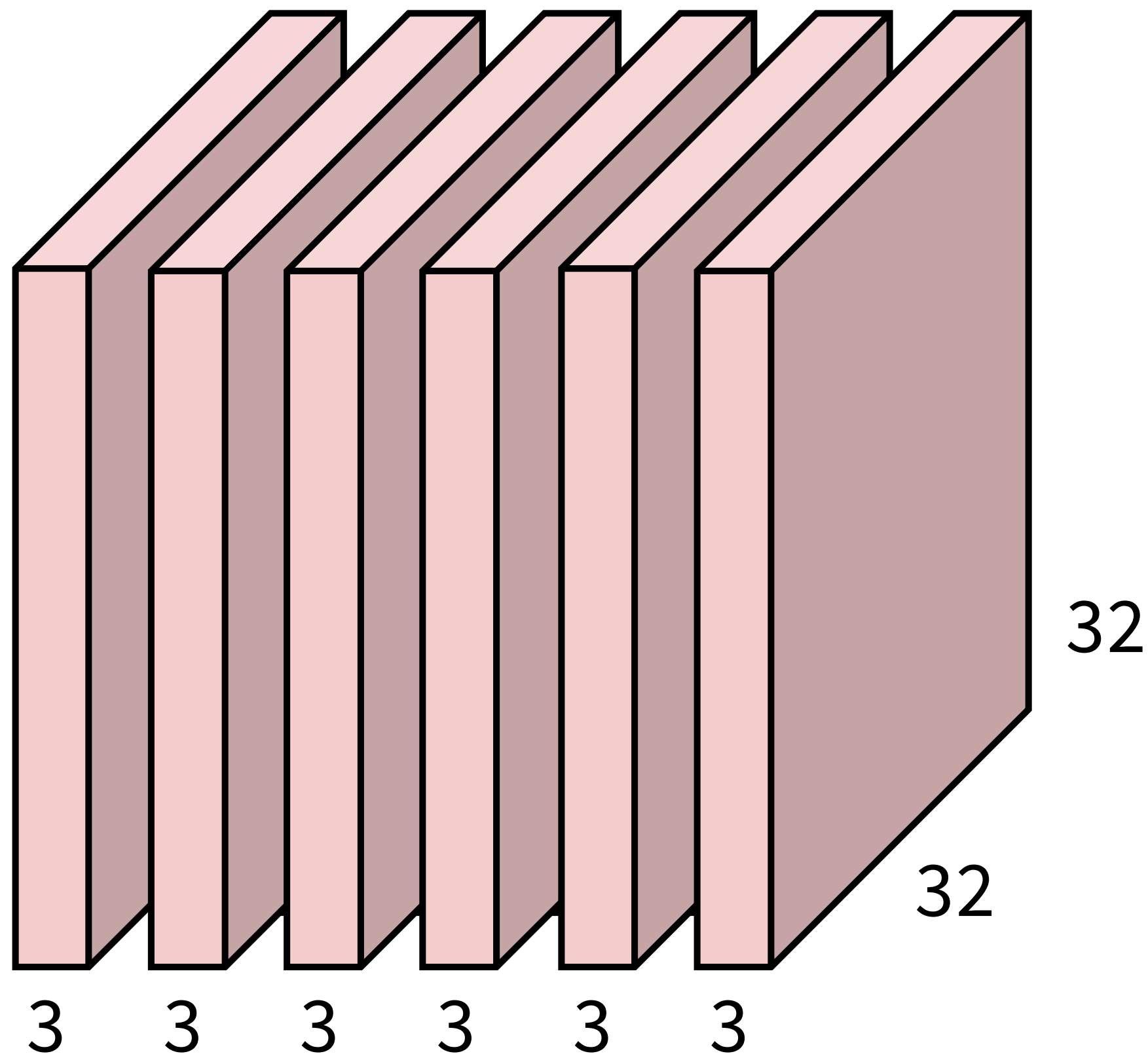
Stack activations to get a
6x28x28 output image!

3D Convolution: High Level Idea



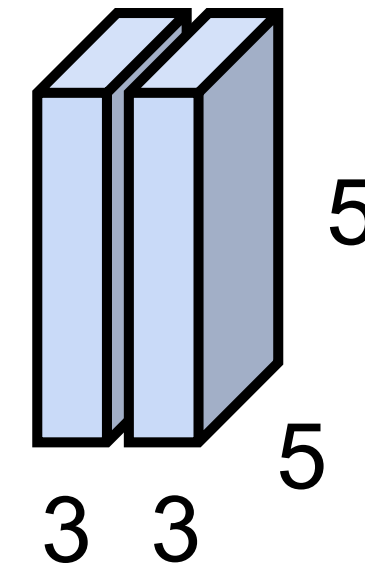
How is 3D Different? Filters

$T \times 3 \times 32 \times 32$ images ($T=6$)



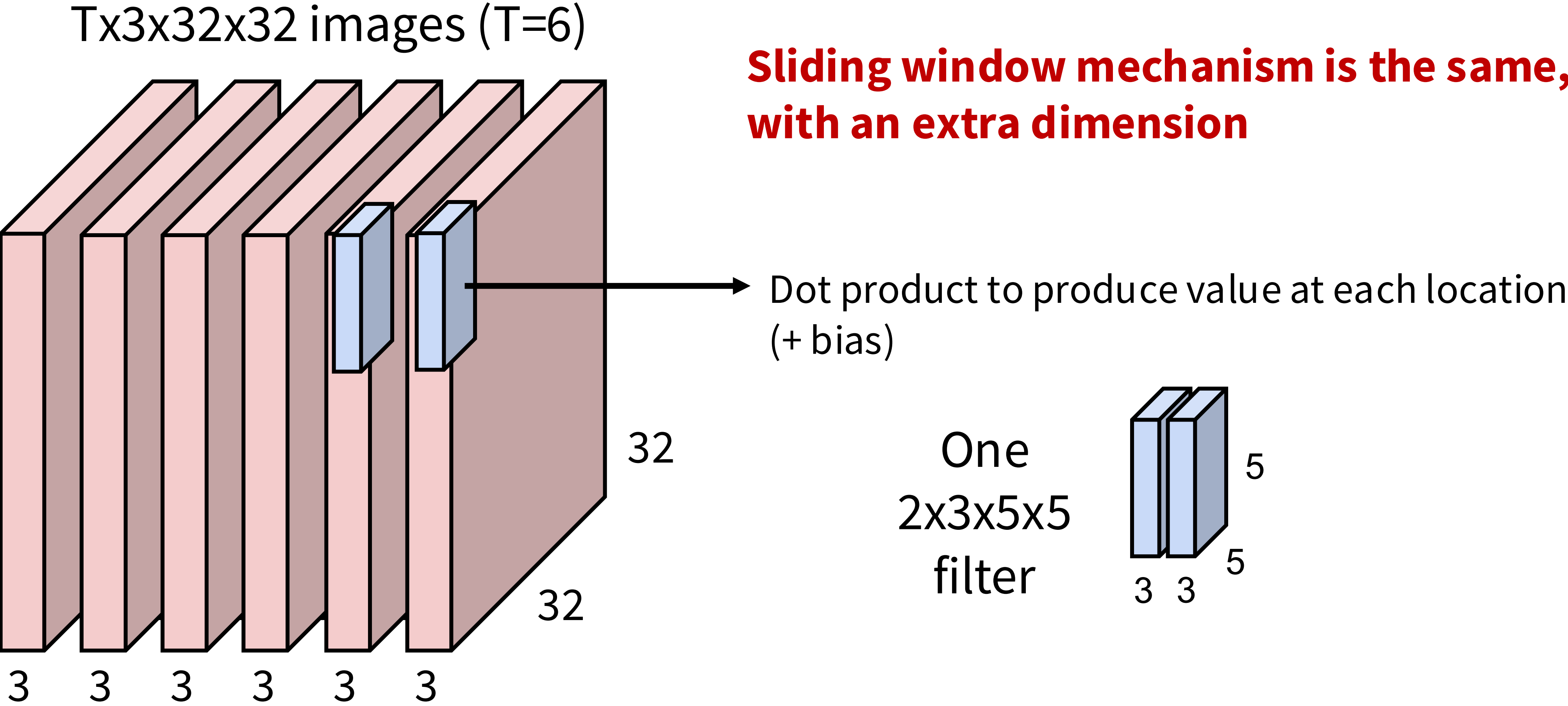
What does one filter look like?

One
 $2 \times 3 \times 5 \times 5$
filter

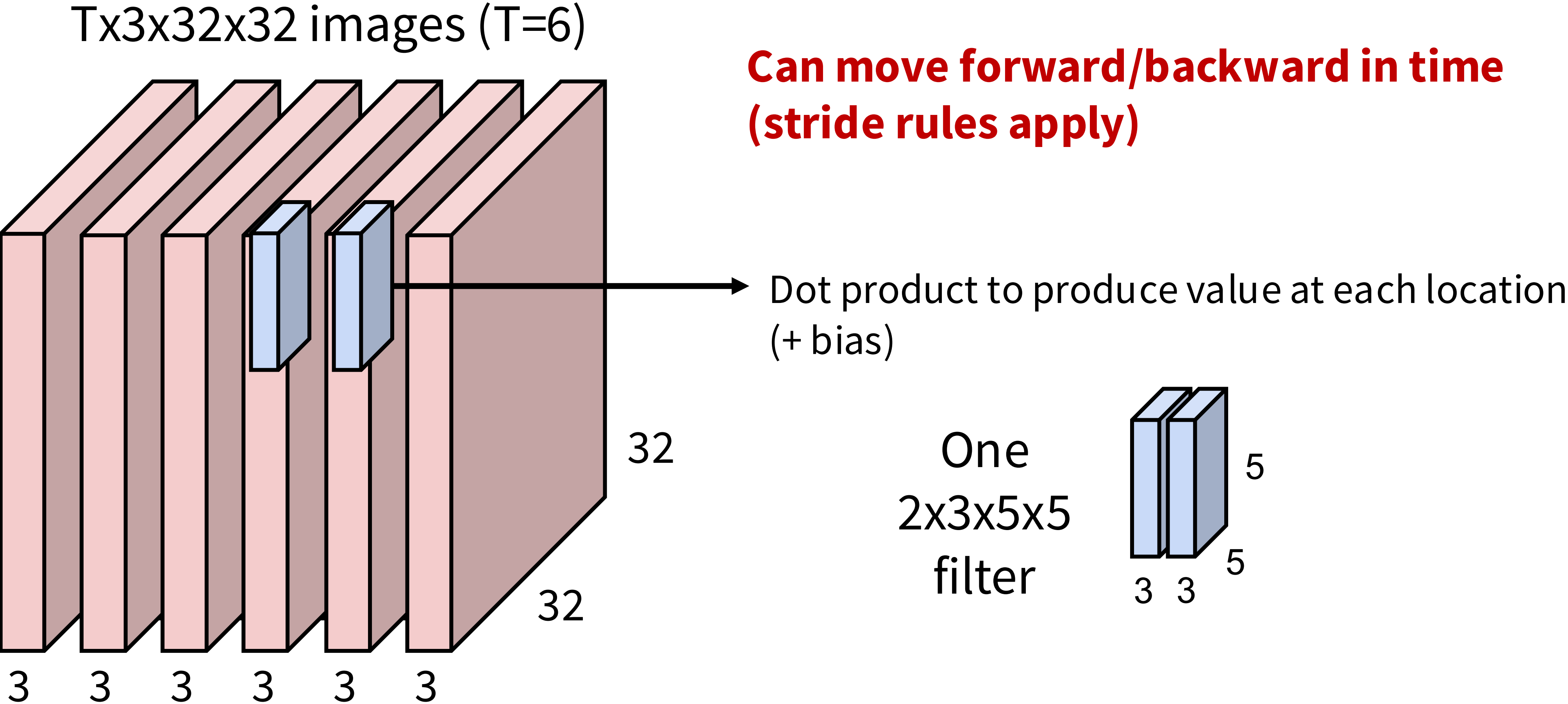


Now has time dimension!

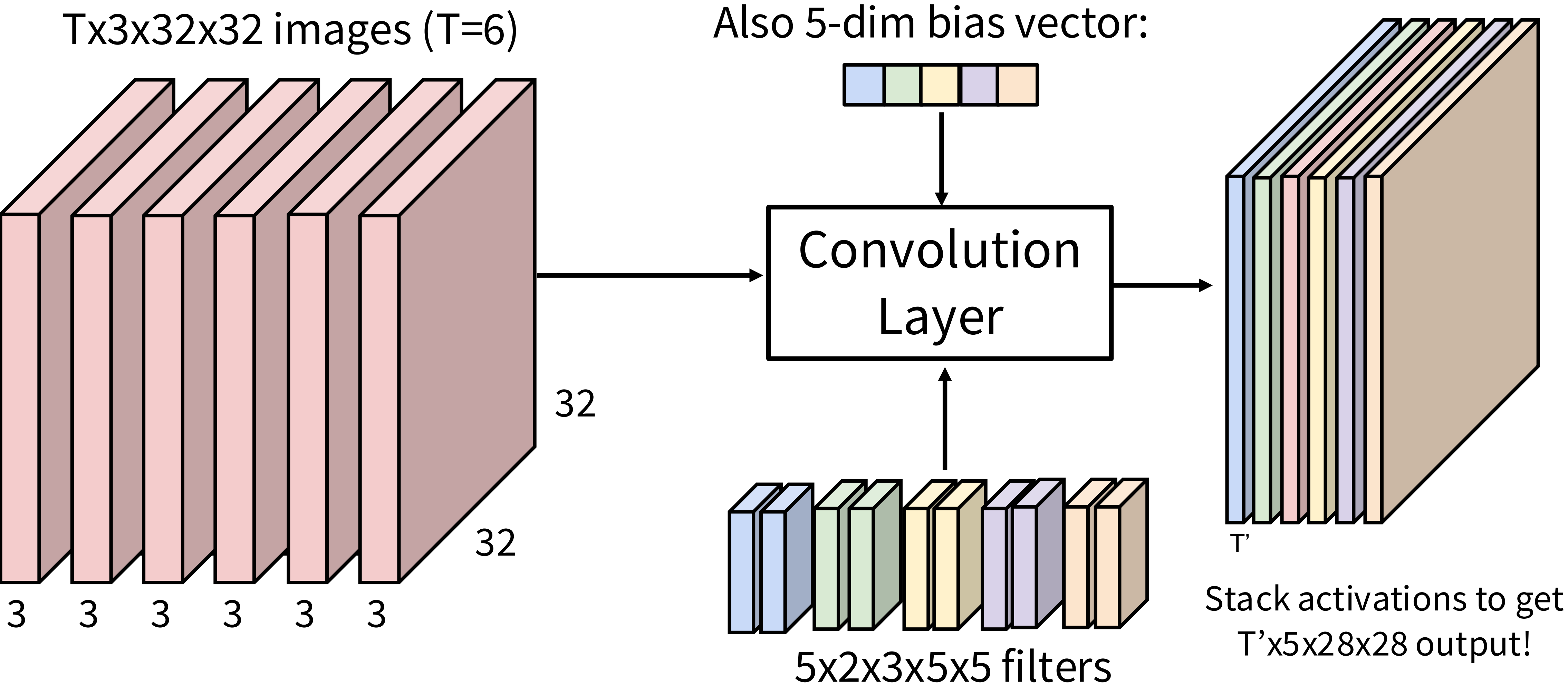
How is 3D Different? Filters



How is 3D Different? Stride



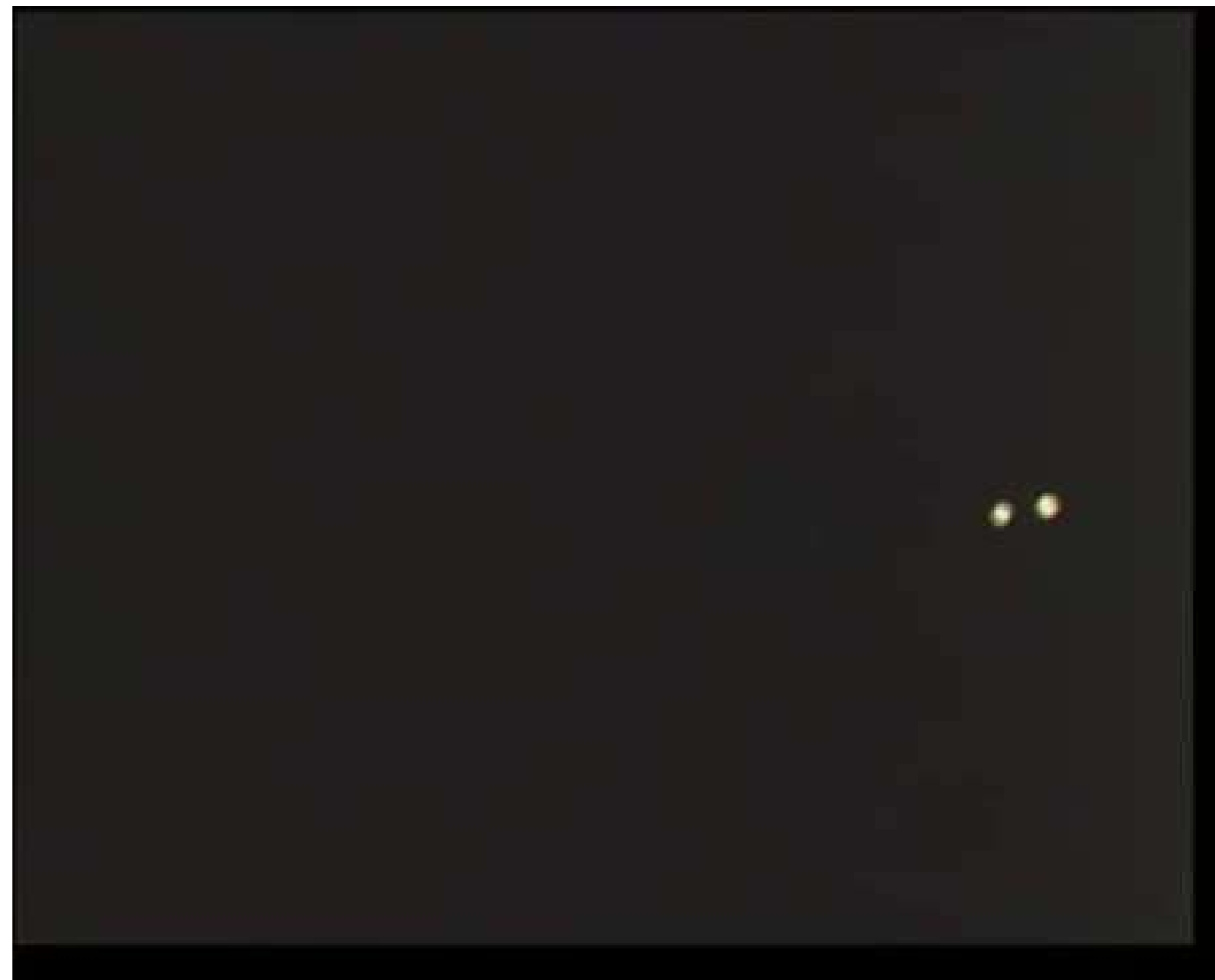
How is 3D Different? Output



Next: Some Tricks To Improve Performance

Trick #1: Recognizing Actions from Motion

Insight: We can easily recognize actions using only motion information



Measuring Motion: Optical Flow

Image at frame t

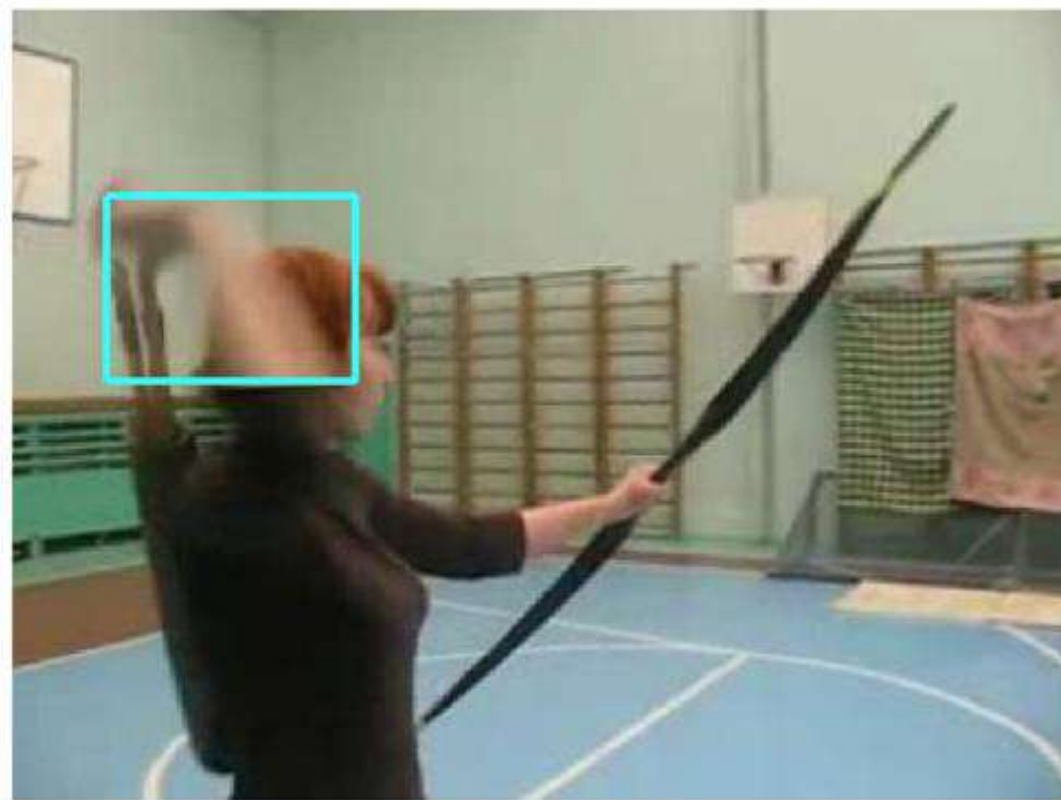
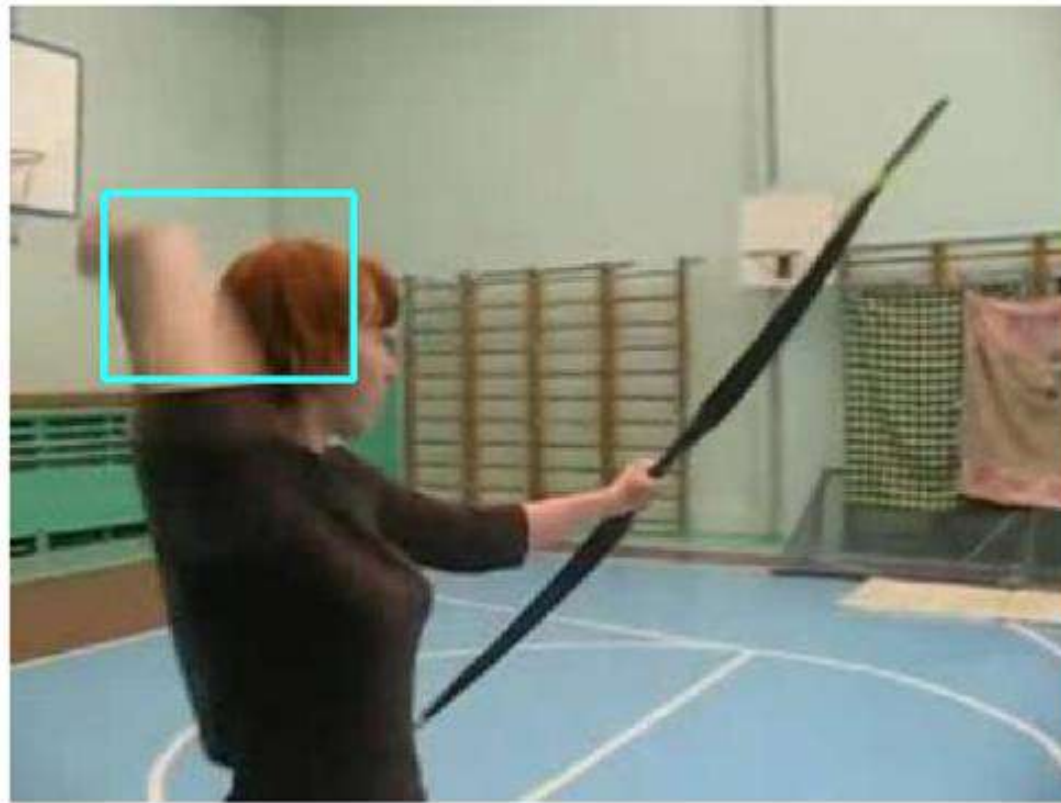
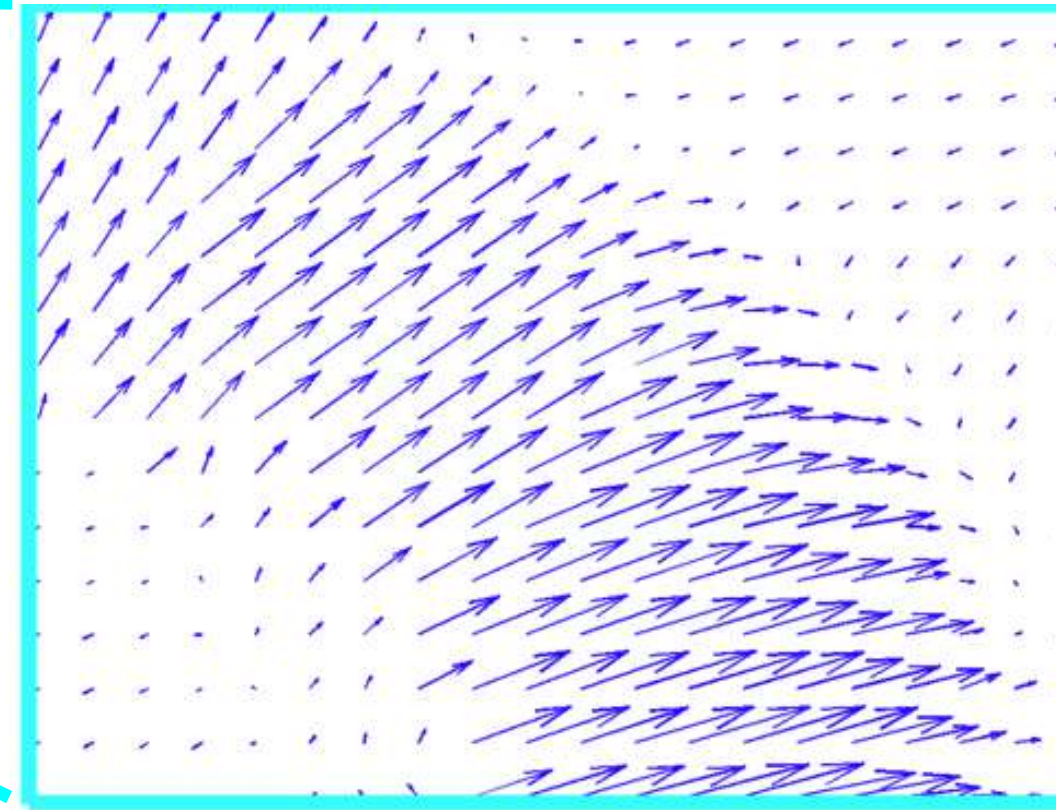
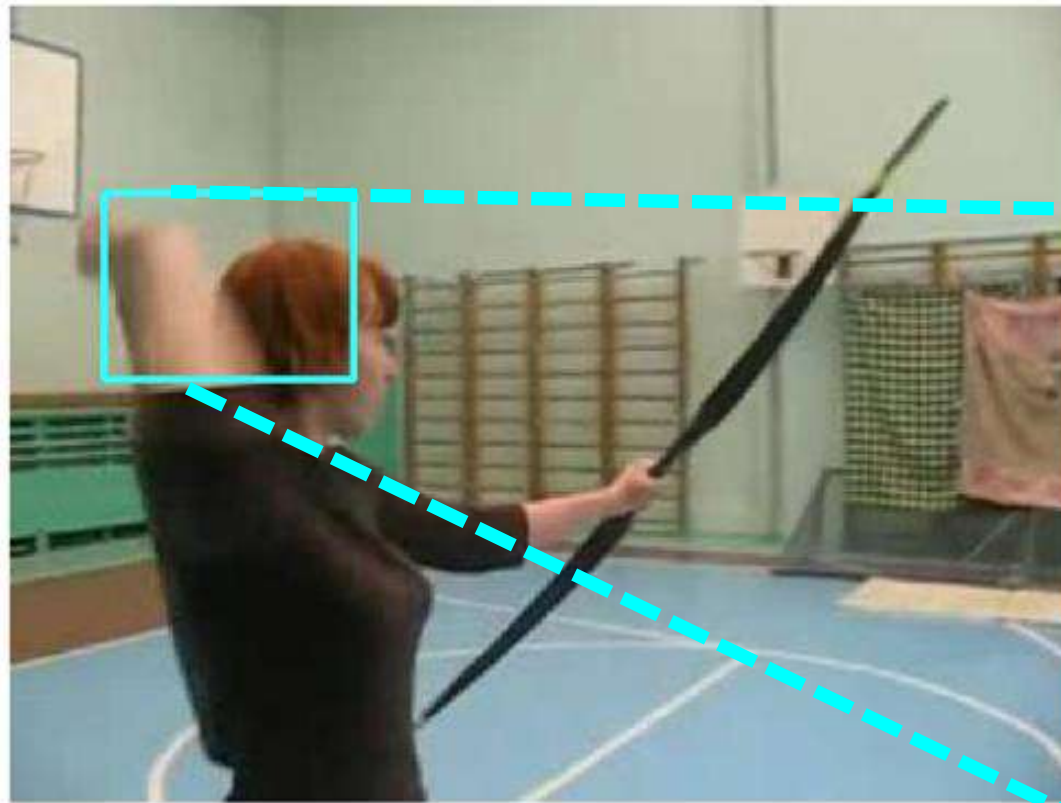


Image at frame $t+1$

Measuring Motion: Optical Flow

Optical flow gives a displacement field F between images I_t and I_{t+1}

Image at frame t



Computed via algorithm or
faster NN approximation

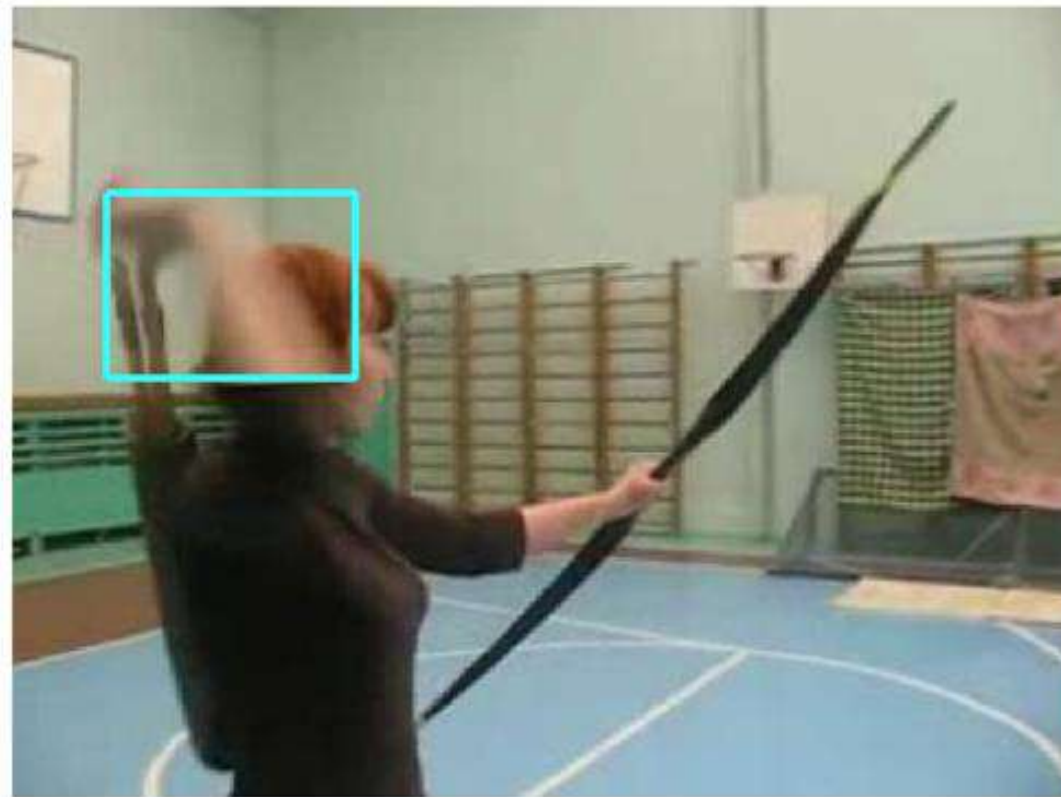


Image at frame $t+1$

Tells where each pixel will
move in the next frame:

$$F(x, y) = (dx, dy)$$

$$I_{t+1}(x+dx, y+dy) = I_t(x, y)$$

Measuring Motion: Optical Flow

Optical flow gives a displacement field F between images I_t and I_{t+1}

Image at frame t

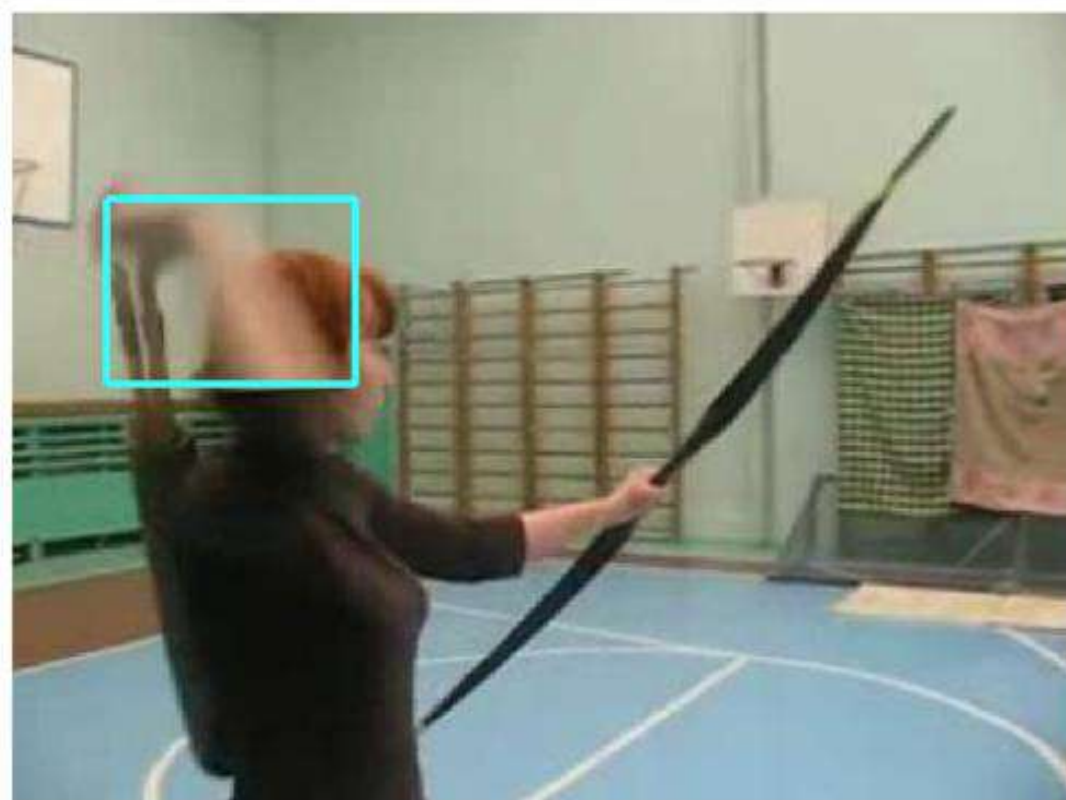
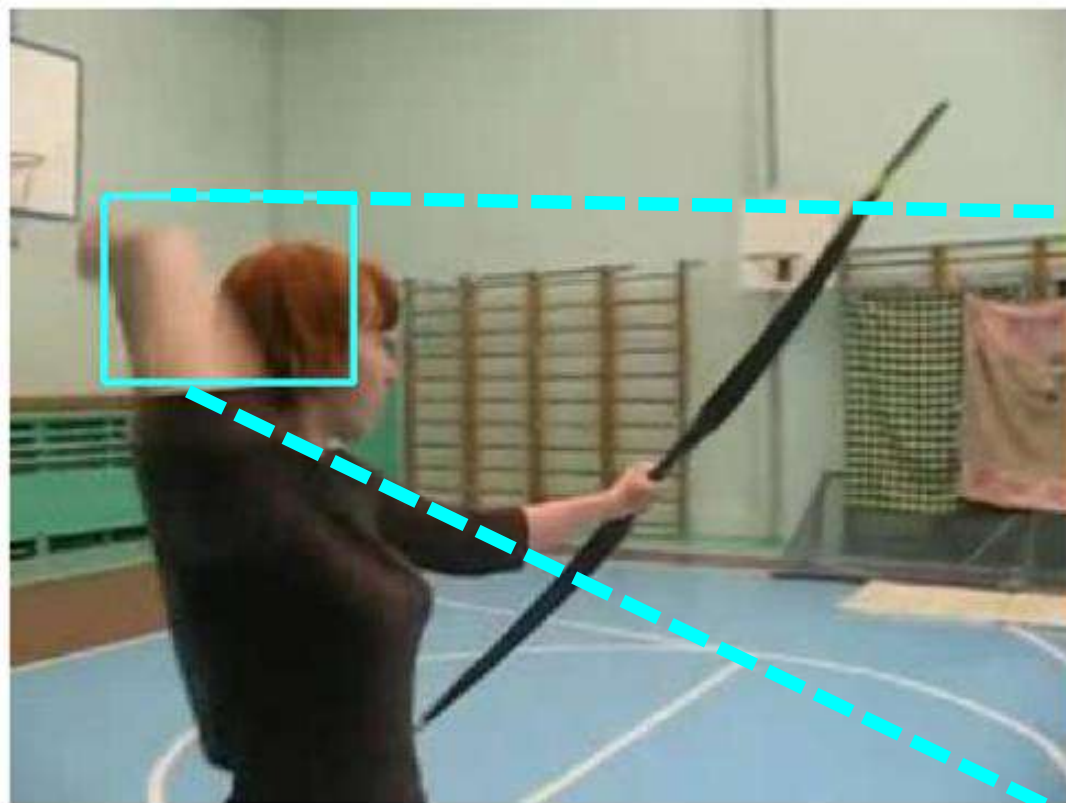
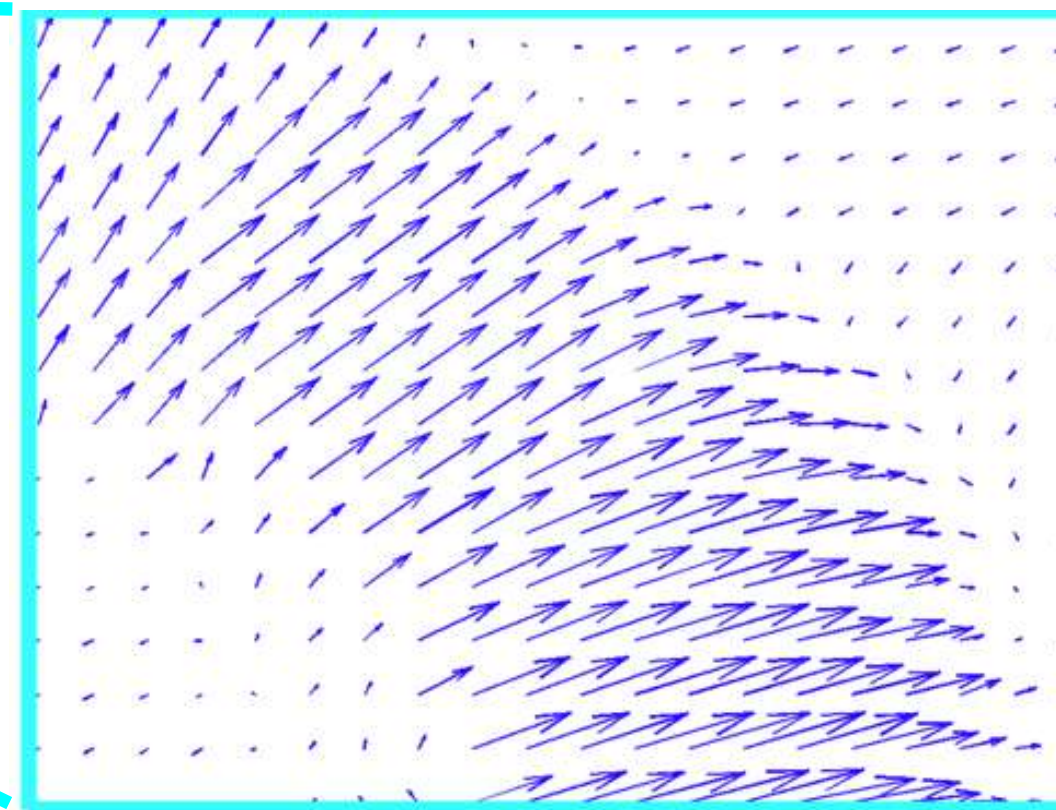


Image at frame $t+1$



Tells where each pixel will move in the next frame:
 $F(x, y) = (dx, dy)$
 $I_{t+1}(x+dx, y+dy) = I_t(x, y)$

Optical Flow highlights local motion

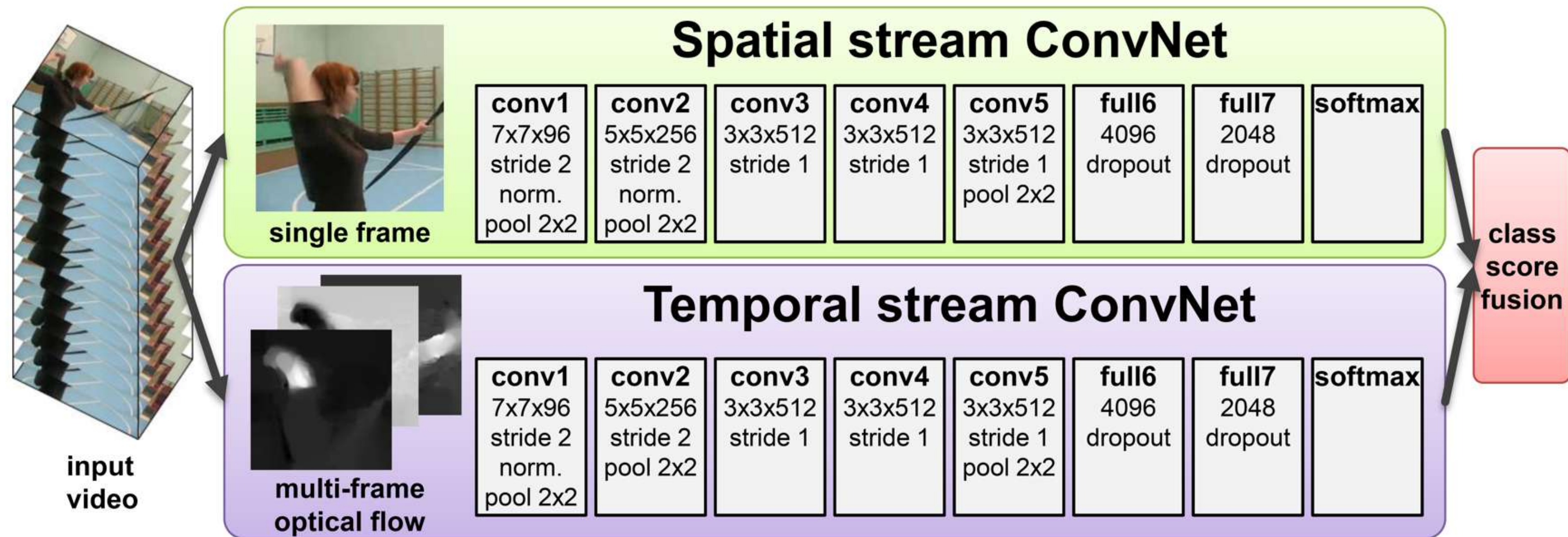
Horizontal flow dx



Vertical Flow dy

Separating Motion and Appearance: Two-Stream Networks

Input: Single Image
 $3 \times H \times W$

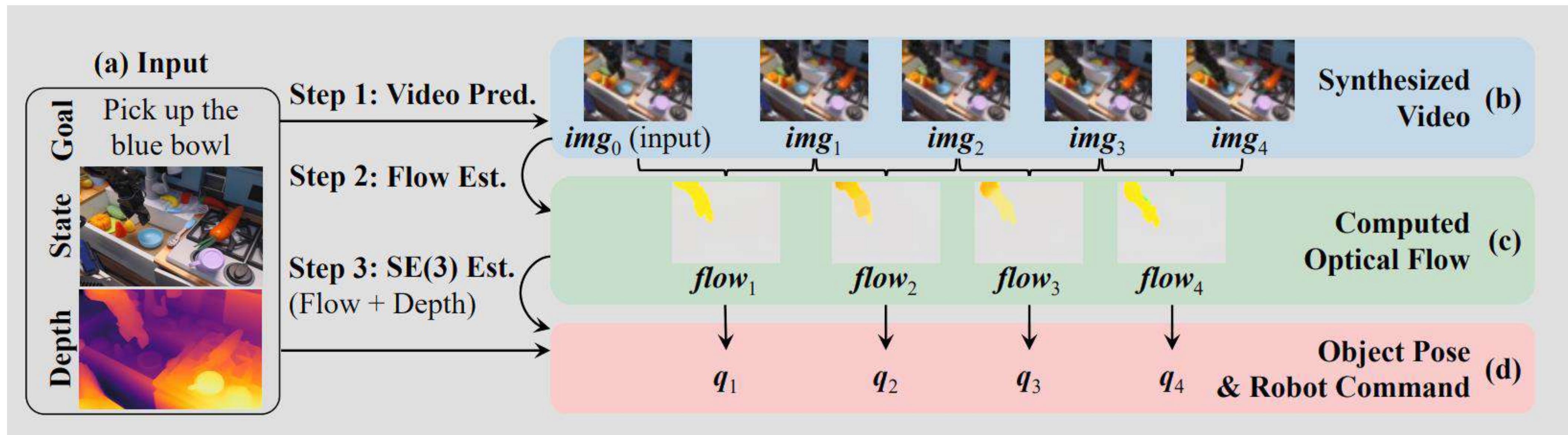


Input: Stack of optical flow:
 $[2 \times (T-1)] \times H \times W$

Early fusion: First 2D conv
processes all flow images

Recently Optical Flow Rarely Used for Video Understanding

Used mainly as an intermediate representation for other tasks
(for example, robotic control)



Trick #2: Inflating 2D Networks to 3D (I3D)

There has been a lot of work on architectures for images.
Can we reuse image architectures for video?

Idea: take a 2D CNN architecture.

Replace each 2D $K_h \times K_w$ conv/pool
layer with a 3D $K_t \times K_h \times K_w$ version

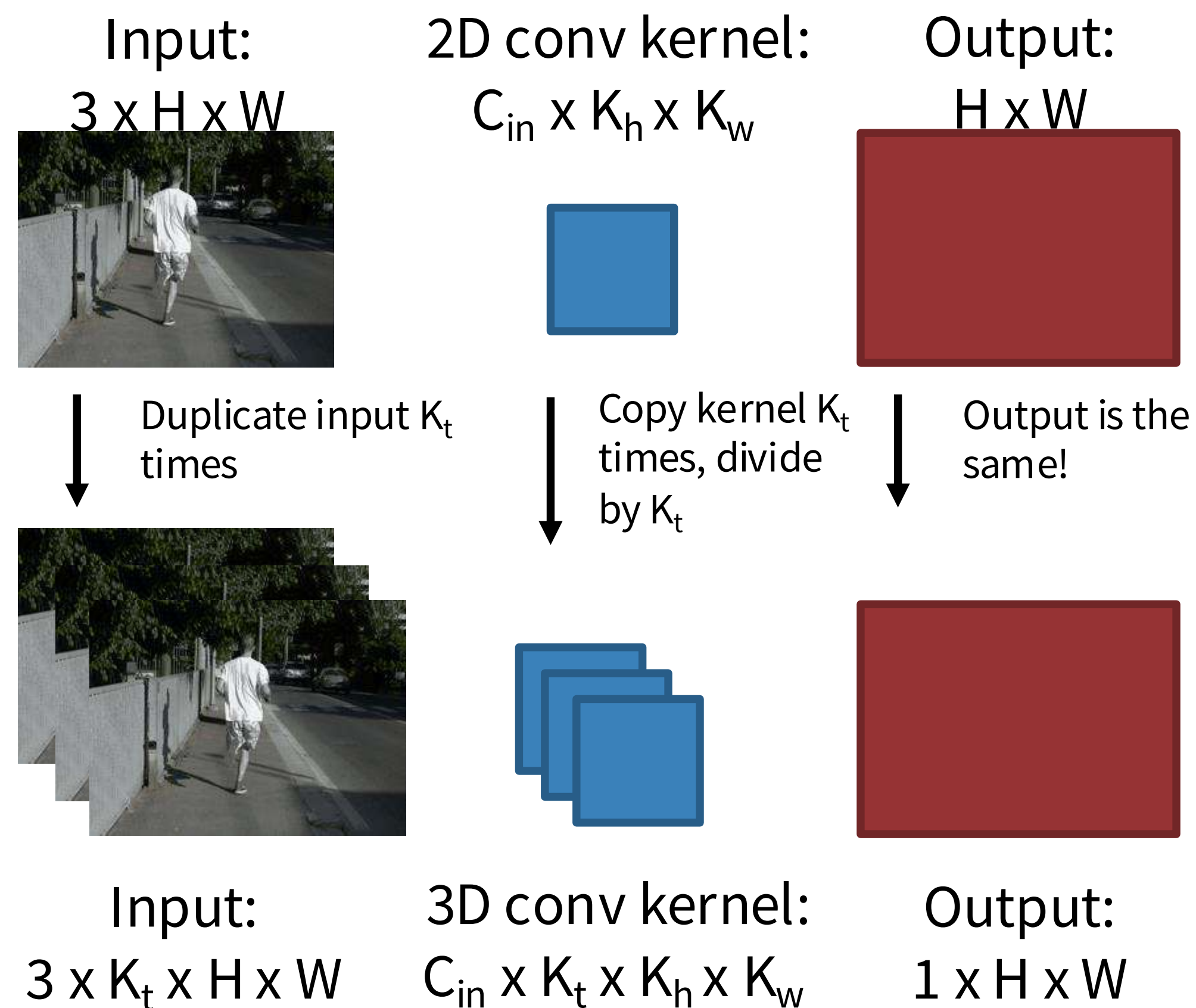
Inflating 2D Networks to 3D (I3D)

There has been a lot of work on architectures for images.
Can we reuse image architectures for video?

Idea: take a 2D CNN architecture.

Replace each 2D $K_h \times K_w$ conv/pool layer with a 3D $K_t \times K_h \times K_w$ version

Can use weights of 2D conv to initialize 3D conv: copy K_t times in space and divide by K_t
This gives the same result as 2D conv given “constant” video input



Inflating 2D Networks to 3D (I3D)

There has been a lot of work on architectures for images.
Can we reuse image architectures for video?

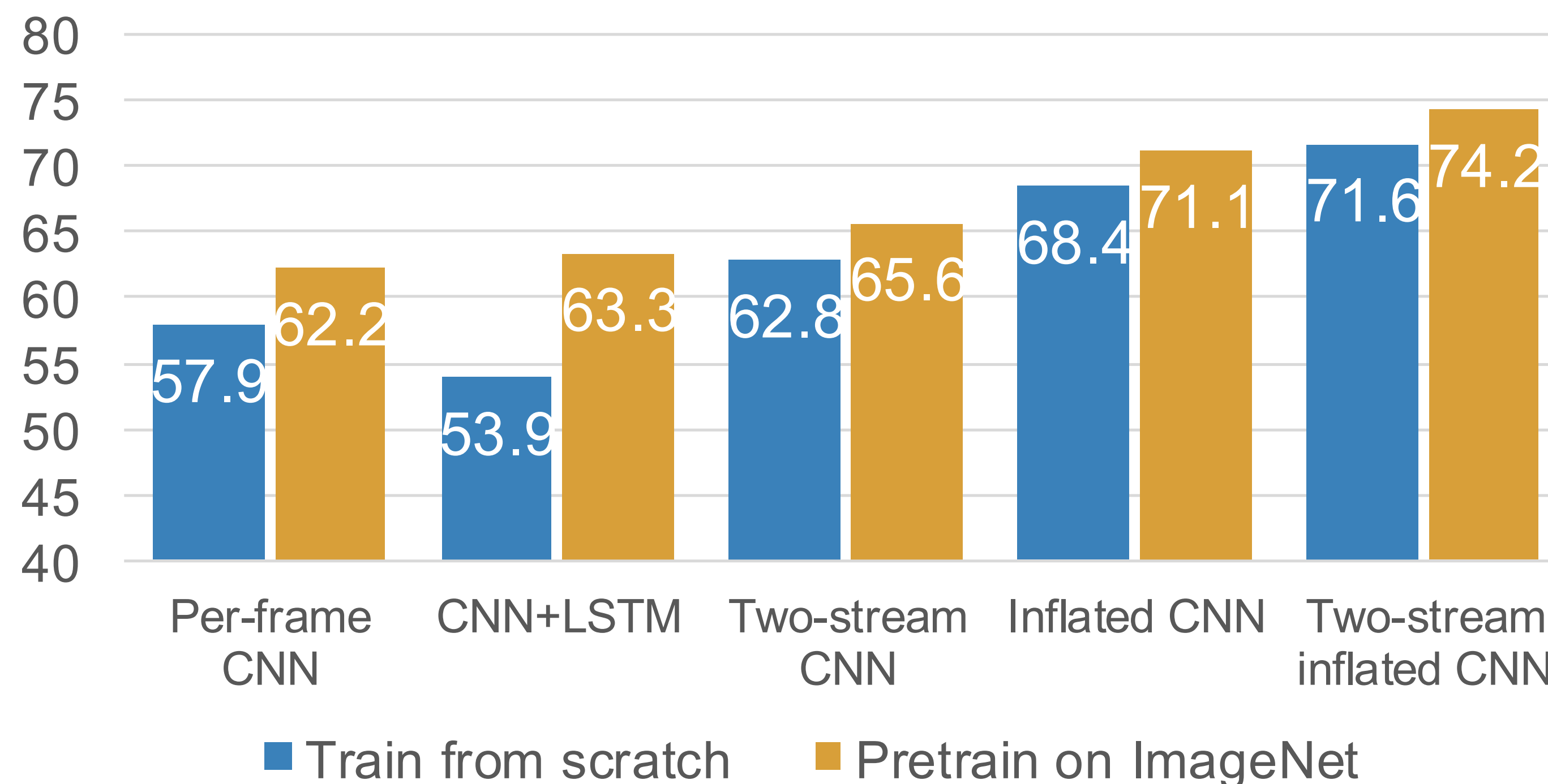
Idea: take a 2D CNN architecture.

Replace each 2D $K_h \times K_w$ conv/pool layer with a 3D $K_t \times K_h \times K_w$ version

Can use weights of 2D conv to initialize 3D conv: copy K_t times in space and divide by K_t

This gives the same result as 2D conv given “constant” video input

Top-1 Accuracy on Kinetics-400



Video Understanding: Evolution of the Field

2014 – 2021 Era of 3D CNNs + RNNs

2021 – 2026 Transformers

Video Understanding: Evolution of the Field

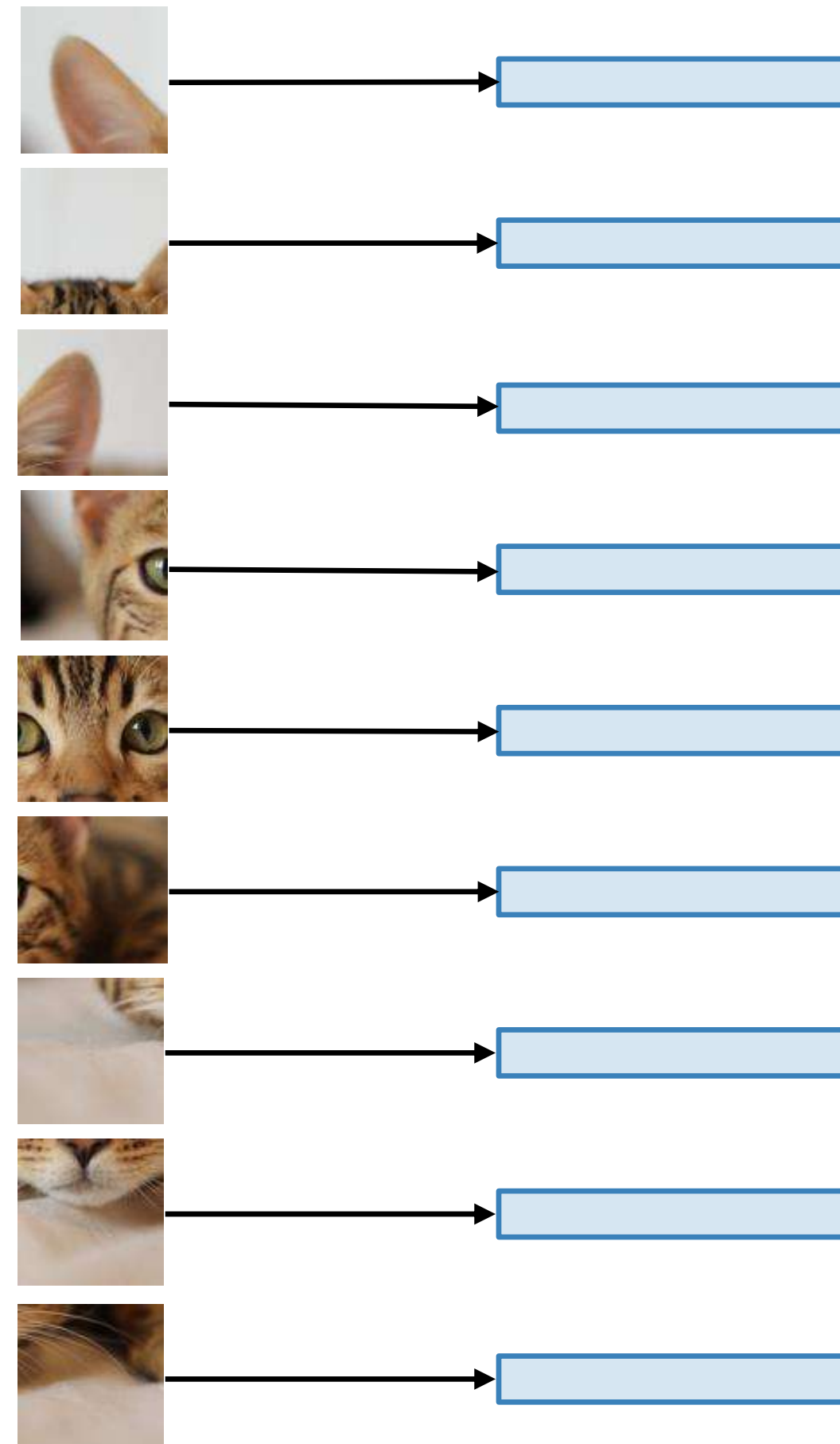
2014 – 2021 Era of 3D CNNs + RNNs

2021 – 2026 Transformers

Image Level: Vision Transformers (ViT)

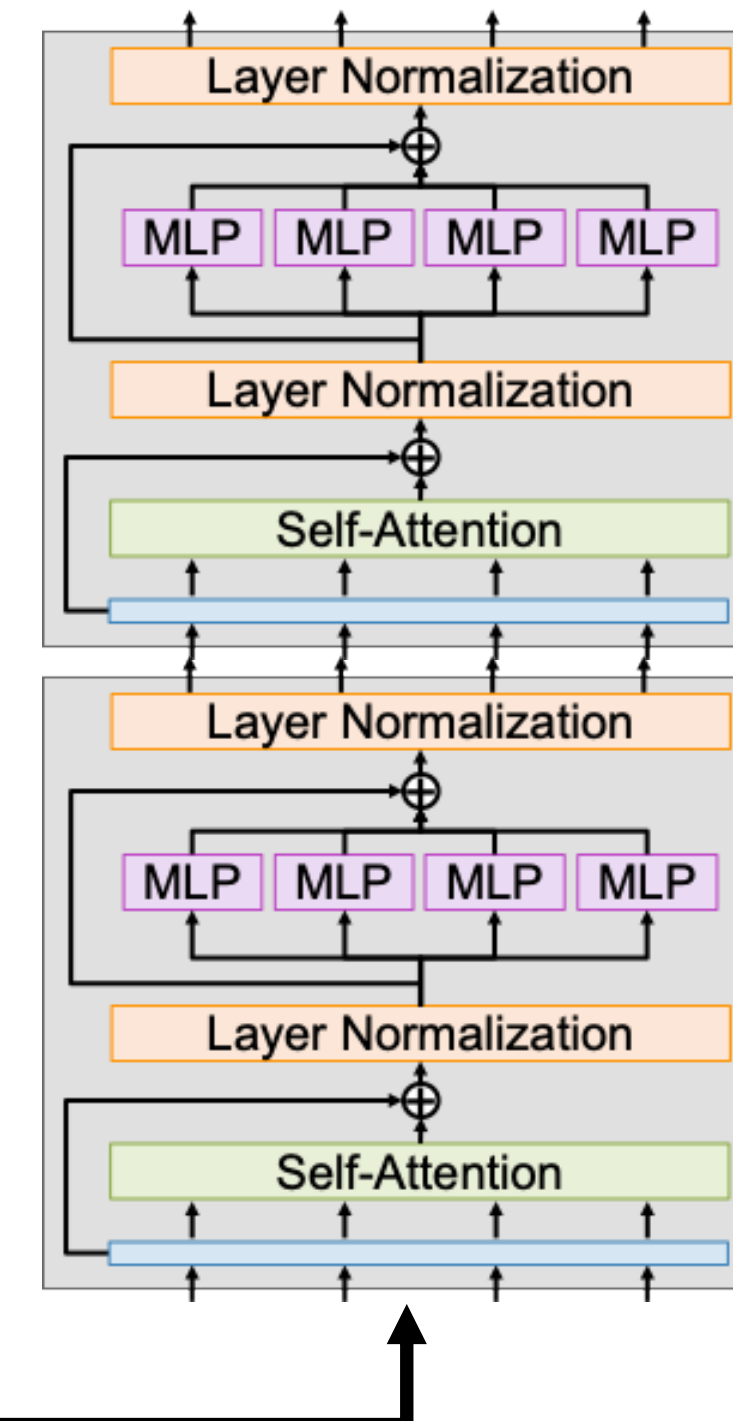


Input image:
e.g. 224x224x3



Break into patches
e.g. 16x16x3

Flatten and apply a linear
transform 768 => D



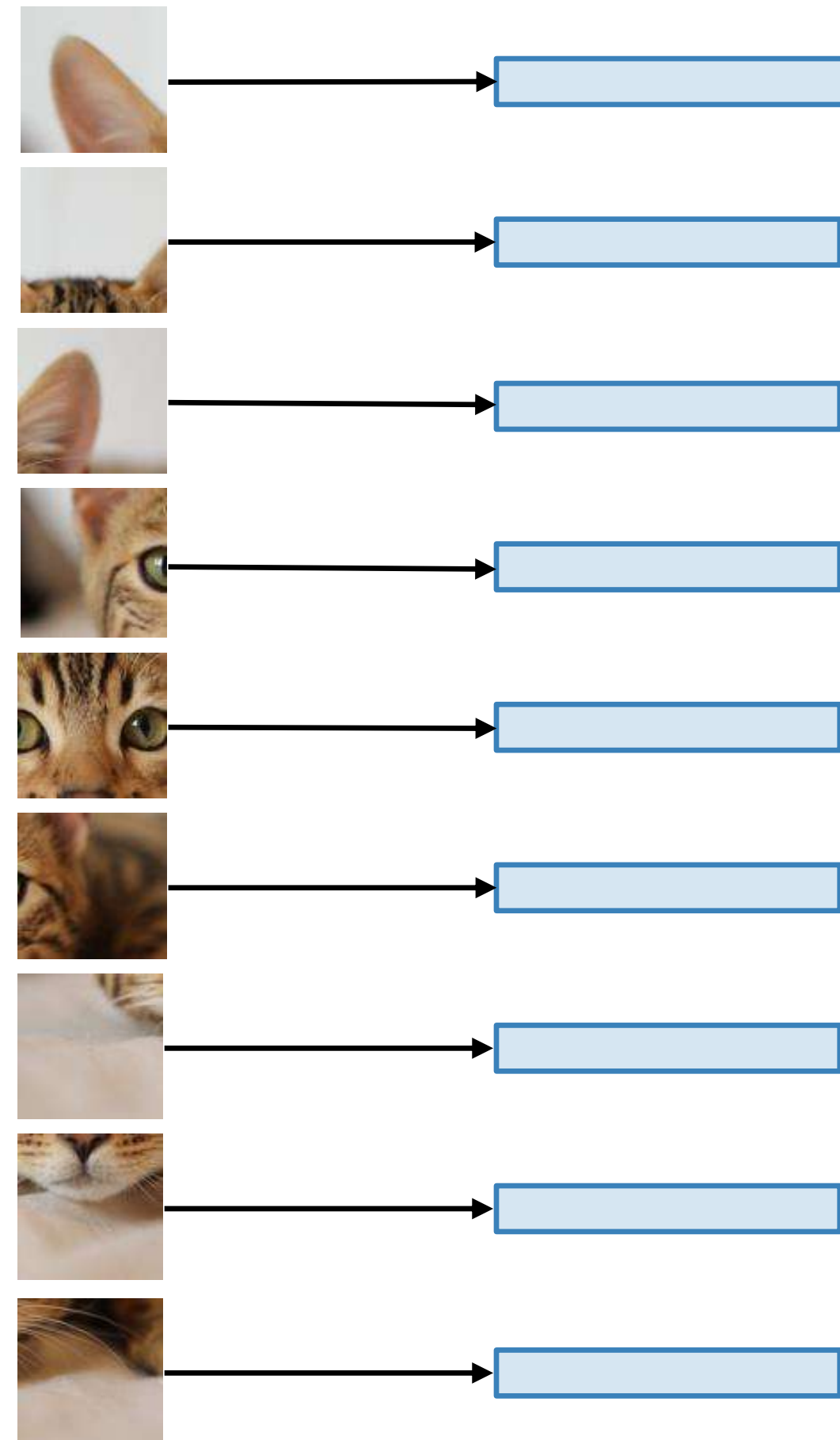
Use positional
encoding to tell
the transformer
the 2D position
of each patch

D-dim vector per patch
are the input vectors to
the Transformer

Video Level: Vision Transformers (ViT)



Input Frame:
e.g. 224x224x3



Break into patches
e.g. 16x16x3

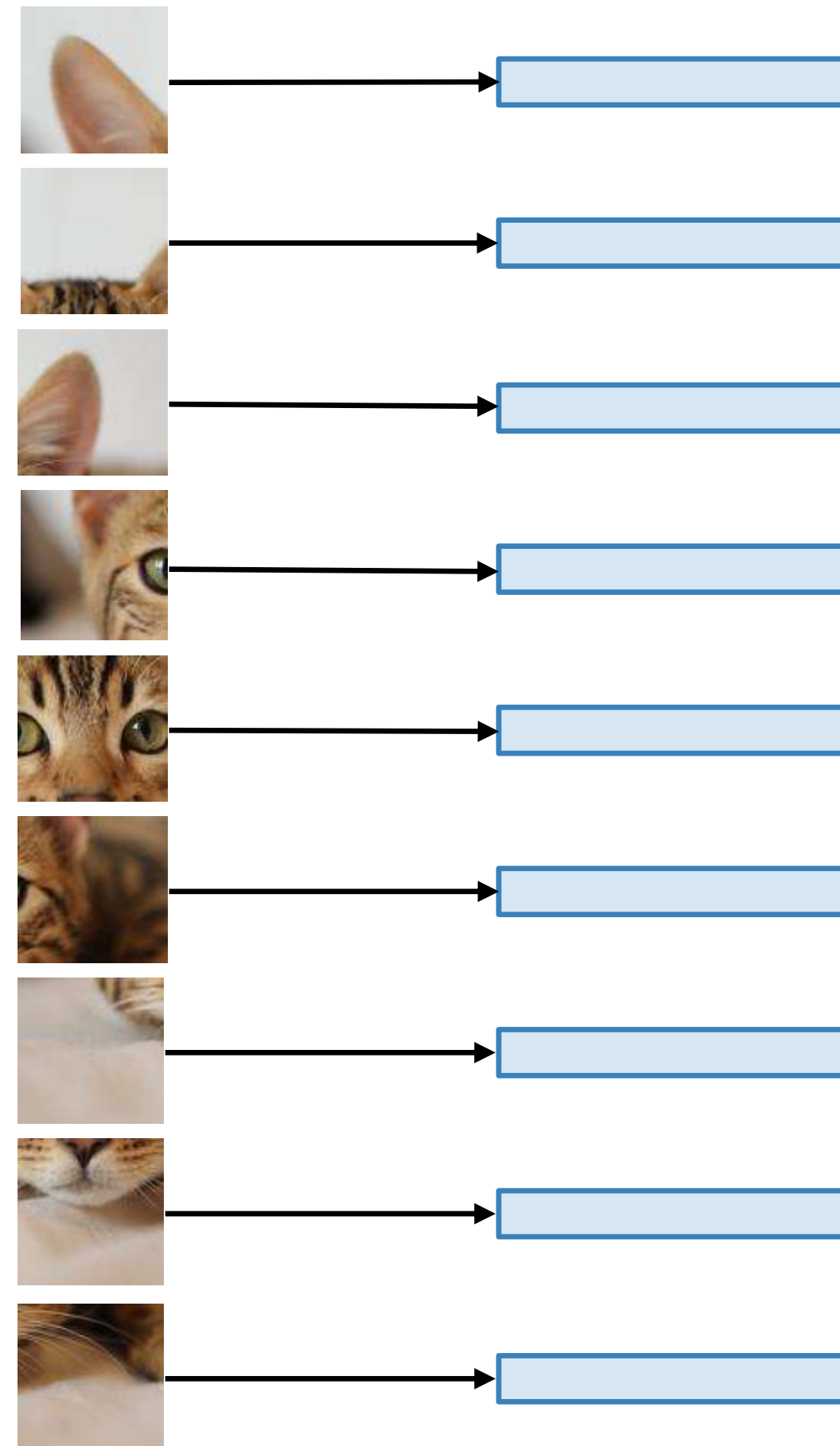
$14 \times 14 = \mathbf{196 \text{ tokens per image}}$. This is a ton!

Small clip: $T = 16 \rightarrow 3136 \text{ tokens}$
~book chapter

Video Level: Vision Transformers (ViT)



Input Frame:
e.g. 224x224x3



Break into patches
e.g. 16x16x3

$14 \times 14 = \mathbf{196 \text{ tokens per image}}$. This is a ton!

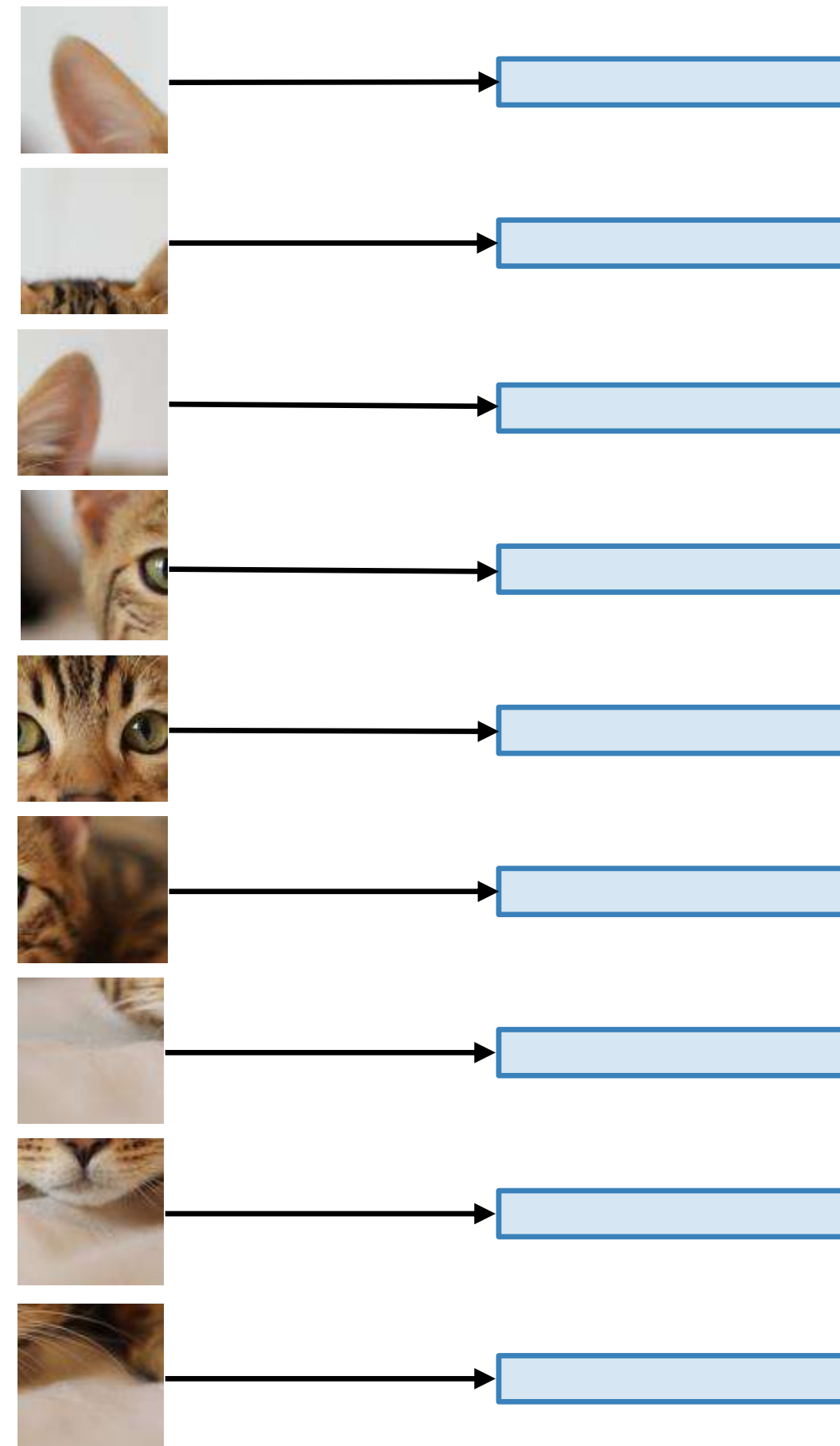
Small clip: $T = 16 \rightarrow 3136 \text{ tokens}$
~book chapter

5 minutes at 1fps: $T = 300 \rightarrow 58.8\text{k tokens}$
~short novel

Video Level: Vision Transformers (ViT)



Input Frame:
e.g. 224x224x3



Break into patches
e.g. 16x16x3

$14 \times 14 = \mathbf{196 \text{ tokens per image}}$. This is a ton!

Small clip: $T = 16 \rightarrow 3136 \text{ tokens}$
~book chapter

5 minutes at 1fps: $T = 300 \rightarrow 58.8\text{k tokens}$
~short novel

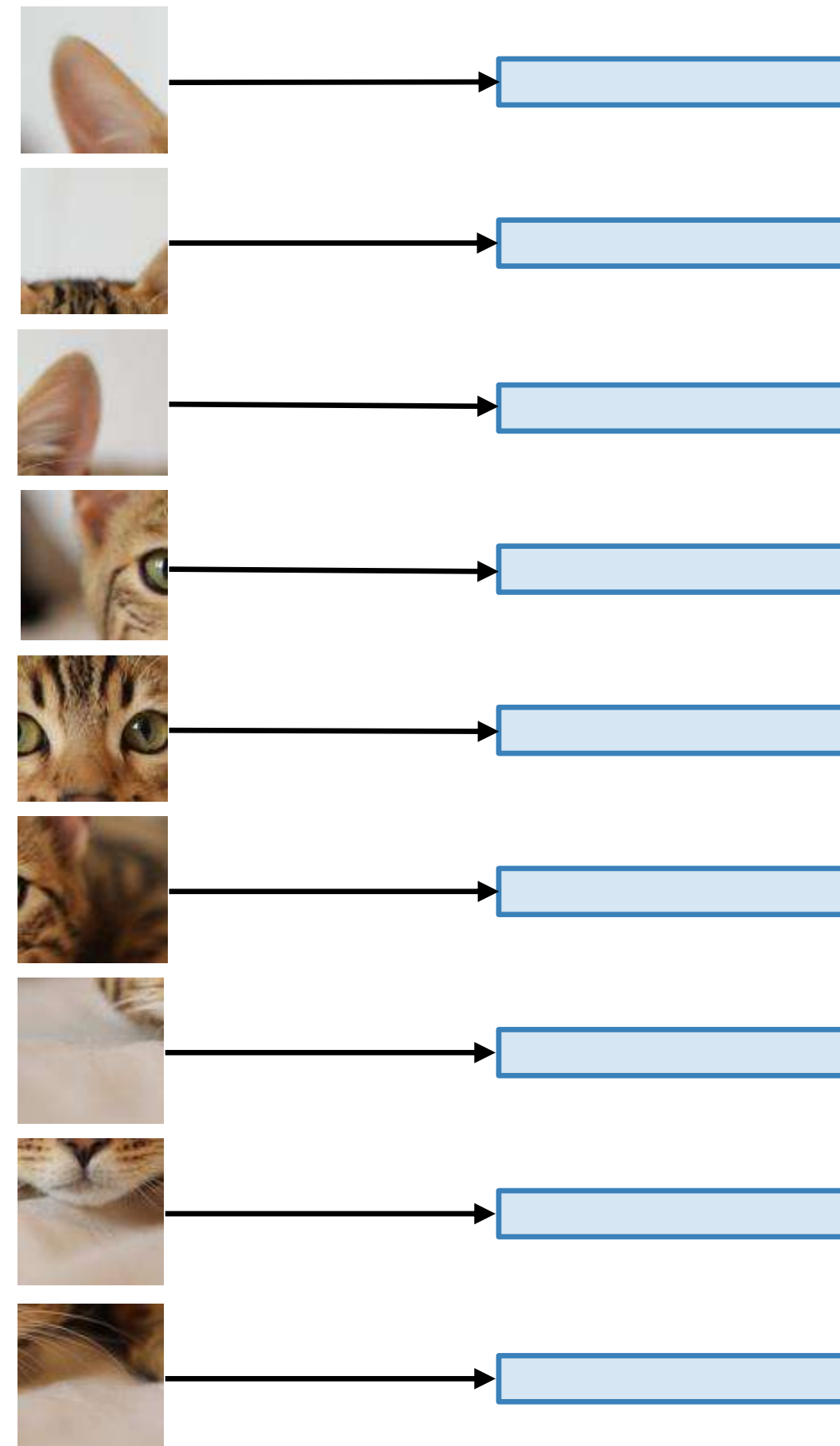
5 minutes at 24fps $\rightarrow 1,411,200 \text{ tokens}$
(~context limit of most modern LLMs)

Compare to human perception speed...

Video Level: Vision Transformers (ViT)



Input Frame:
e.g. 224x224x3



Break into patches
e.g. 16x16x3

$14 \times 14 = \mathbf{196 \text{ tokens per image}}$. This is a ton!

Small clip: $T = 16 \rightarrow 3136$ tokens
~book chapter

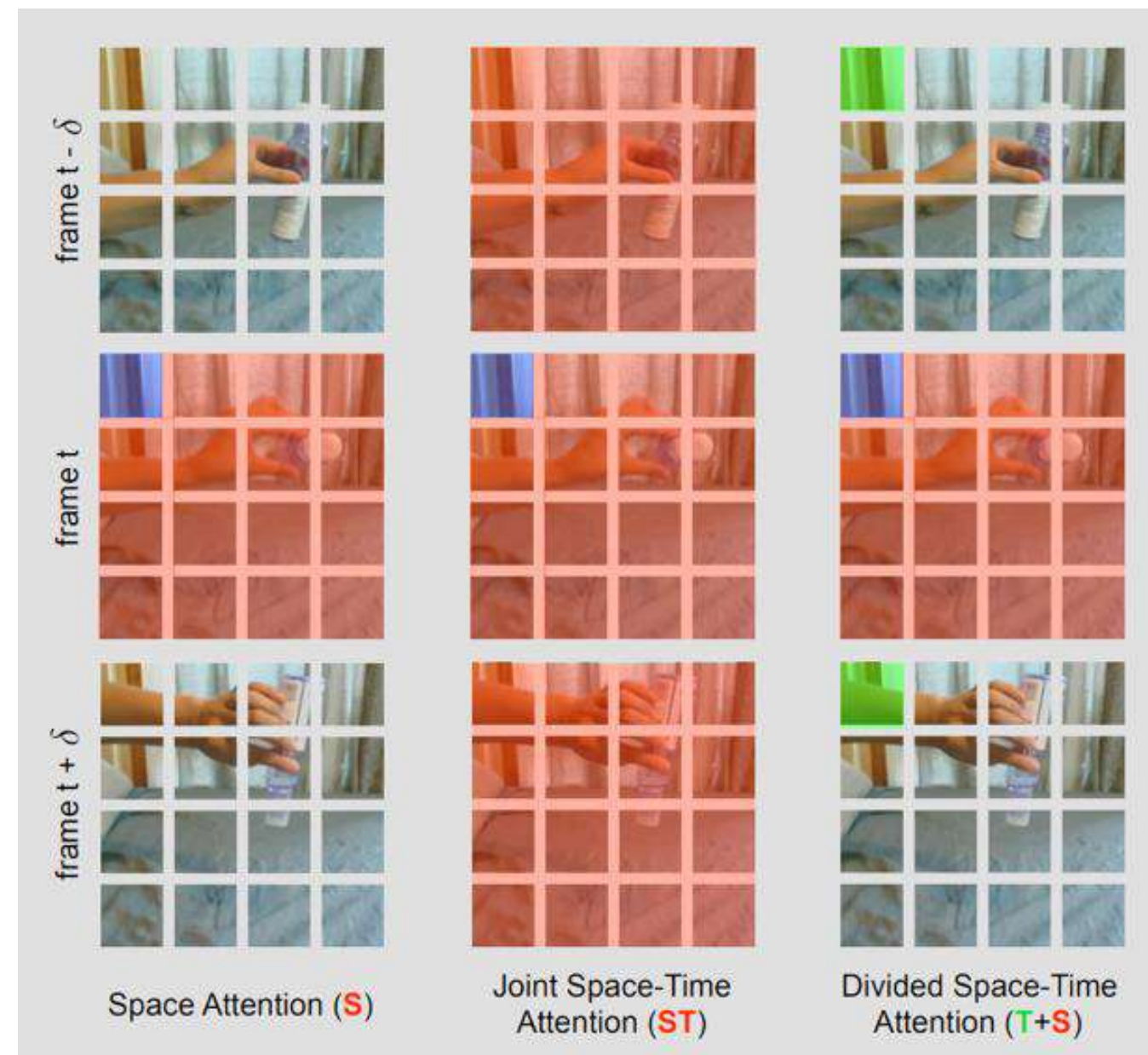
5 minutes at 1fps: $T = 300 \rightarrow 58.8\text{k}$ tokens
~short novel

5 minutes at 24fps $\rightarrow 1,411,200$ tokens
(~context limit of most modern LLMs)

Naïve patch-level attention scales very poorly! How to solve this?

Two Broad Strategies for Improving Video Efficiency

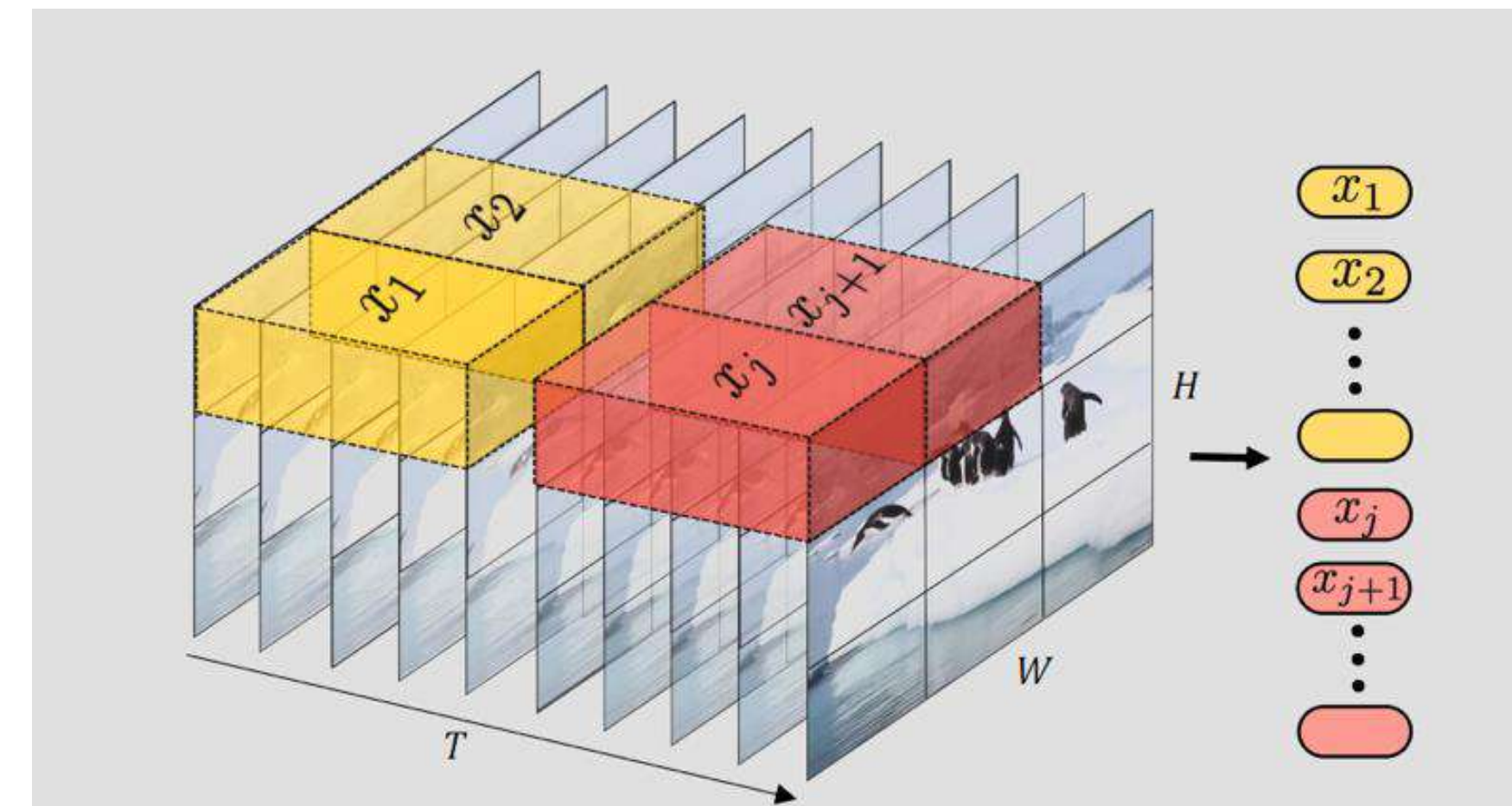
Modify attention operator



Bertasius et al, "Is Space-Time Attention All You Need for Video Understanding?", ICML 2021

Liu, Ze, et al. "Video Swin Transformer" CVPR 2022.

Reduce the number of tokens



Arnab et al, "ViViT: A Video Vision Transformer", ICCV 2021

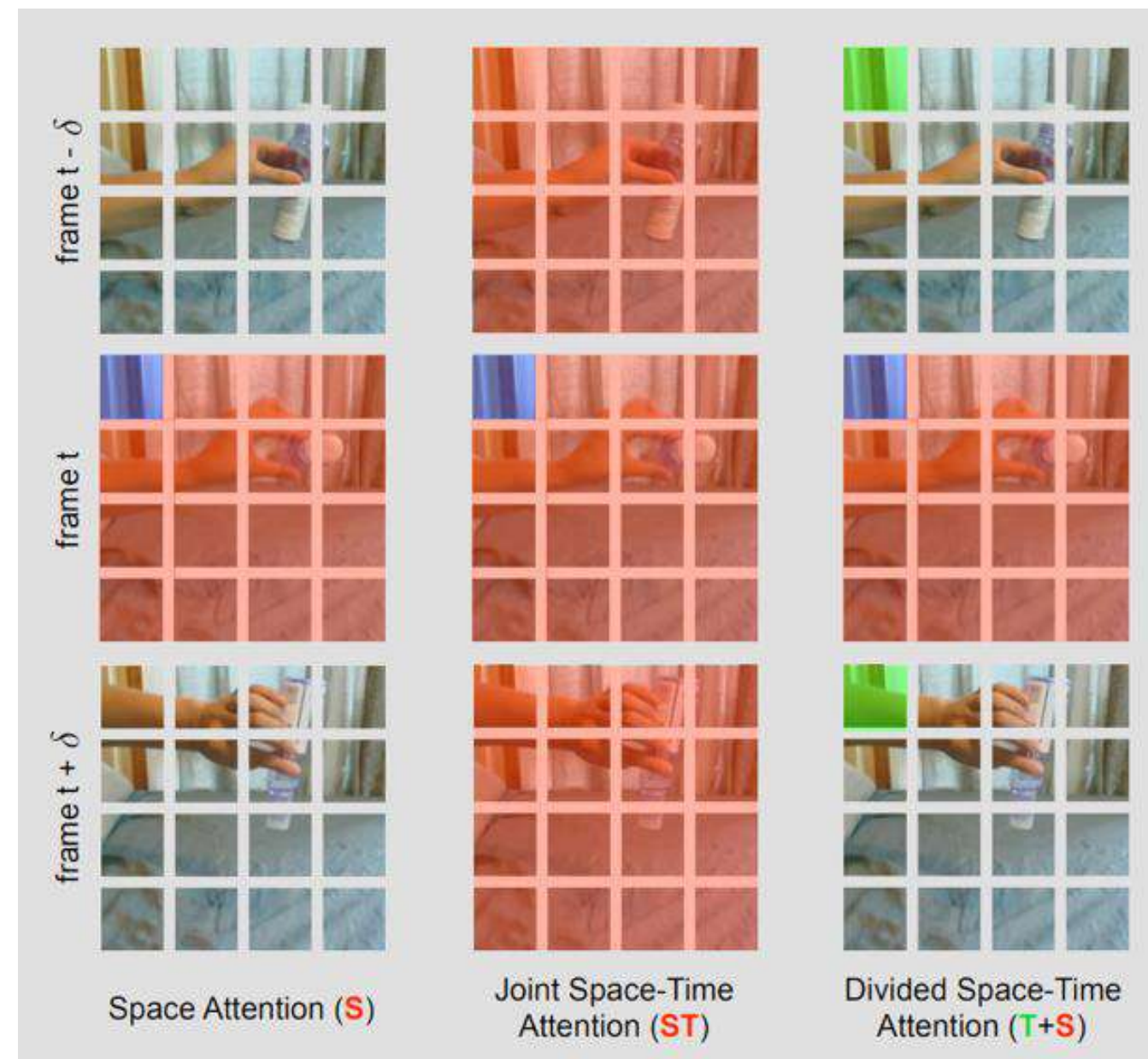
Fan et al, "Multiscale Vision Transformers", ICCV 2021

Li et al, "MViTv2: Improved Multiscale Vision Transformers for Classification and Detection", CVPR 2022

Many different approaches for both -- we will only cover a couple in each category today!

Two Broad Strategies for Improving Video Efficiency

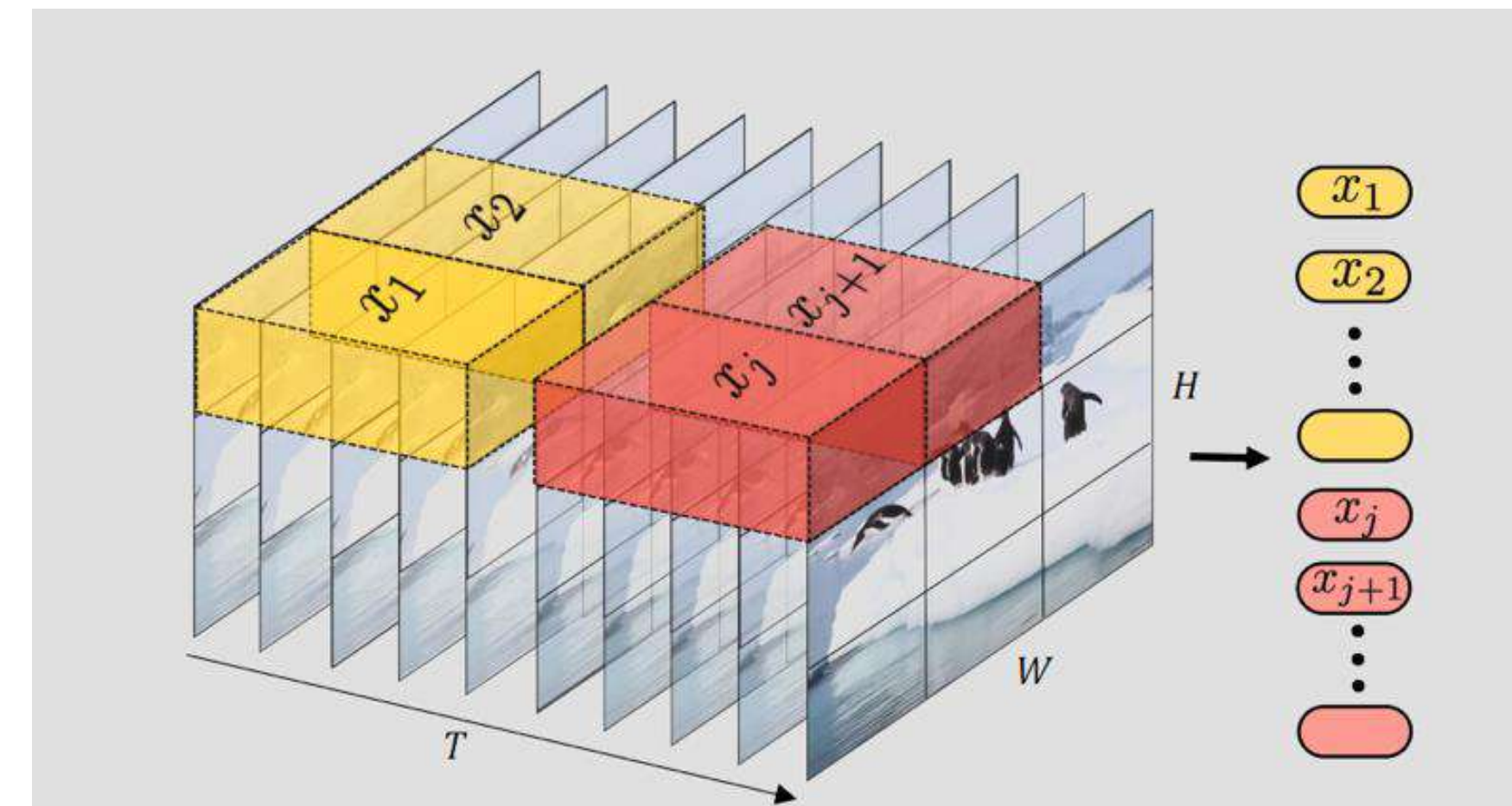
Modify attention operator



Bertasius et al, "Is Space-Time Attention All You Need for Video Understanding?", ICML 2021

Liu, Ze, et al. "Video Swin Transformer" CVPR 2022.

Reduce the number of tokens

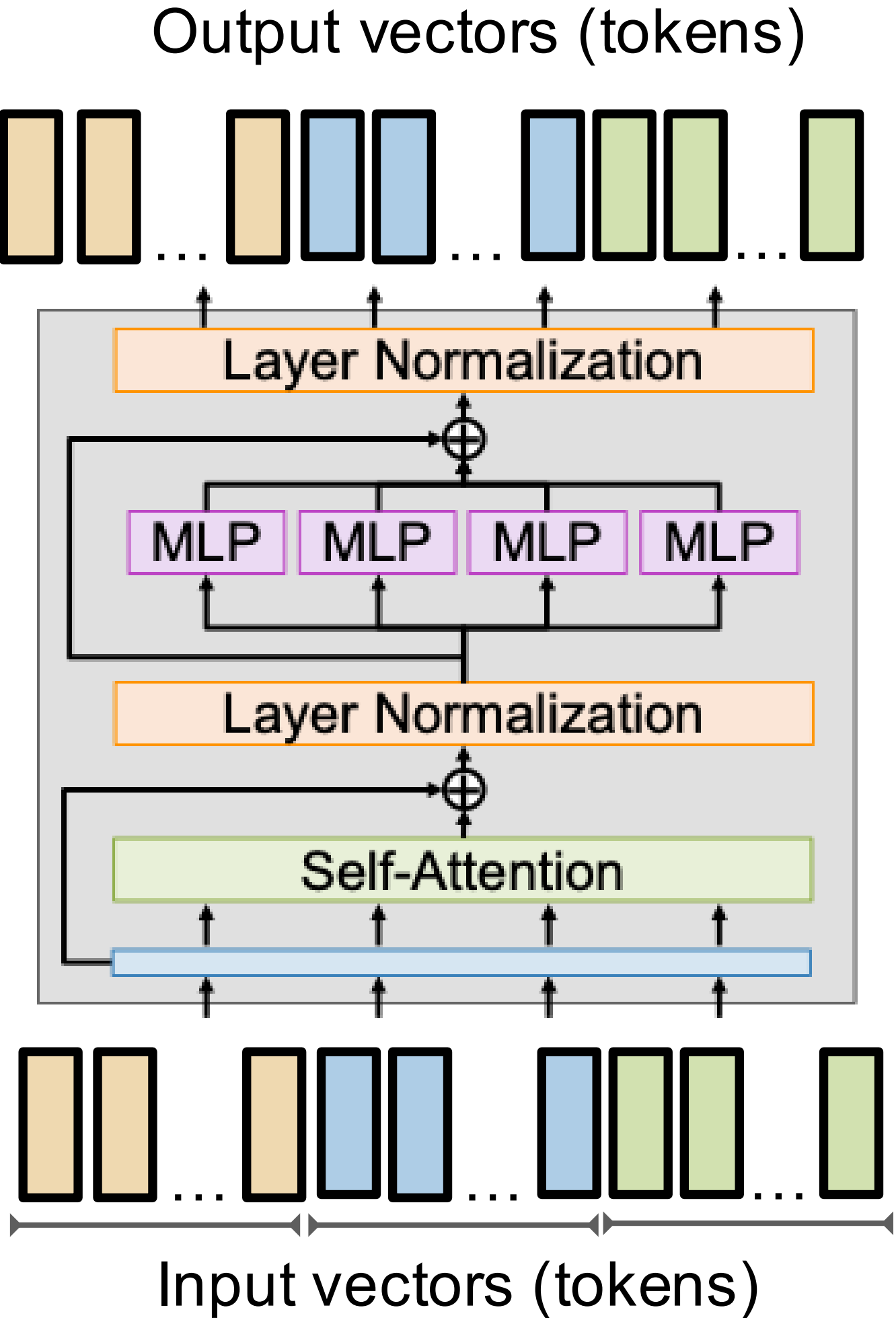


Arnab et al, "ViViT: A Video Vision Transformer", ICCV 2021

Fan et al, "Multiscale Vision Transformers", ICCV 2021

Li et al, "MViTv2: Improved Multiscale Vision Transformers for Classification and Detection", CVPR 2022

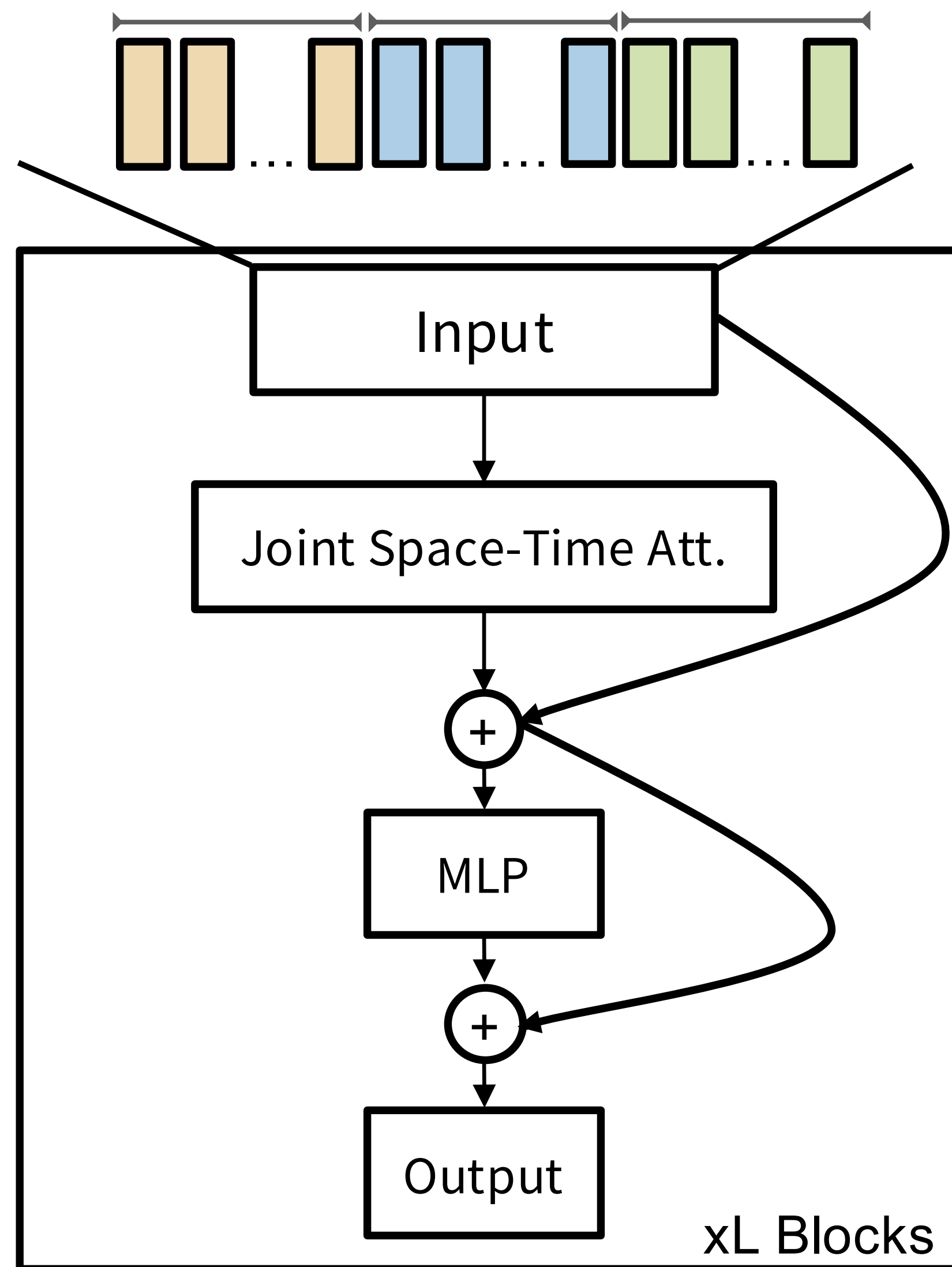
Recall: Transformer Block



Assume fixed N patches per frame as before (3 shown).

Figures modified from: Bertasius et al, “Is Space-Time Attention All You Need for Video Understanding?”, ICML 2021

Joint Space-Time Attention

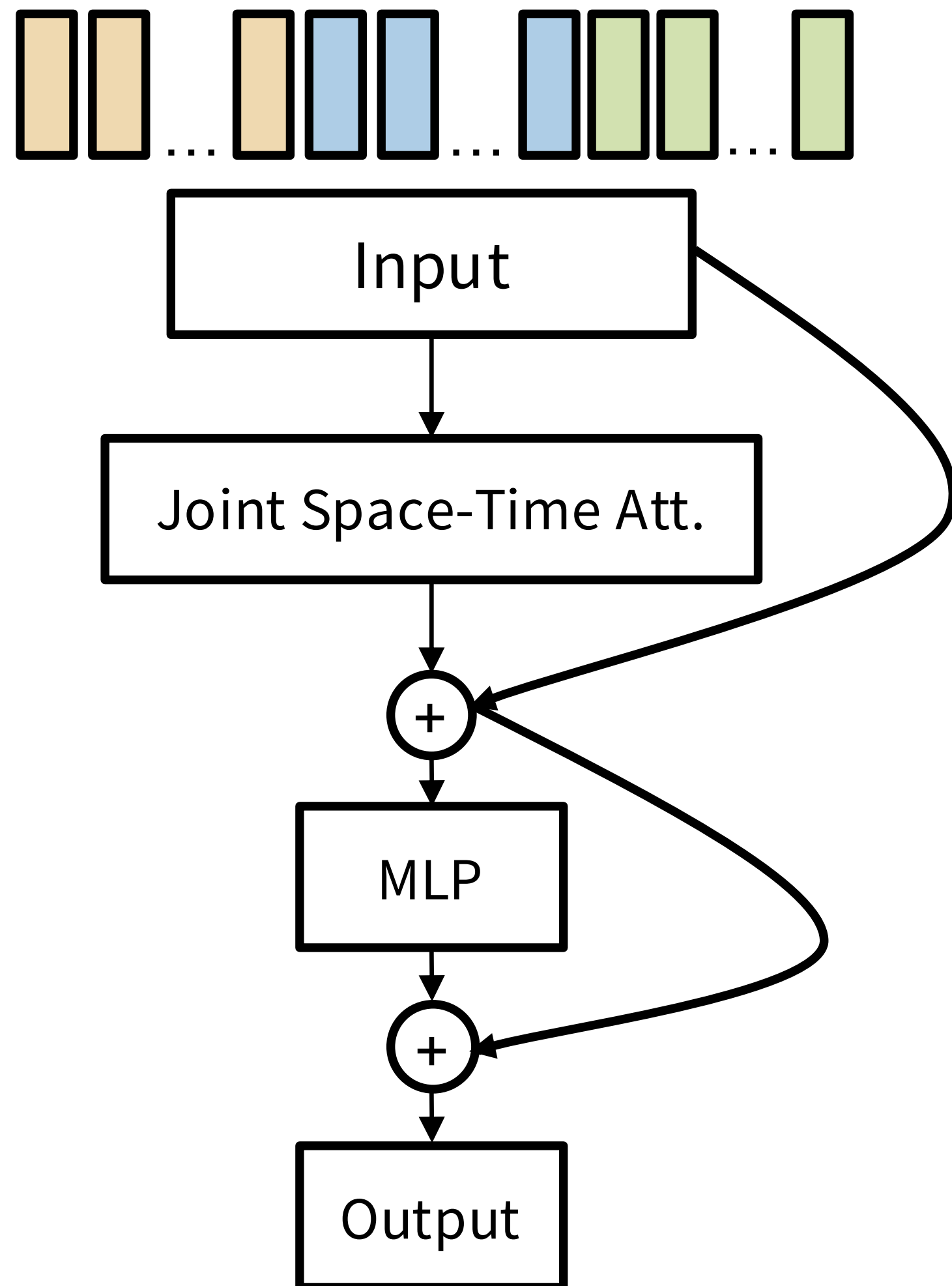


Extremely inefficient!

Recall: 5 minutes at 24fps $\rightarrow$ 1,411,200 tokens

For simplicity, normalization layers not shown in diagram.

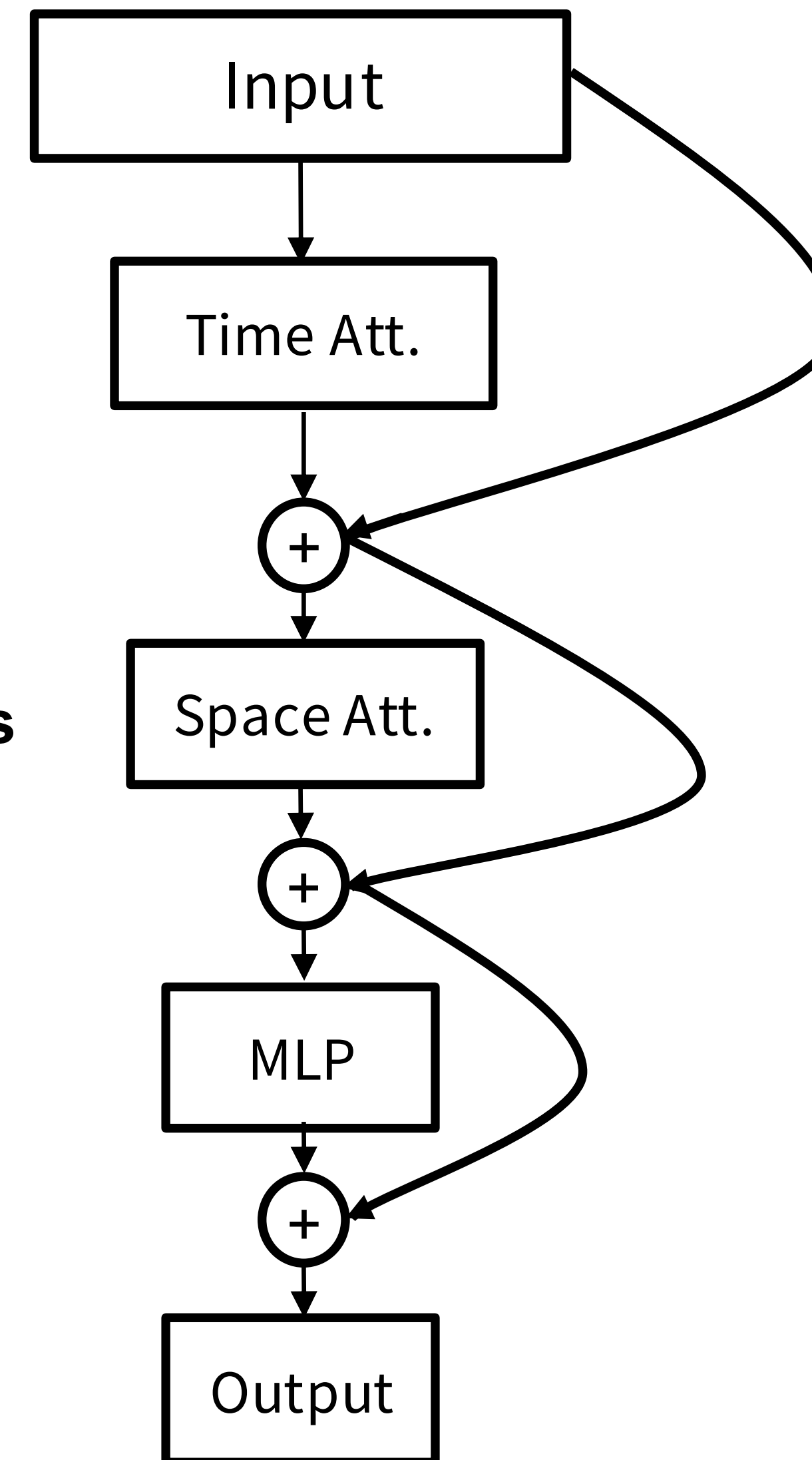
Joint vs Divided Space-Time Attention



Key Idea: We can Divide Time and Space Attention Operators into 2 steps

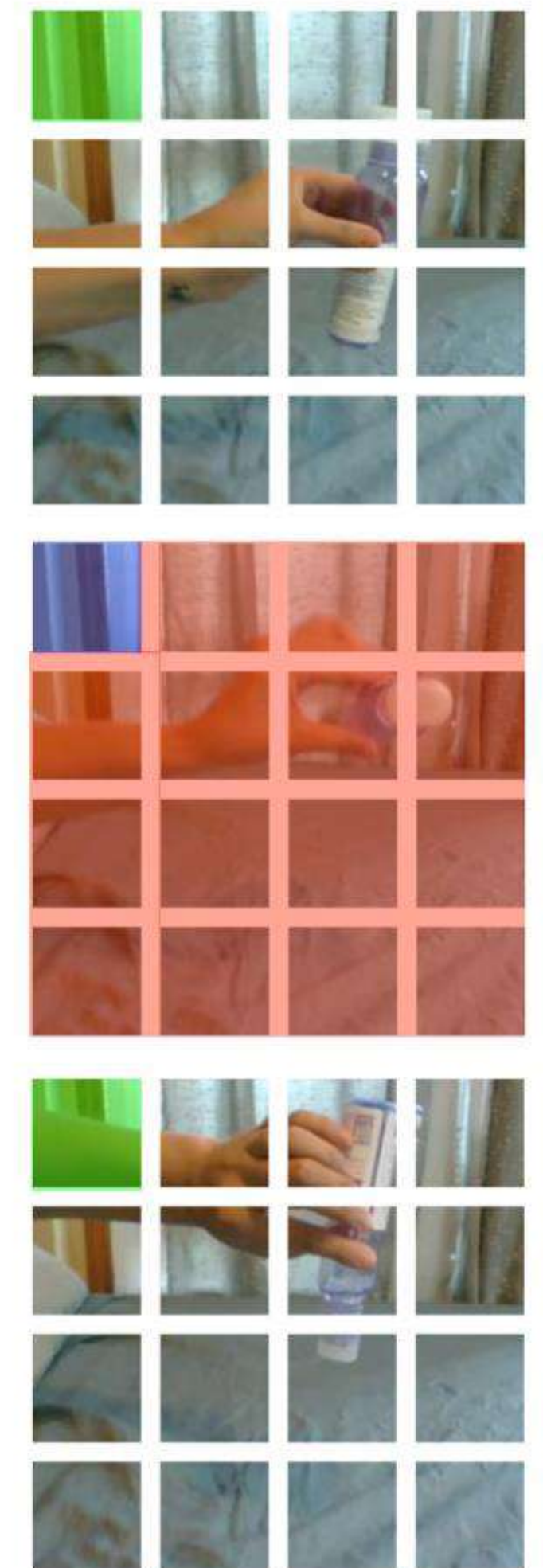
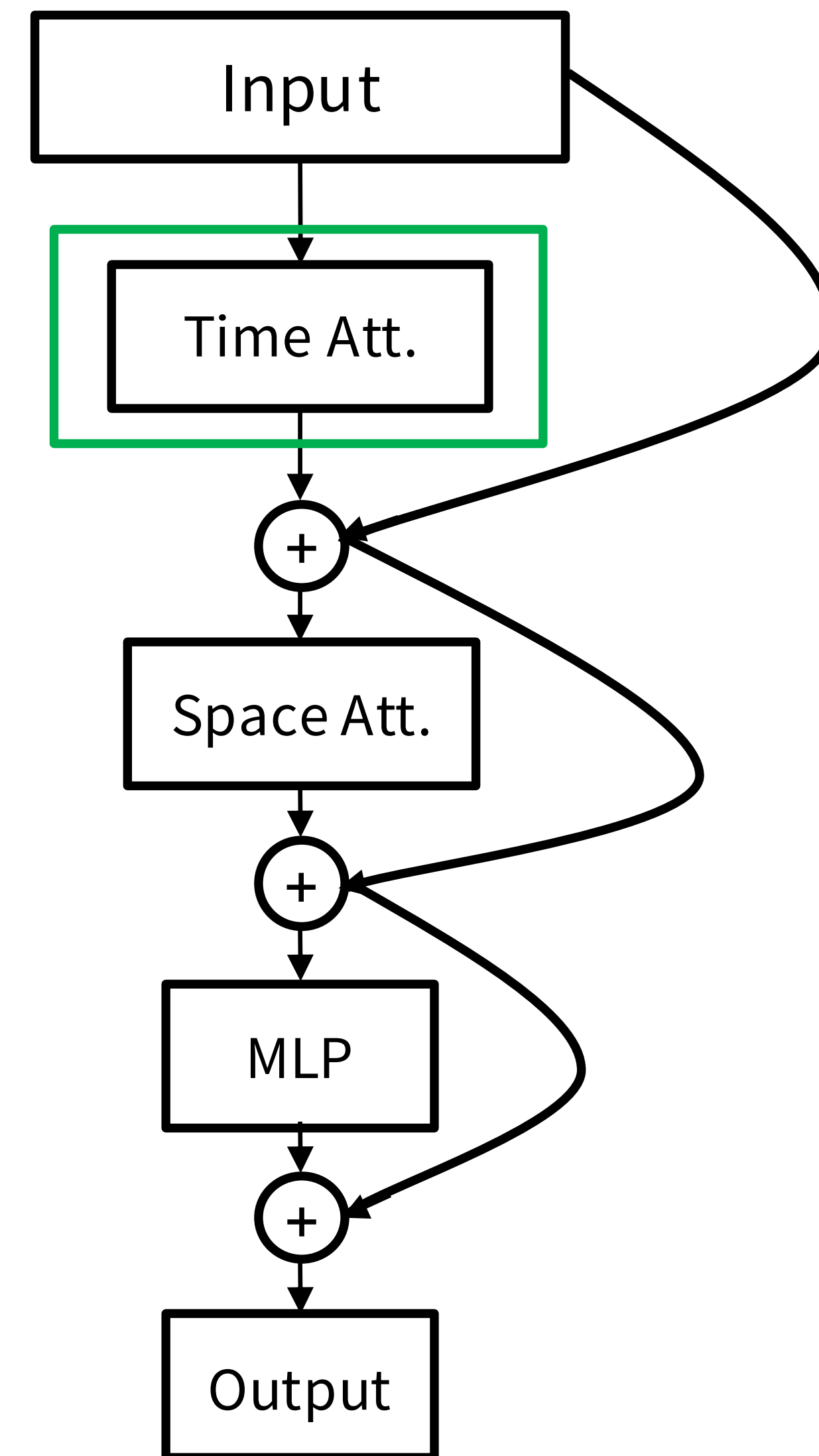
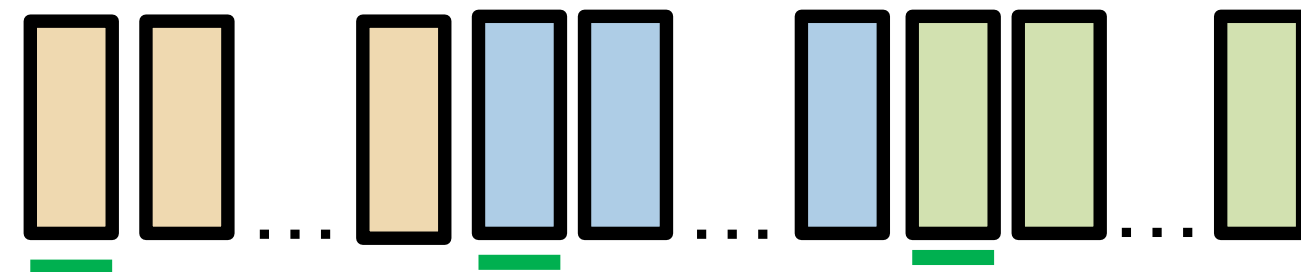


How will this help? Each can operate on a different set of tokens



Divided Space-Time Attention

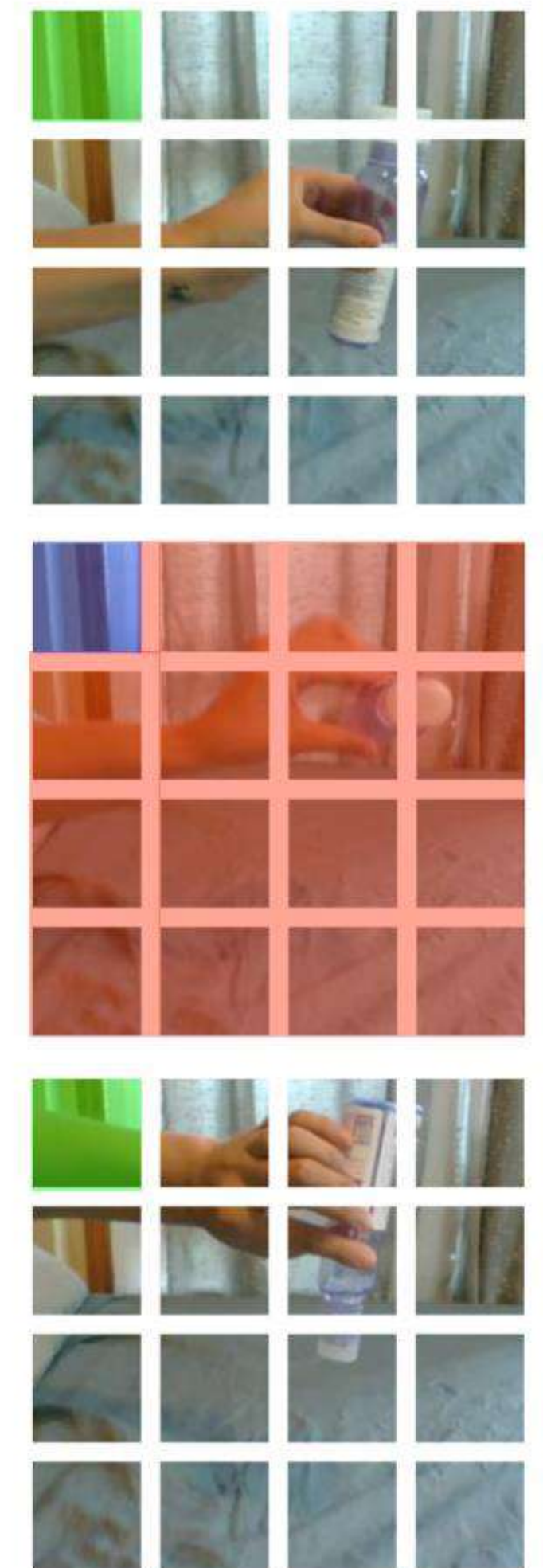
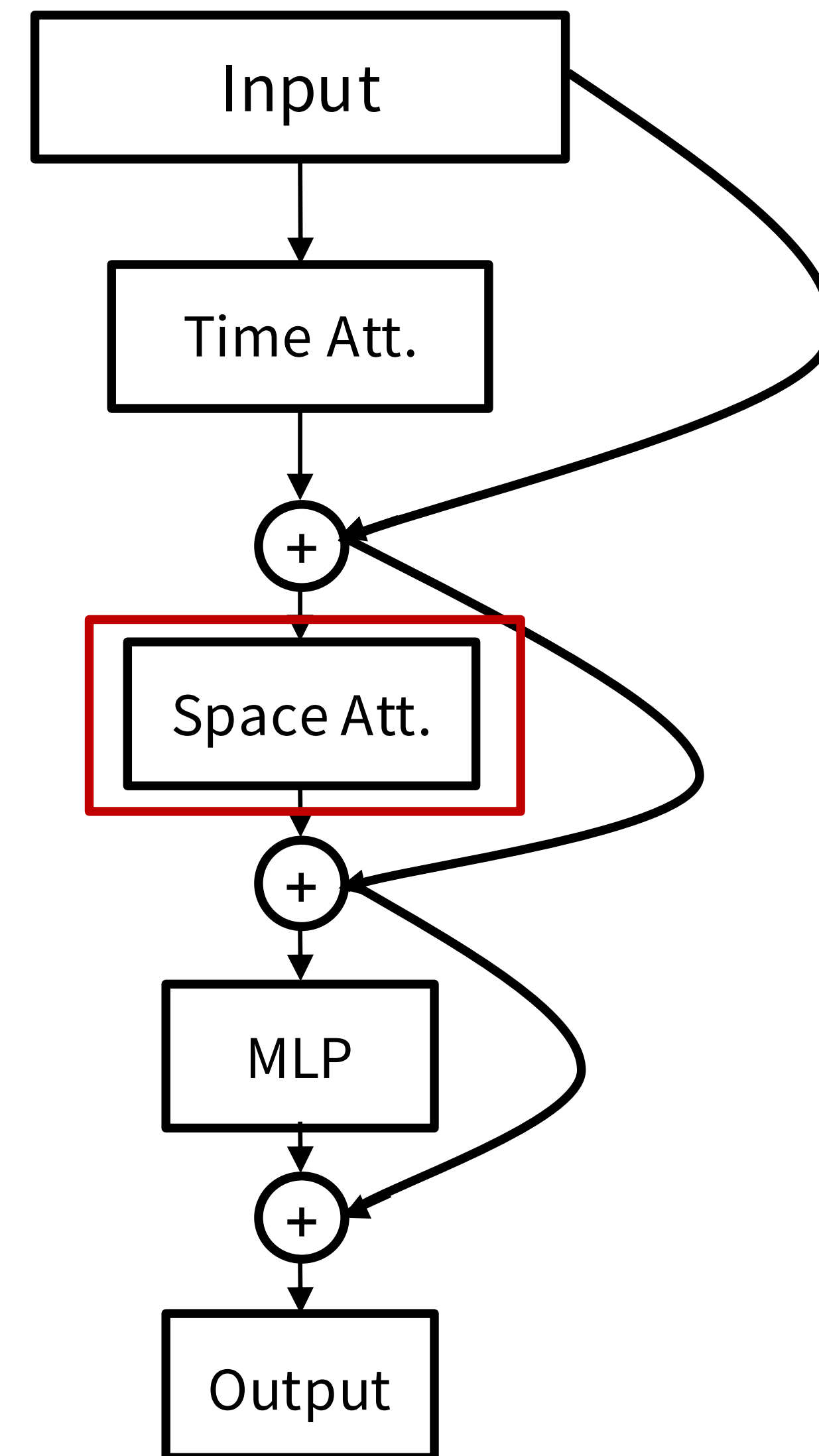
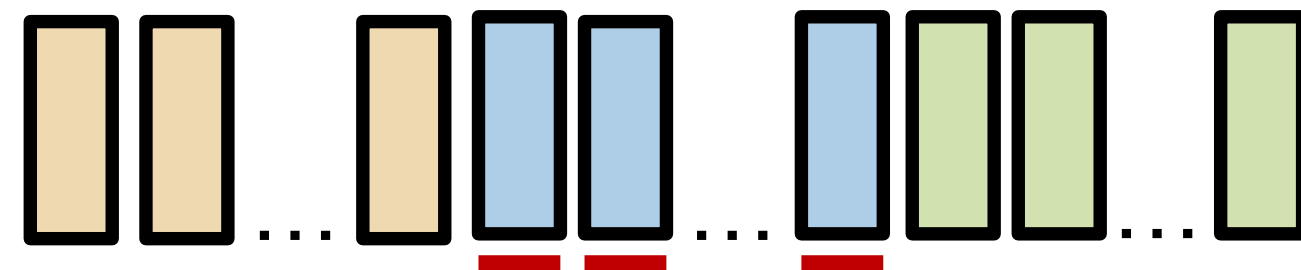
During “Time” Attention Operation, each token attends to the same spatial location across different frames.



Divided Space-Time Attention (**T**+**S**)

Divided **Space**-Time Attention

During “**Space**” Attention Operation, each token attends to other tokens within the same frame

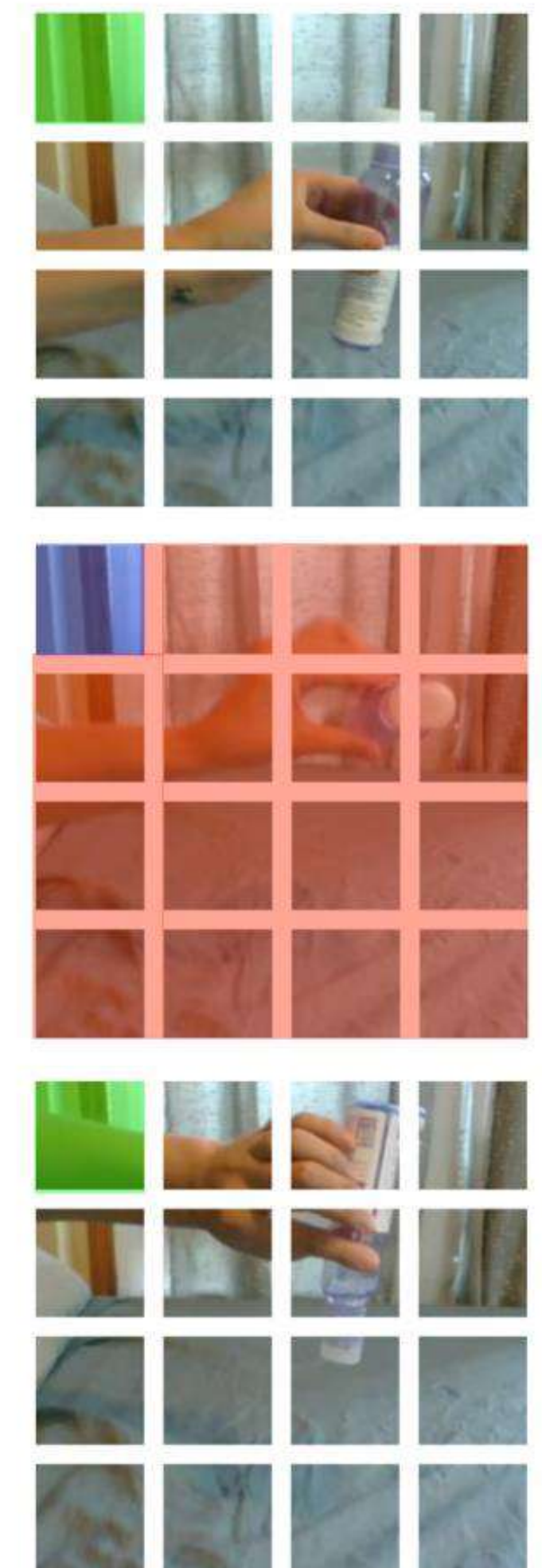
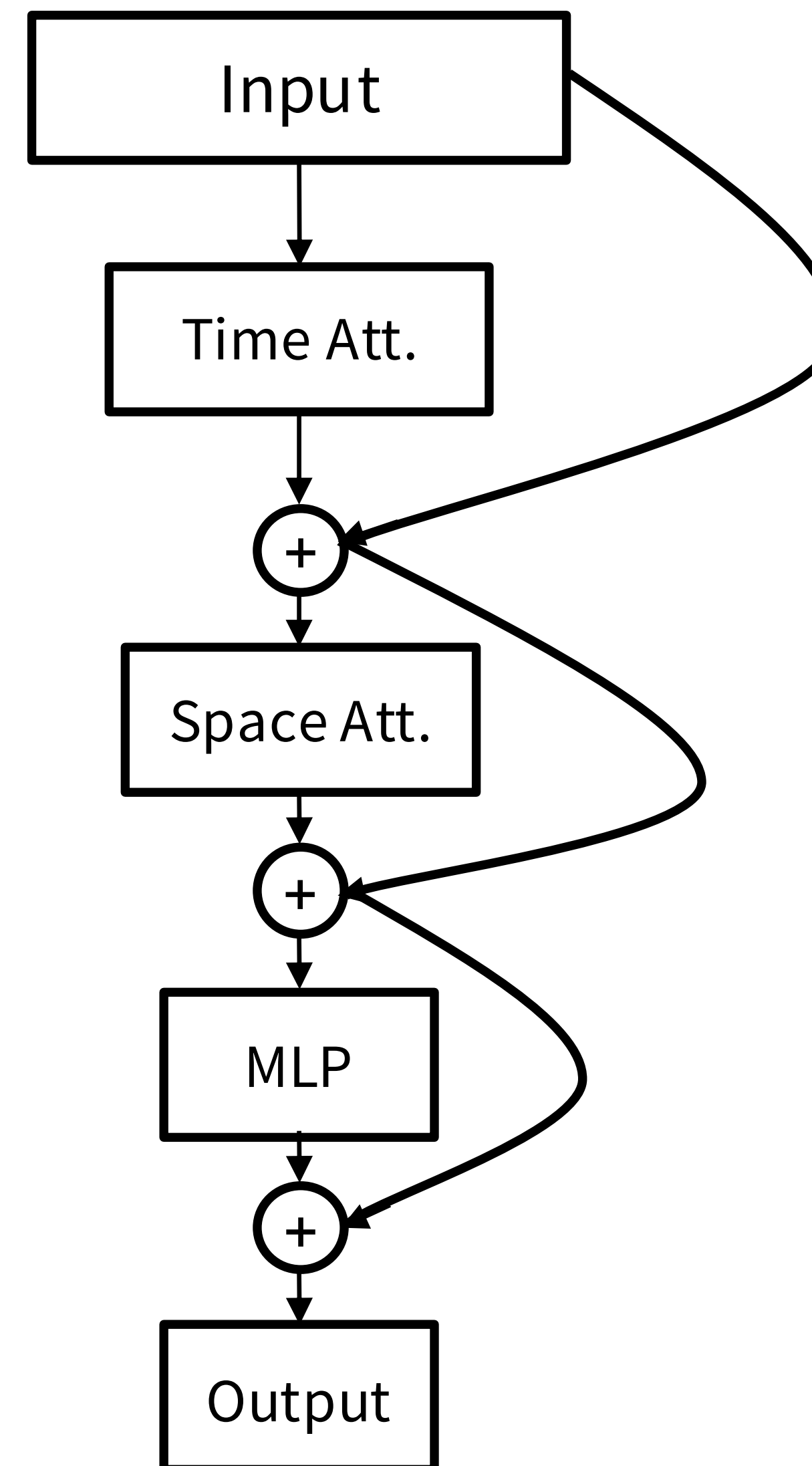
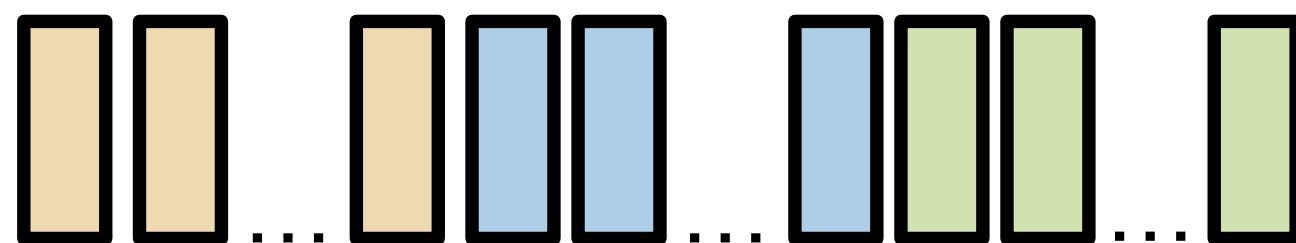


Divided Space-Time Attention (**T**+**S**)

Divided Space Time Attention

Why does this work well?

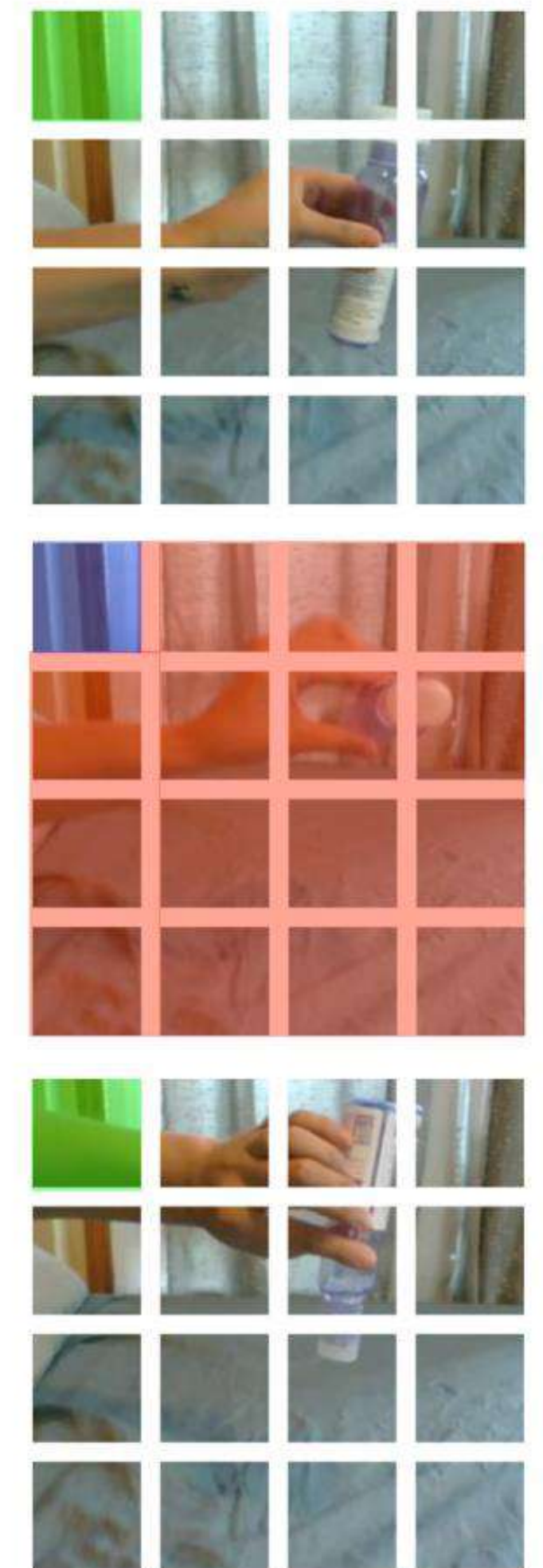
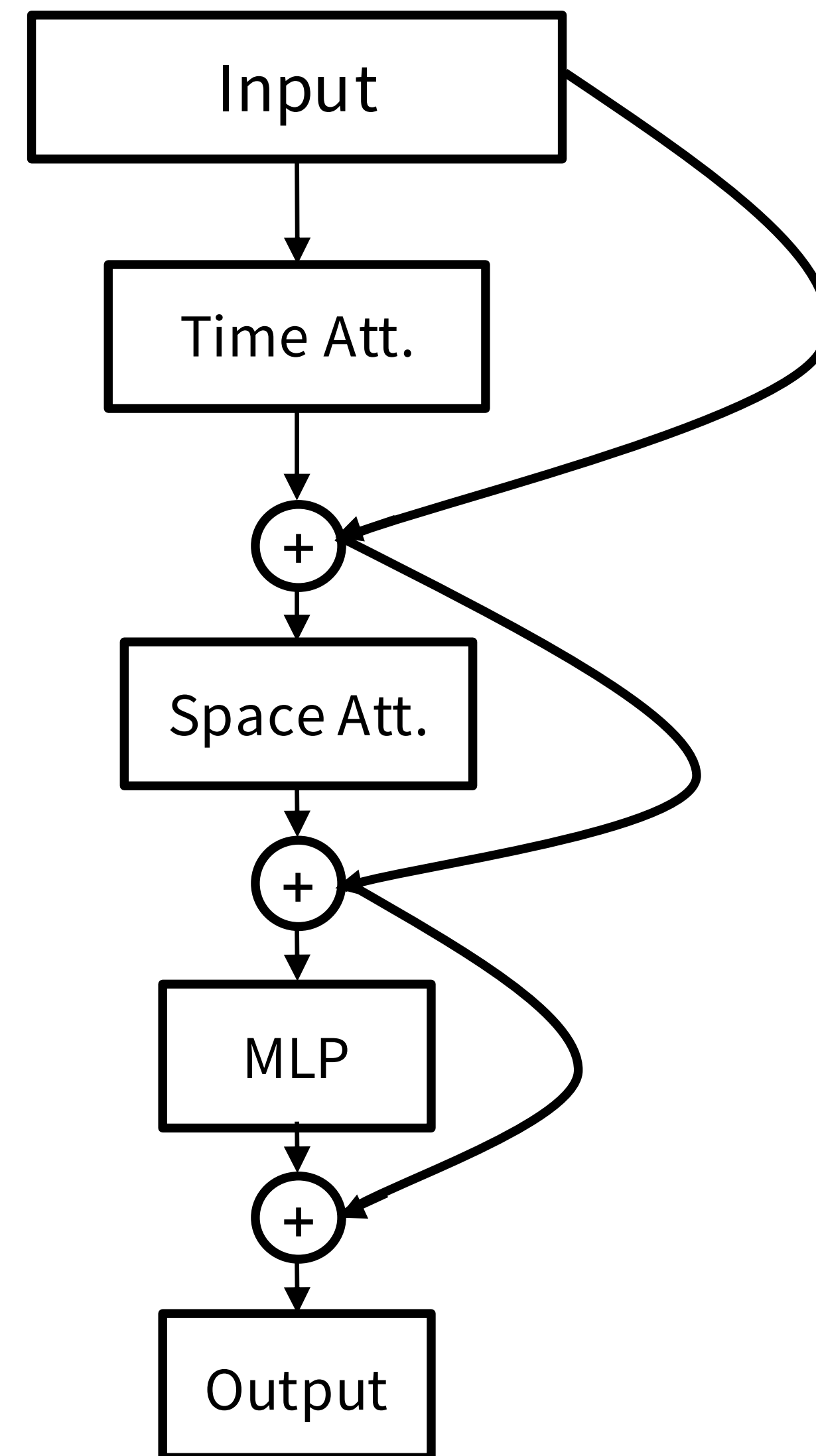
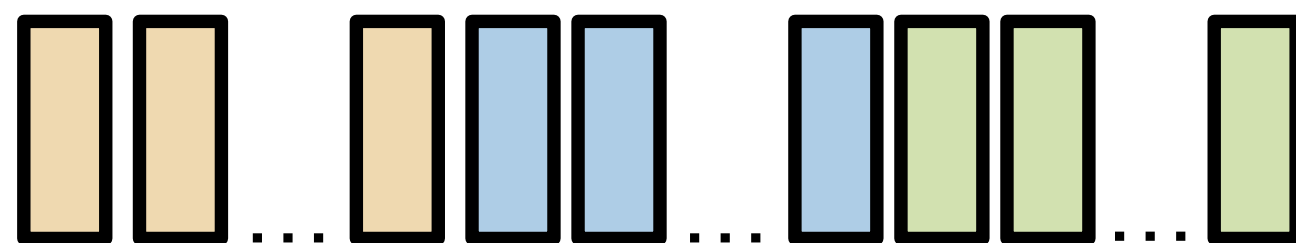
1. Reduces per-token computational cost from $O(NT)$ to $O(N + T)$
2. Information can still propagate across space/time after many blocks.



Divided Space-Time Attention (**T**+**S**)

Divided Space Time Attention

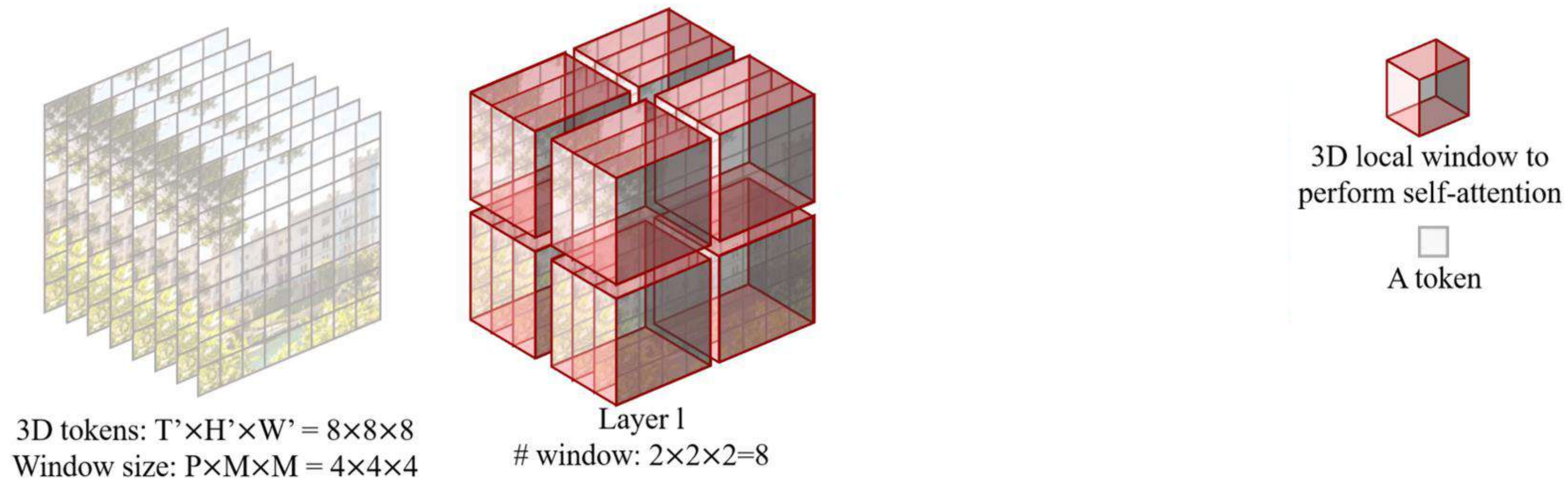
But, there are other efficient attention operators that operate over small sets of tokens!



Divided Space-Time Attention (**T**+**S**)

Approach #2: Video Swin Transformer

Key idea: Restrict self-attention to small local space-time cubes.
Very similar to 3D CNNs, but with self attention inside each cube.

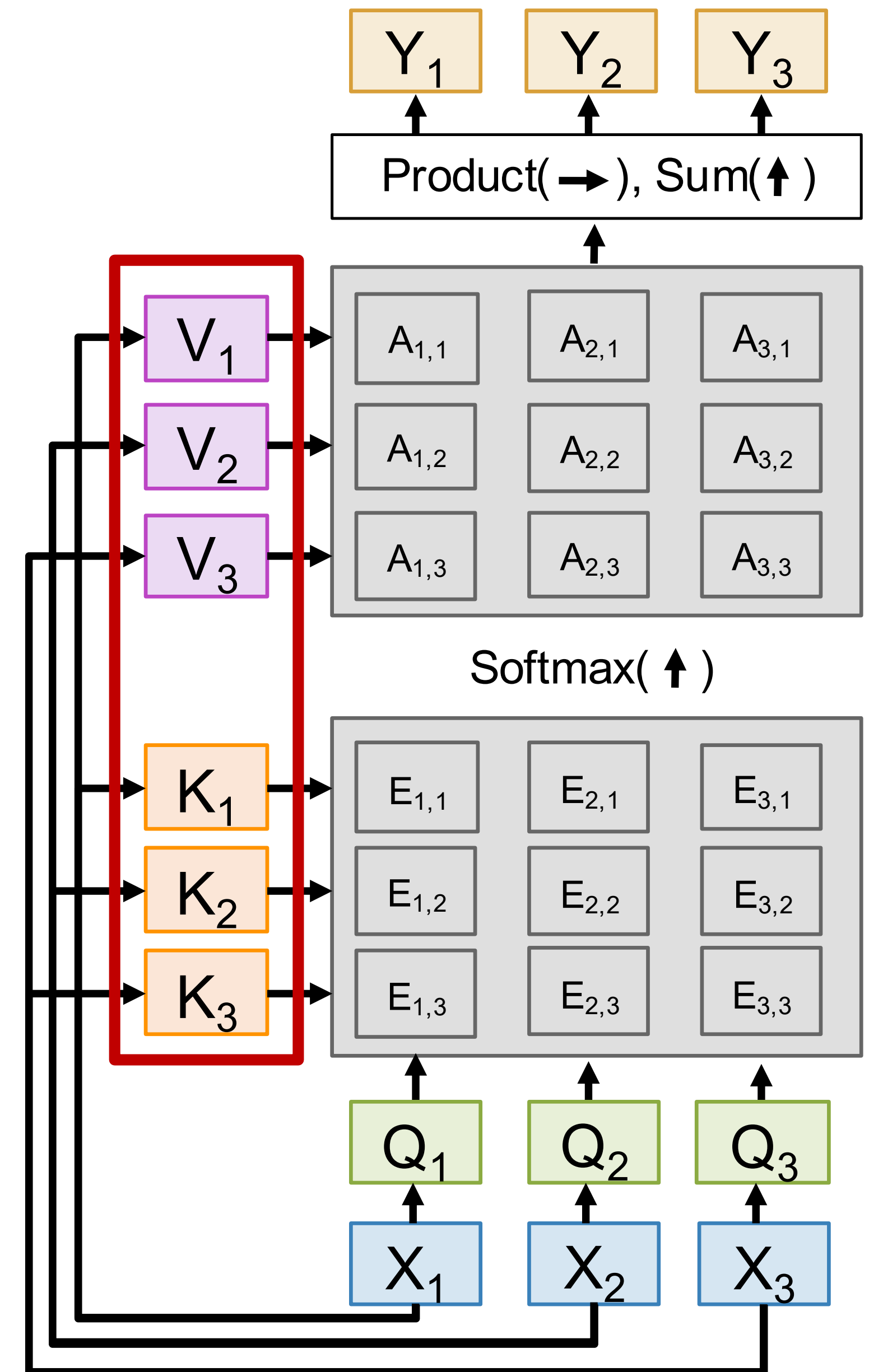


Approach #3: MViT

Multiscale Vision Transformers

Key idea: **Compress the size of K and V.**

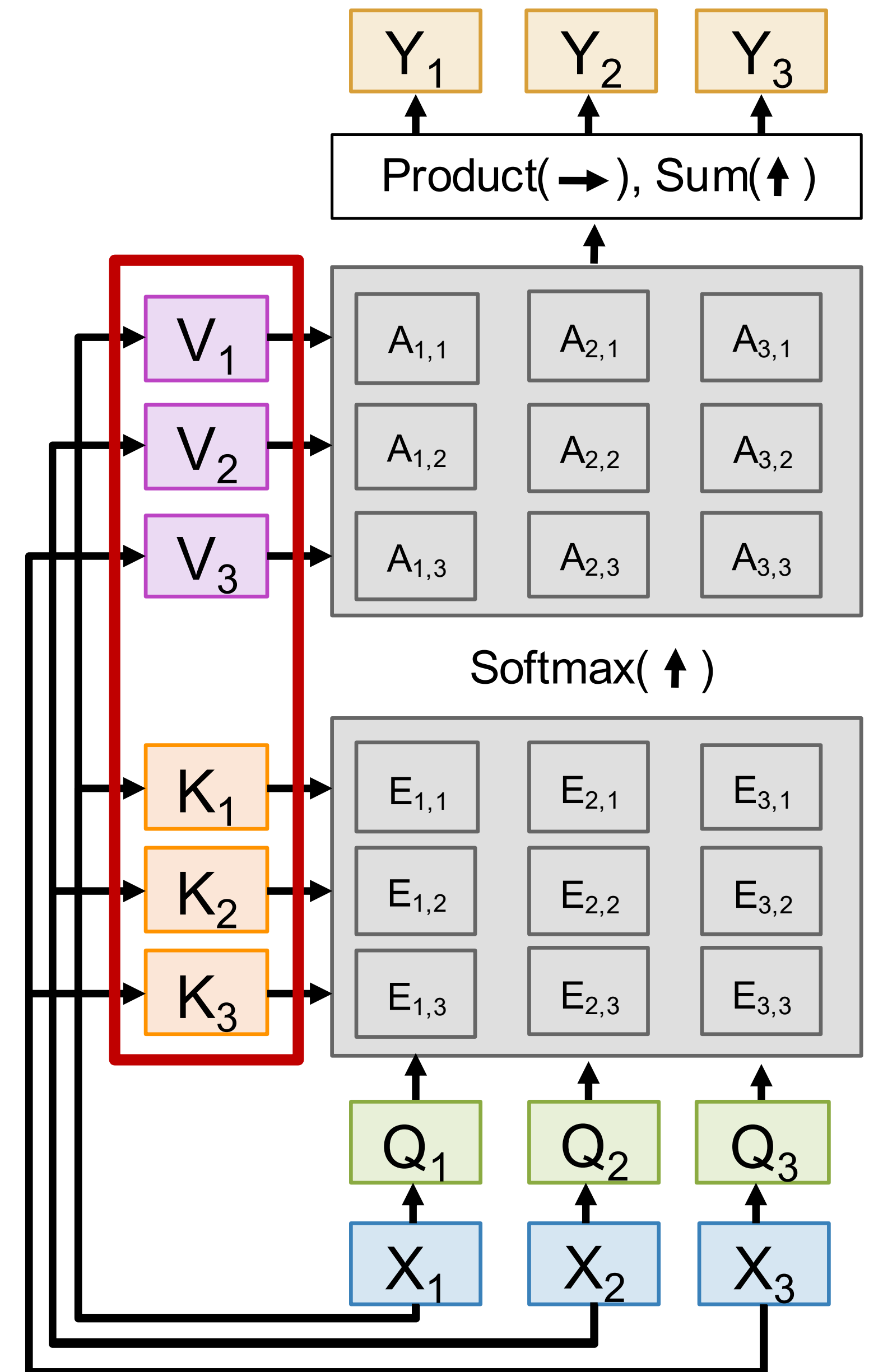
(Output length remains unchanged)



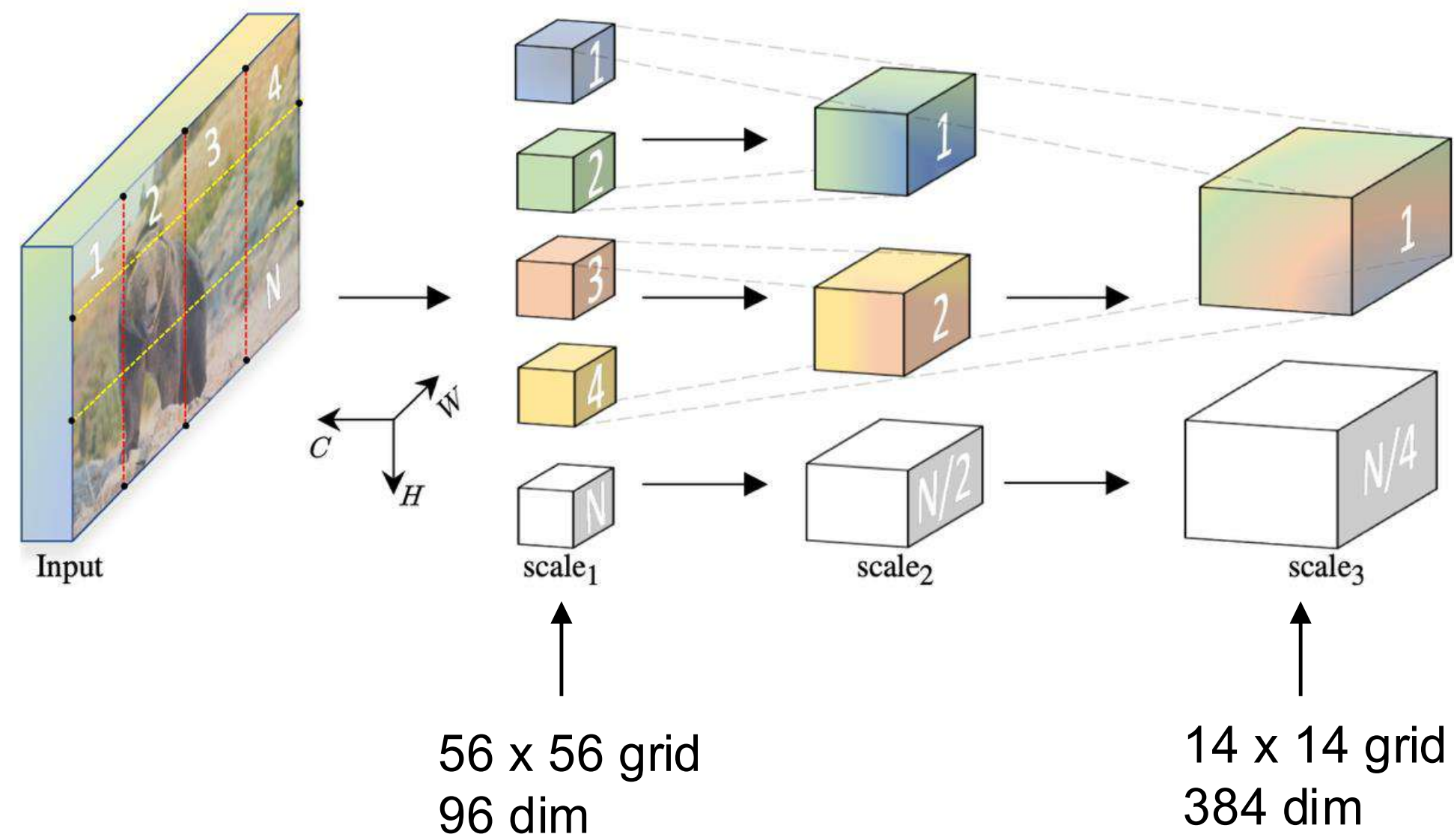
Multiscale Vision Transformers

Aggregate and shrink the K and V sequences
via convolution before computing attention.

56 x 56 grid of K/V vectors becomes 14 x 14 grid
(e.g., 4×4 kernel with stride 4)



Multiscale Vision Transformers

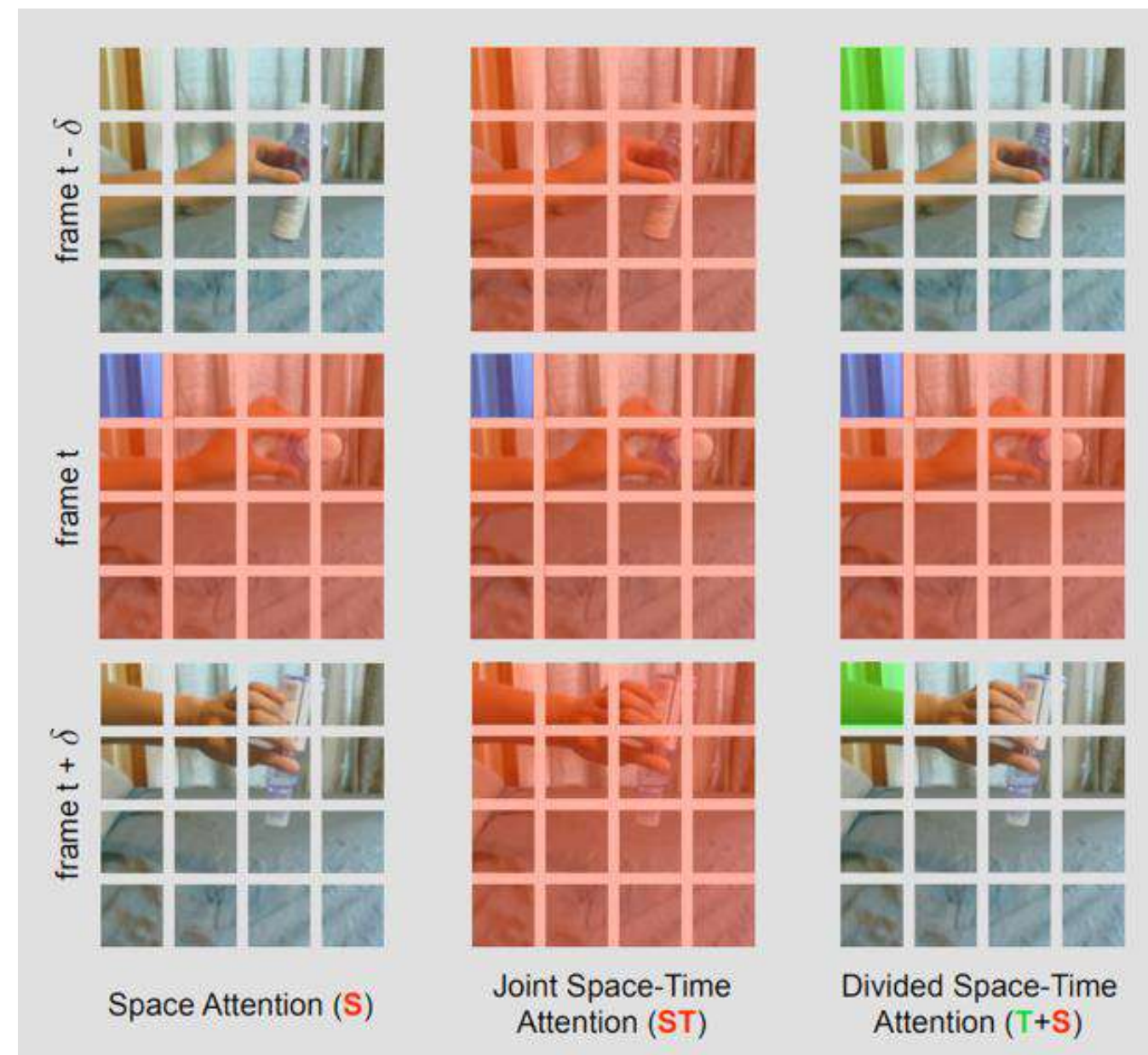


Progressively half spatial dimensions + double channels (same as ResNets)

Figure adapted from: Fan et al., "Multiscale Vision Transformers," ICCV 2021.

Two Broad Strategies for Improving Video Efficiency

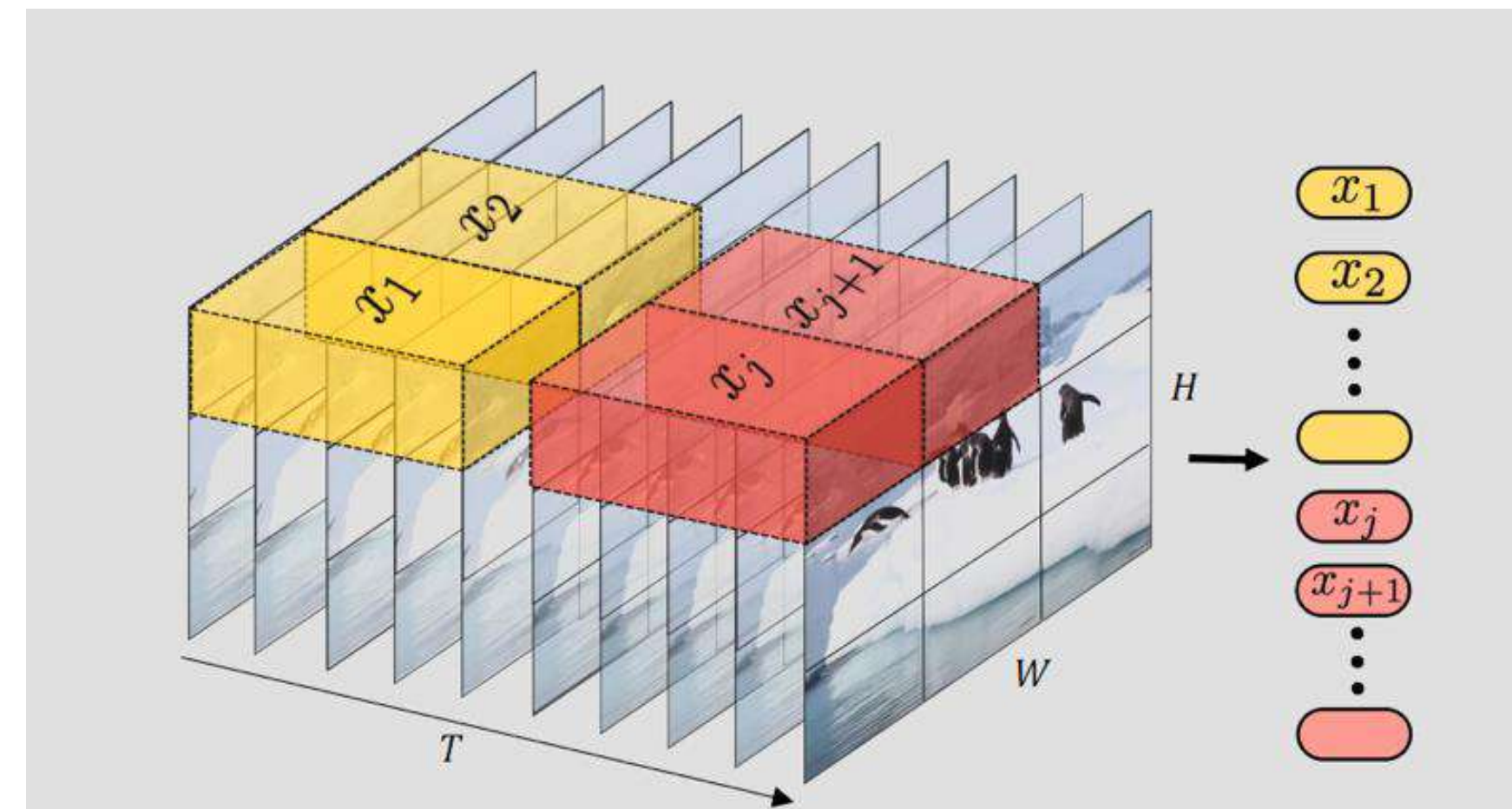
Modify attention operator



Bertasius et al, "Is Space-Time Attention All You Need for Video Understanding?", ICML 2021

Liu, Ze, et al. "Video Swin Transformer" CVPR 2022.

Reduce the number of tokens



Arnab et al, "ViViT: A Video Vision Transformer", ICCV 2021

Fan et al, "Multiscale Vision Transformers", ICCV 2021

Li et al, "MViTv2: Improved Multiscale Vision Transformers for Classification and Detection", CVPR 2022

How to reduce the number of tokens?

One solution: Tubelets

More efficient than patches!

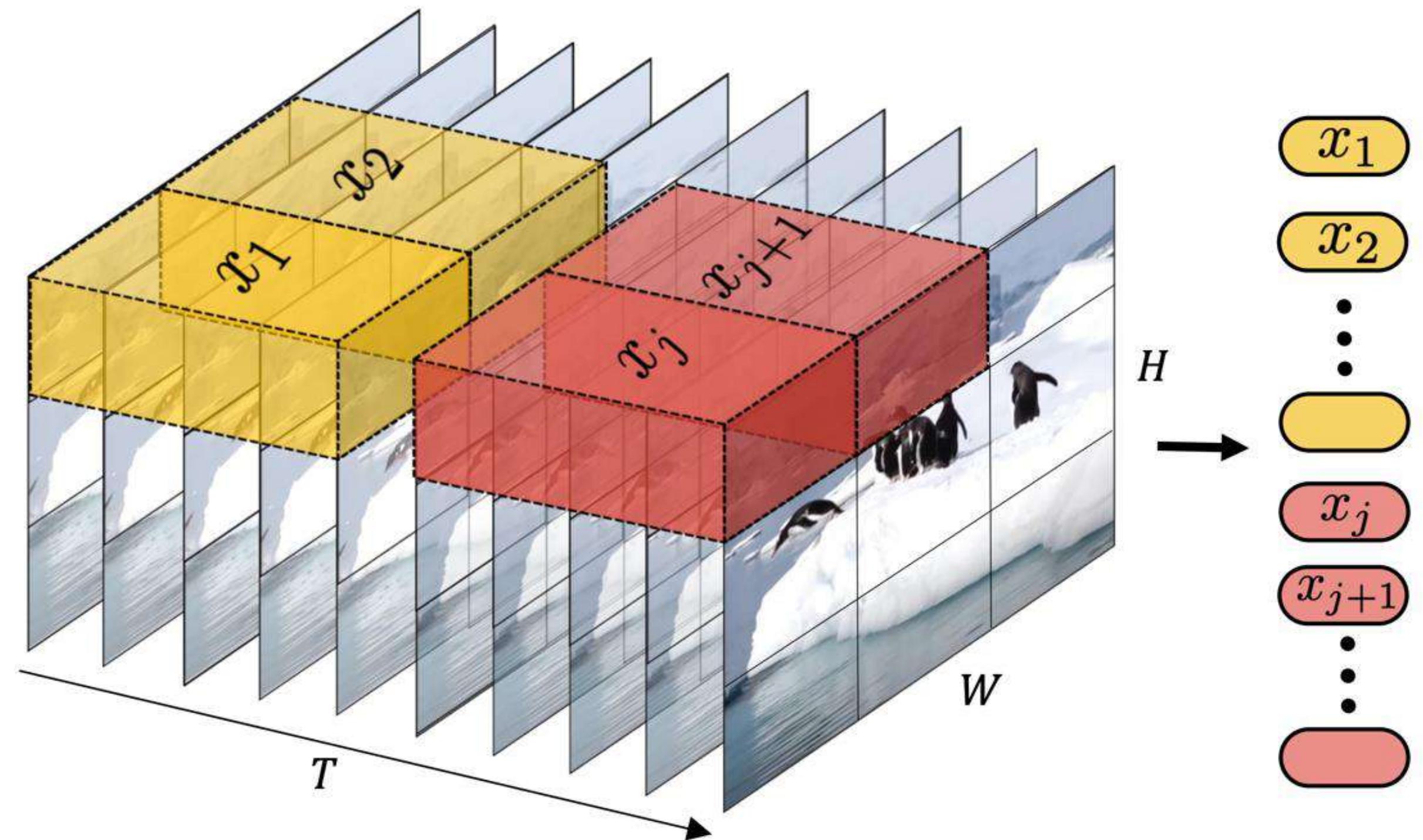


Figure from: Arnab et al, "ViViT: A Video Vision Transformer", ICCV 2021

How to reduce the number of tokens?

One solution: Tubelets

More efficient than patches!

Some intuition:

- Patches do not contain motion information, but tubelets do
- Significantly fewer tokens than patches!

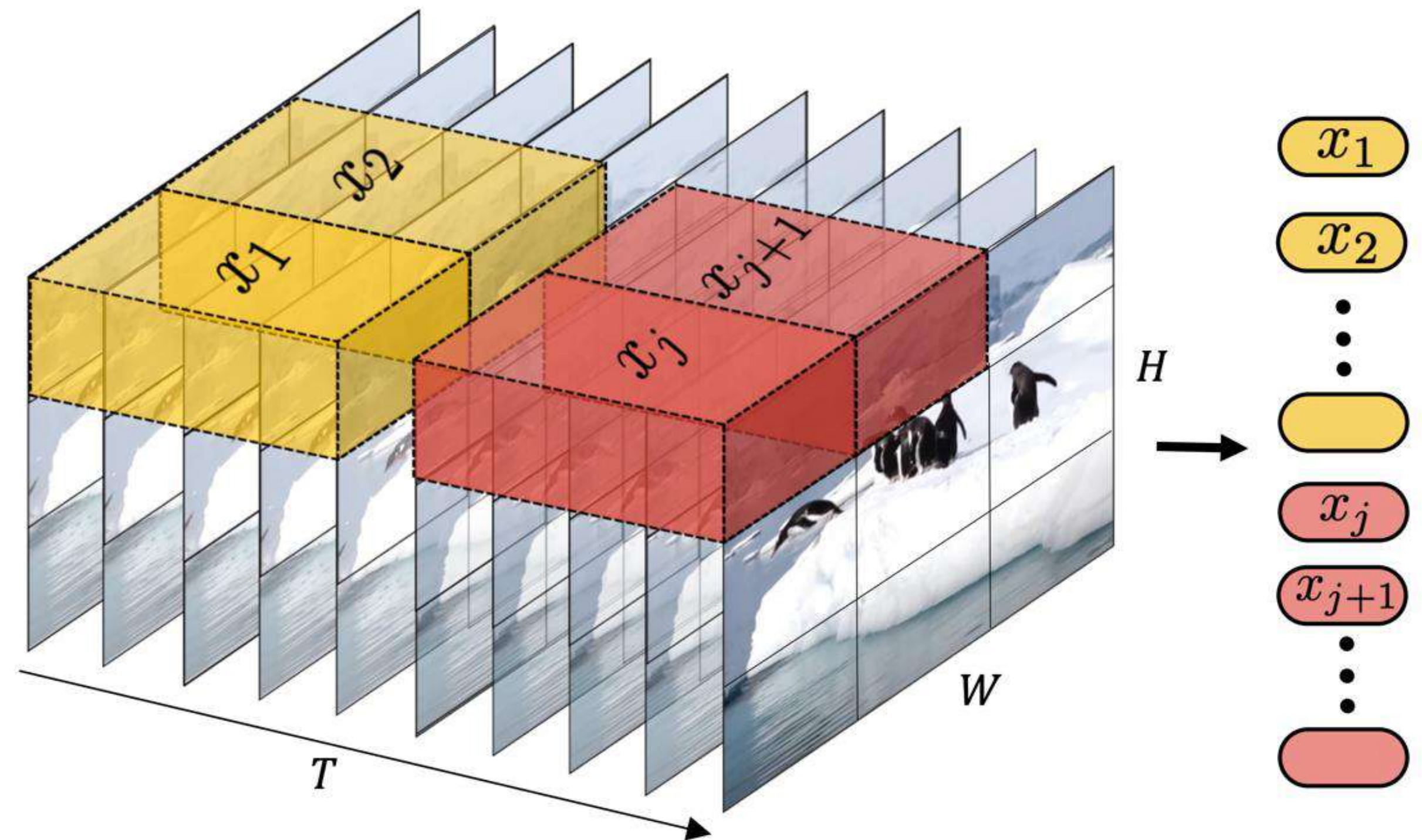


Figure from: Arnab et al, "ViViT: A Video Vision Transformer", ICCV 2021

How to reduce the number of tokens?

One solution: Tubelets

More efficient than patches!

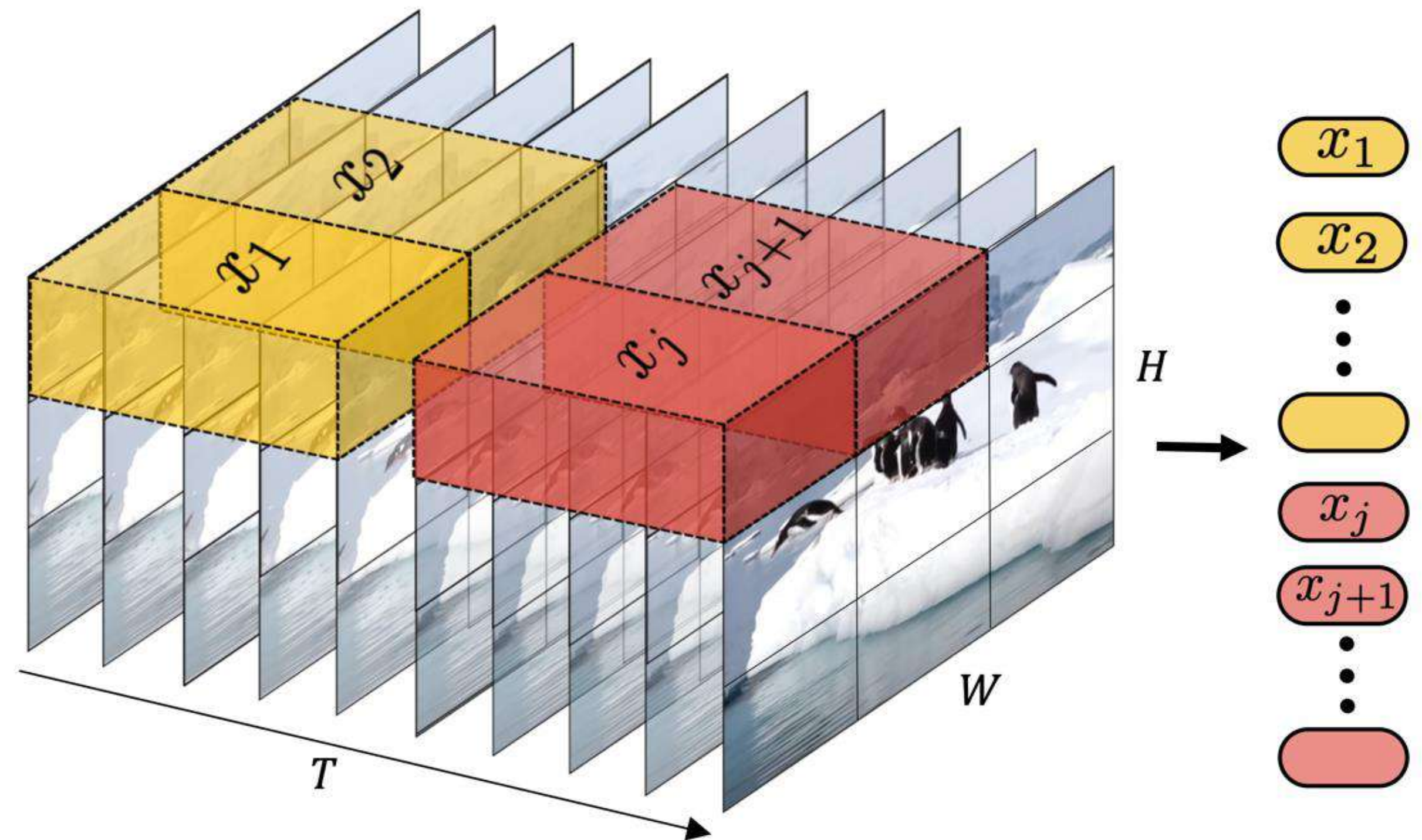
Some intuition:

- Patches do not contain motion information, but tubelets do
- Significantly fewer tokens than patches!

Tubelets over 4 frames:

4x reduction in tokens $\rightarrow$ 16x reduction in compute

Still project from tubelet dim to the transformer dim.



How to reduce the number of tokens?

Employed by *tons* of video understanding transformers (e.g. ViViT, VideoMAE, Video Swin, MViT, V-JEPA)

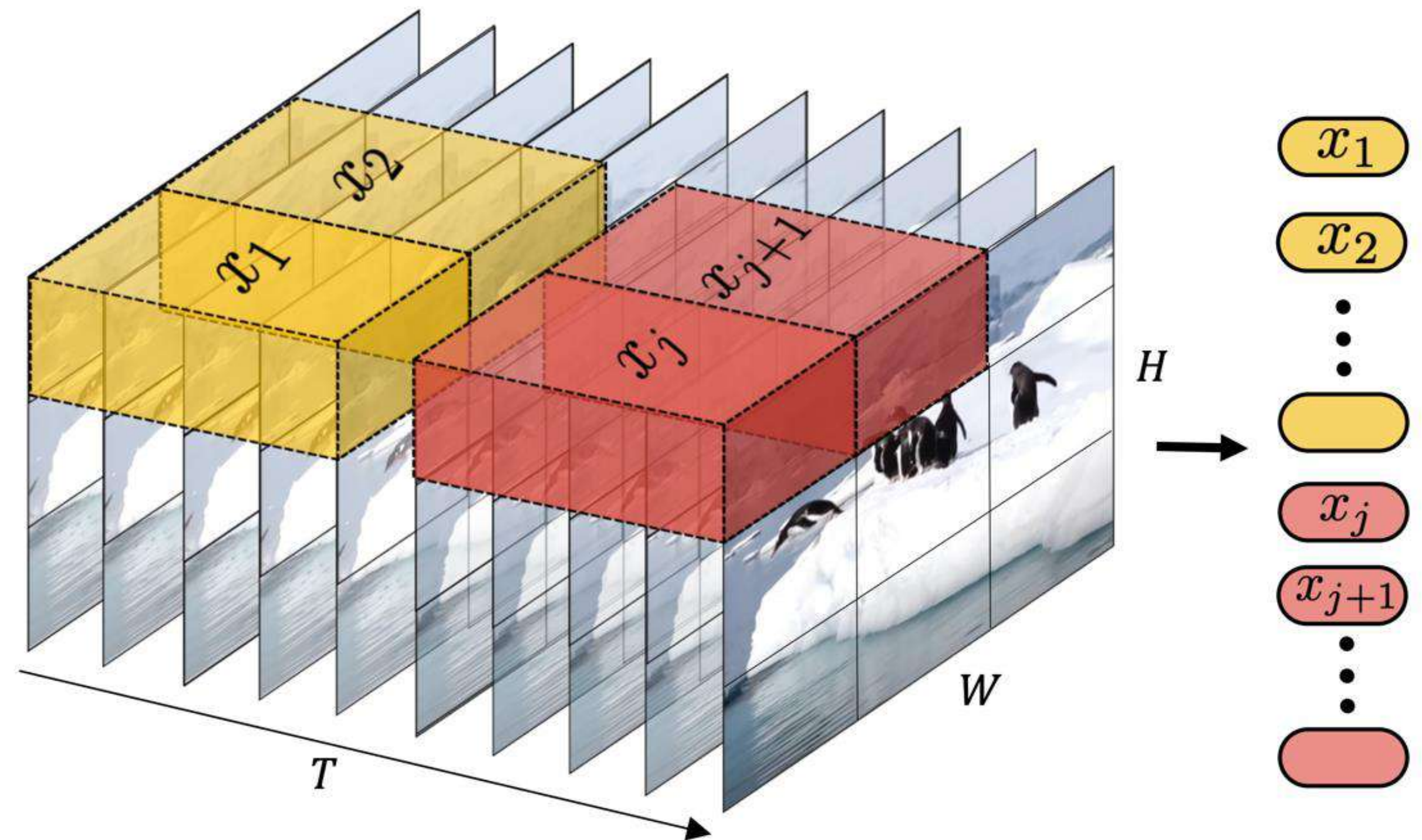


Figure from: Arnab et al, "ViViT: A Video Vision Transformer", ICCV 2021

How to reduce the number of tokens?

Lots of other approaches to reduce token count

(e.g. adaptive token selection, token merging, learned compression, etc.)

Will discuss some in the lecture 16!

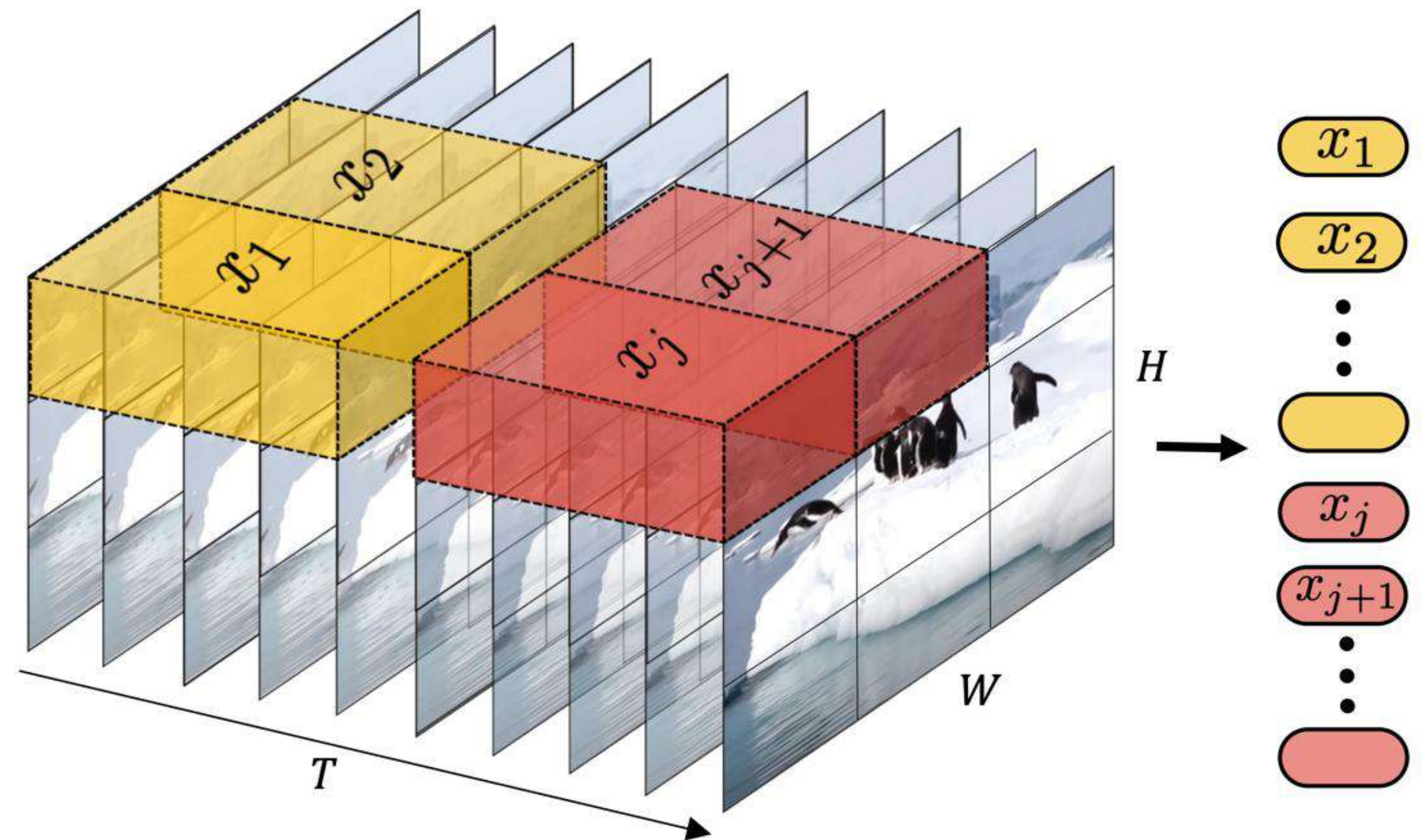


Figure from: Arnab et al, "ViViT: A Video Vision Transformer", ICCV 2021

So far: Classify short clips



Videos: Recognize actions



Swimming
Running
Jumping
Eating
Standing

Temporal Action Localization

Given a long untrimmed video sequence, identify frames corresponding to different actions

Running

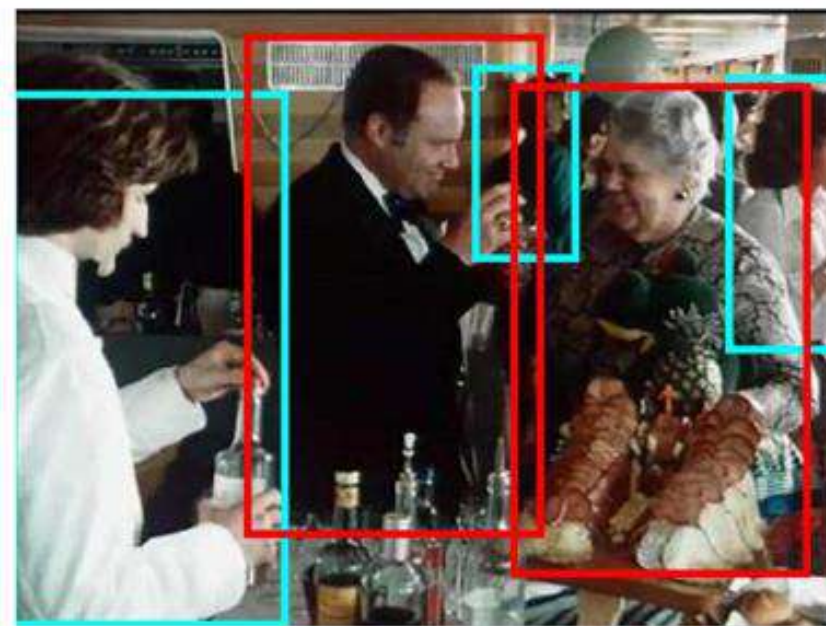
Jumping



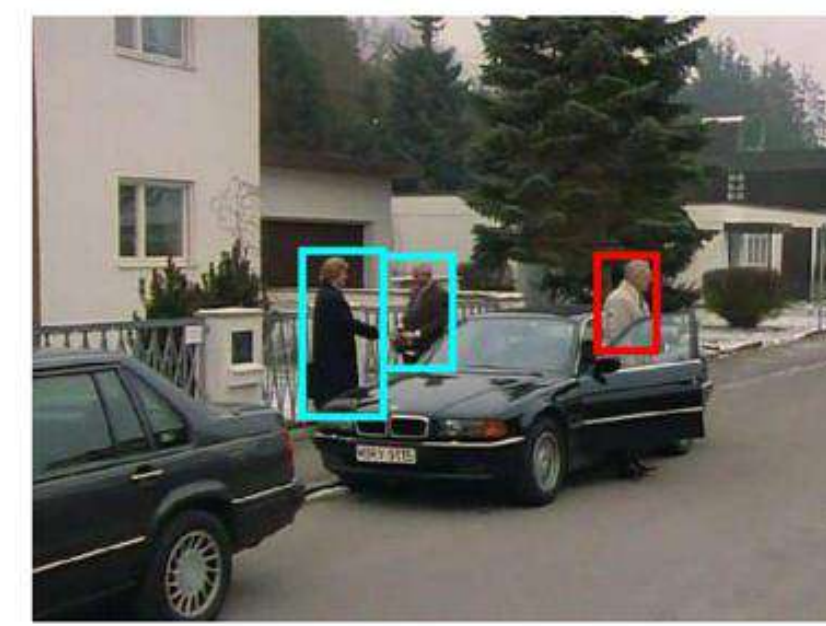
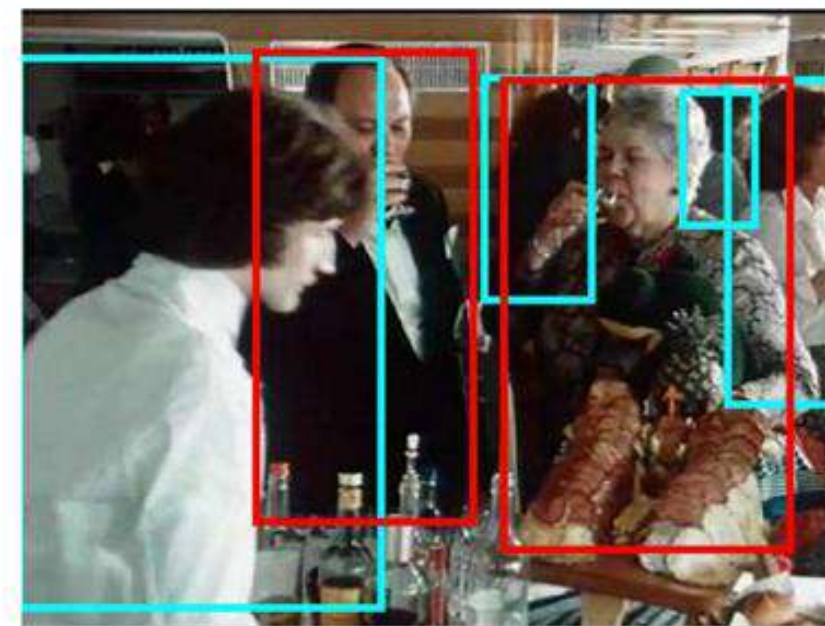
Can use architectures similar to Faster R-CNN: first generate temporal proposals then classify

Spatio-Temporal Detection

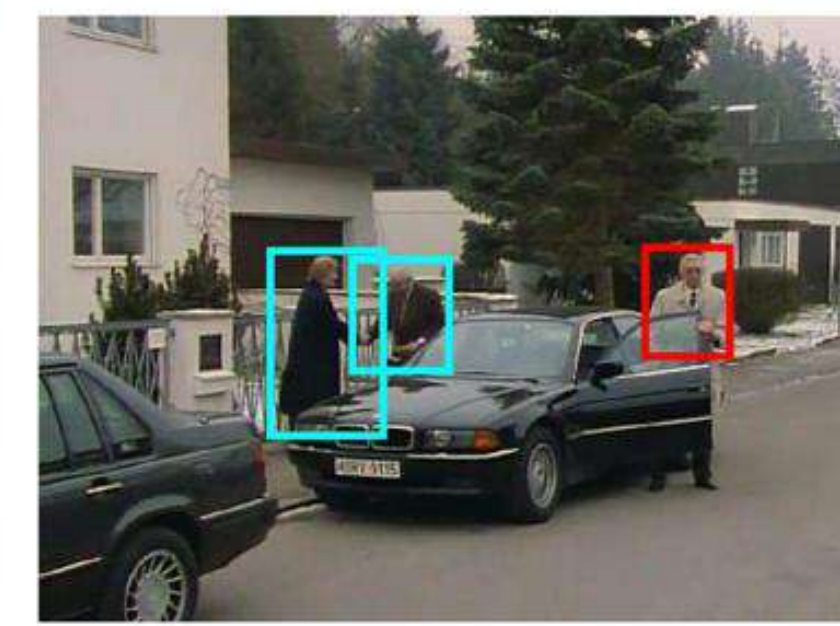
Given a long untrimmed video, detect all the people in both space and time and classify the activities they are performing.
Some examples from AVA Dataset:



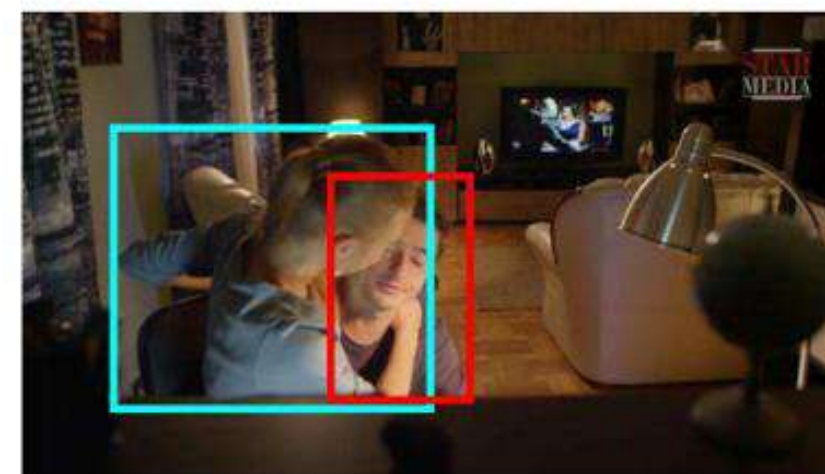
clink glass → drink



open → close



grab (a person) → hug



look at phone → answer phone



Beyond Short Video Streams

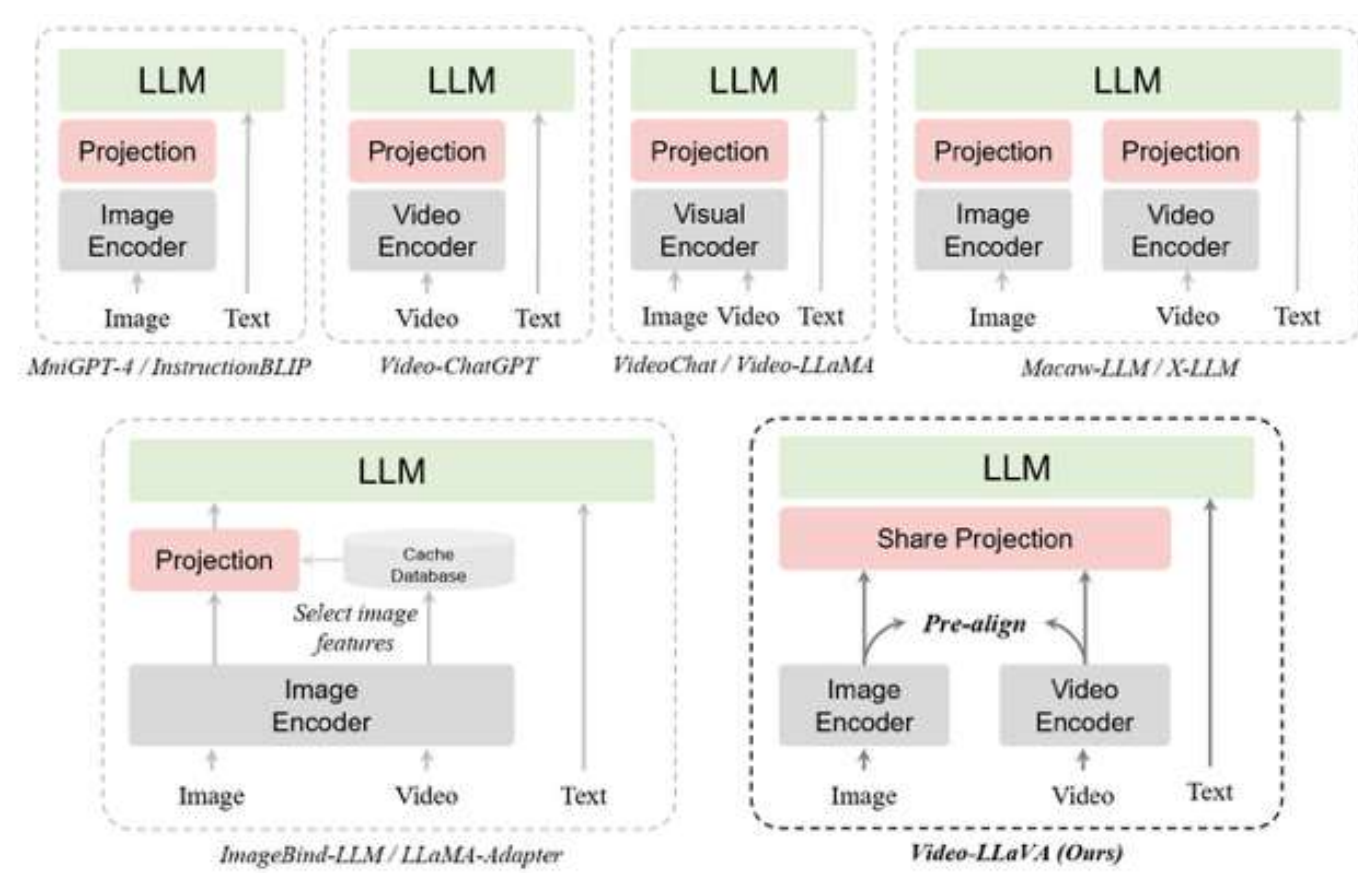
...



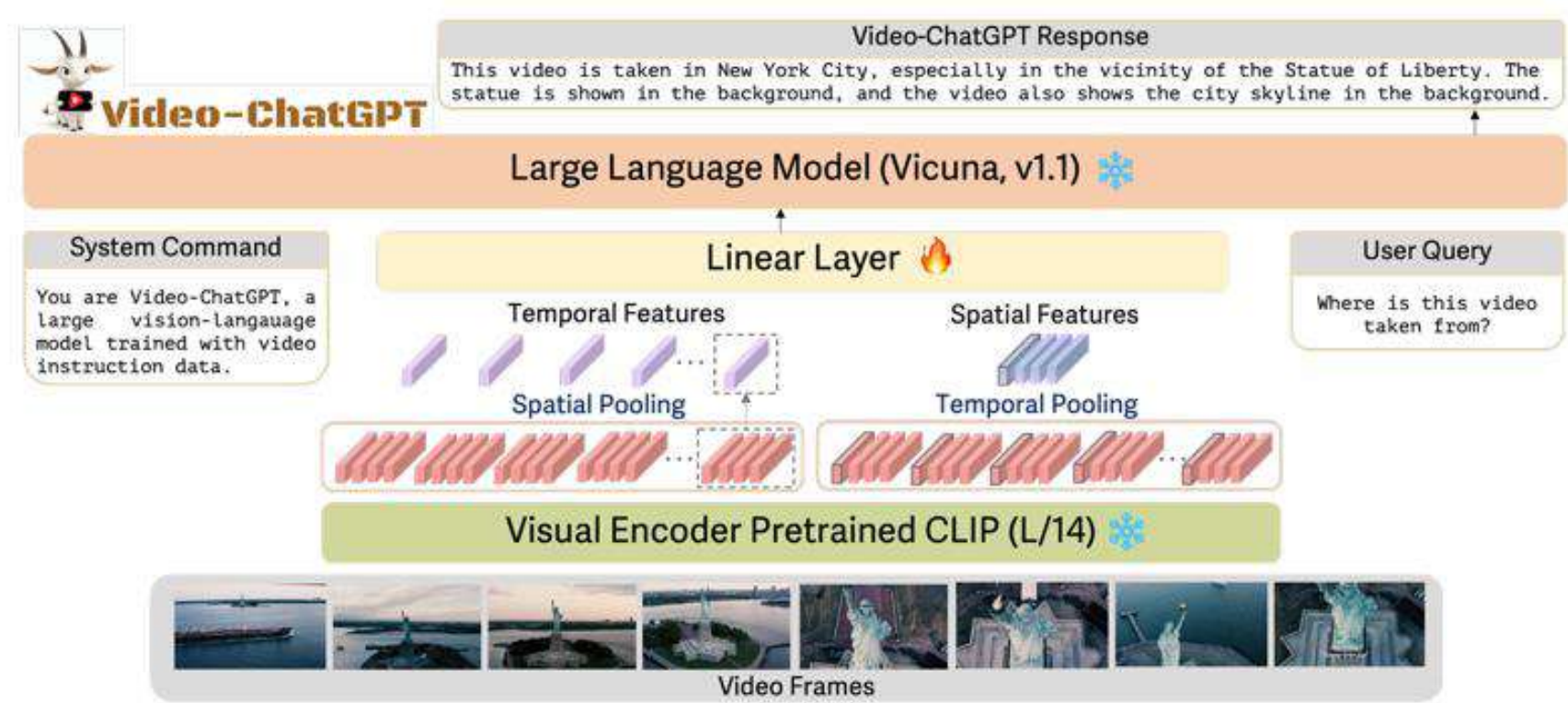
...

Audio, Language, and Long-form Video Understanding...

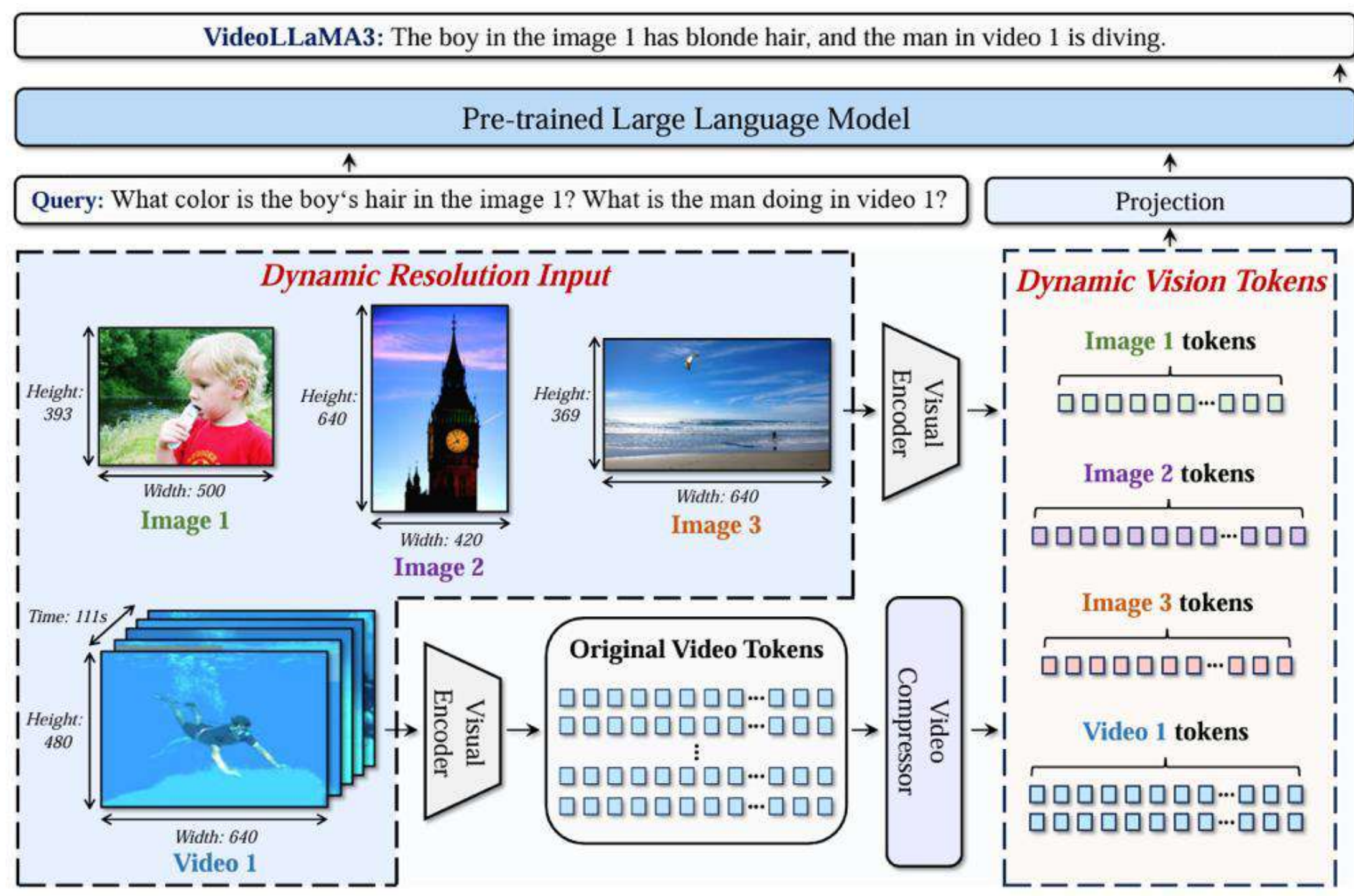
Video Language Understanding (VideoLLMs)



Video-LLaVA: Learning United Visual Representations by Alignment Before Projection. Lin et al. EMNLP 2024



Video-ChatGPT: Towards Detailed Video Understanding via Large Vision and Language Models. Maaz et al. ACL 2024.

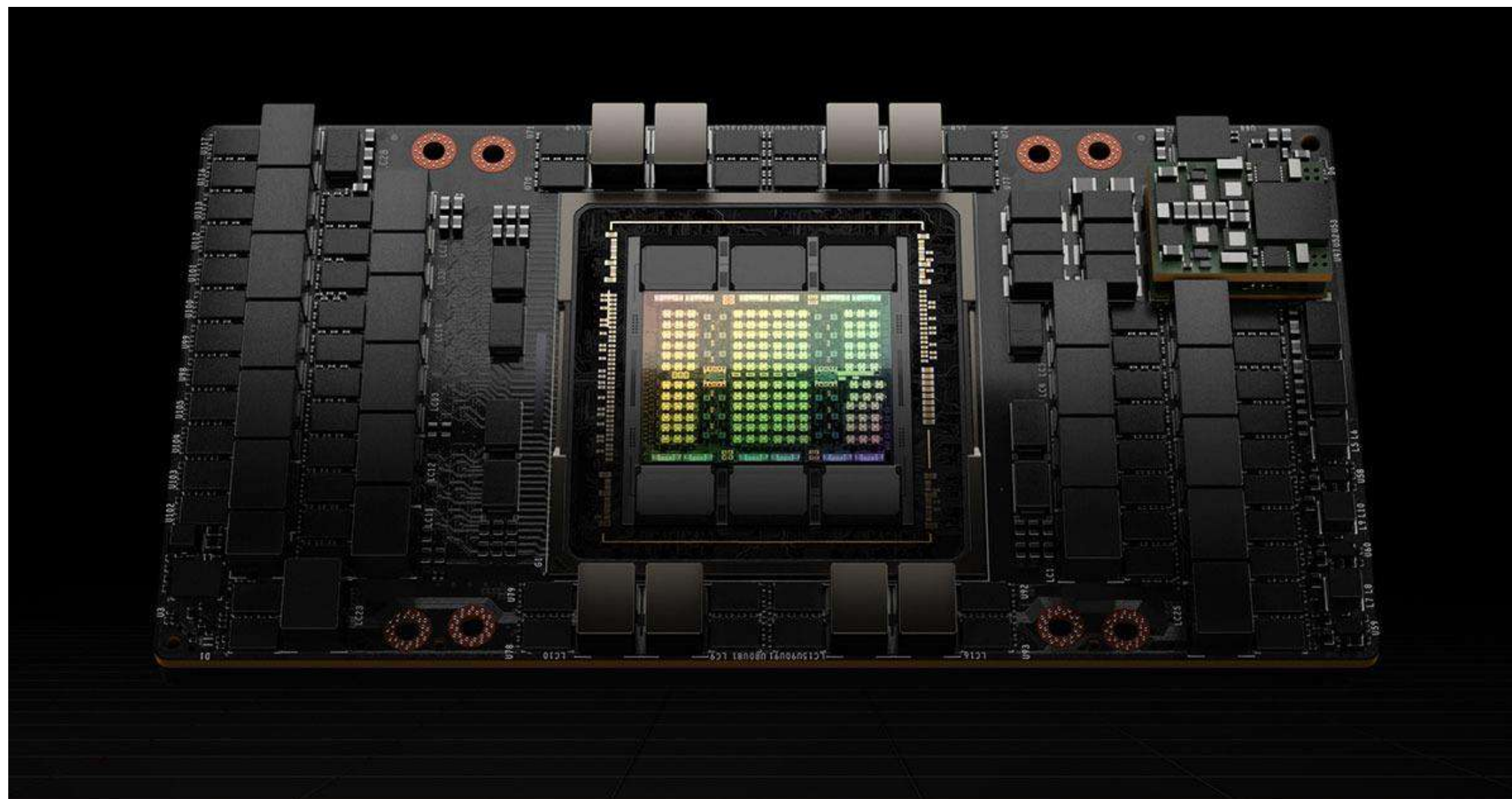


VideoLLaMA 3: Frontier Multimodal Foundation Models for Video Understanding. Zhang et al. arXiv 2025

Next Lecture: Large Scale Distributed Training

GPUs and How to Train On Them

A bit about GPU hardware



How to train on lots of GPUs

